

C++

top

КОМПЬЮТЕРНАЯ
АКАДЕМИЯ

</>

C++

ПРОГРАММИРОВАНИЕ С ИСПОЛЬЗОВАНИЕМ ТЕХНОЛОГИИ



J A V A

Урок № 8

Введение в Spring

Материалы урока прикреплены к данному PDF-файлу. Для доступа к материалам, урок необходимо открыть в программе Adobe Acrobat Reader.

Содержание

История возникновения	7
Что такое Spring?	9
Цели и задачи Spring.....	16
Архитектура Spring	19
Контейнер компонентов.....	23
Понятие контекста приложения	25
Bootstrap Spring Framework.....	29
Конфигурация Spring Framework.....	34
Создание конфигурации XML	36
Создание гибридной конфигурации	41
Настройка Spring с помощью Java используя @Configuration	51
Использование профилей определения Bean.....	61
Аннотация @requestmapping	70
Ограничения URL.....	71
Ограничения методов HTTP-запроса.....	75
Ограничения параметров запроса.....	77
Ограничения заголовка запроса	78
Ограничения типа контента.....	78
Определение параметров метода контроллера	79
Аннотированные свойства запроса	80
Объекты формы ввода.....	86
Преобразования тела запроса и сущностей.....	88
Типы моделей	94

Типы представлений	95
Сущности тела ответа	95
Spring MVC. Архитектура Spring MVC	99
Использование явных представлений и имен представлений.....	101
Использование переадресации представления	101
Конфигурация отображения представления	103
Создание представления JSP.....	103
Использование неявных представлений с атрибутами модели	105
Конфигурация преобразования имен представлений	105
Использование @ModelAttribute.....	106
Возврат объектов ответа	107
Настройка конвертеров сообщений	108
Настройка согласования контента.....	112
Использование @ResponseBody	115
Преимущества использования объектов формы	117
Добавление объектов формы в модель.....	119
Использование Spring Framework <form> Tags.....	120
Получение предоставленных данных формы.....	124
Примеры использования Spring MVC и объектов формы	126
Расширение Model-View-Controller при помощи Controler-Service-Repository	141
Определение различных типов логики программы...	142

Репозитории обеспечивают логику сохранения	145
Сервис обеспечивают бизнес-логику.....	146
Контроллеры обеспечивают логику пользовательского интерфейса	147
Использование корневого контекста приложения вместо контекста веб-приложения	150
Повторное использование контекста корневого приложения для нескольких пользовательских интерфейсов.....	151
Перемещение бизнес-логики с контроллеров на сервисы.....	155
Использование репозитория для хранения данных	162
Понятие веб-сервисов REST и SOAP	174
Что такое SOAP?.....	175
Что такое REST?.....	178
Понятие открытости.....	181
Использование URL-адресов, методов и кодов состояния HTTP.....	186
OPTIONS	187
HEAD и GET	189
POST	190
PUT	191
PATCH.....	192
DELETE	193
Настройка и тестирование веб-сервисов RESTful с помощью Spring MVC	194

Разделение контроллеров с стереотипными аннотациями	195
Создание отдельных контекстов приложений Web и REST	197
Ошибки обработки условий в веб-сервисах RESTful	201
Изменение кодов ответа HTTP	202
Объявление обработчиков исключений.....	202
Использование шаблона рекомендаций контроллера	203
Сопоставление запросов RESTful с методами контроллера	207
Улучшение обнаружения с конечной точкой индекса	215
Тестирование конечных точек веб-службы.....	218
Выбор инструмента тестирования.....	219
Выполнение запросов к Вашему веб-сервису.....	220
Использование Spring-сервисов для SOAP	225
Controler-Service-Repository	226
Написание контракта — первый XSD и WSDL	237
Добавление конфигурации SOAP-диспетчера	242
Создание конечной точки SOAP	247
Spring Boot.....	254
Что такое Spring Boot? Цели и задачи Spring Boot	254
Примеры использования Spring Boot.....	257
Домашнее задание.....	261

История возникновения

В 1996 году Java являлся еще молодой и перспективной платформой. Многие разработчики перешли на Java, для того чтобы создавать богатые и динамические веб-приложения с помощью апплетов. Затем им стало ясно, что в отличие от других языков, на Java можно писать сложные приложения, которые состоят из отдельных частей.

В декабре того же года вышла спецификация JavaBeans 1.00-A, которая определила модель программного компонента для Java. Эта спецификация обусловила приемы программирования позволяющие многократно использовать простые объекты Java и с легкостью компоновать их в более крупные приложения, но в основном JavaBeans использовались в качестве модели для создания элементов пользовательского интерфейса. Промышленным разработчикам казалось, что эти элементы слишком простые для сложных задач и им требовалось нечто большее.

Для сложных приложений зачастую требуются такие сервисы, как поддержка транзакций, безопасность и распределенные вычислительные службы, которые напрямую не предоставляются спецификацией JavaBeans. И в марте 1998 года Sun Microsystems опубликовала версию 1.0 спецификации *Enterprise JavaBeans* (EJB). Эта спецификация расширила понятие серверных Java компонентов и предоставила необходимые бизнес-услуги, но не смогла поддерживать простоту исходной спецификации JavaBean. Кроме названия, EJB мало похож на исходную спецификацию JavaBeans.

Несмотря на то, что многие успешные приложения были построены на базе EJB, EJB не достигла своей цели: упростить разработку промышленных приложений. Стоит отметить, что декларативное программирование EJB упрощает многие инфраструктурные аспекты разработки, такие как транзакции и безопасность, но при этом усложняет разработку, тем что требует создание дескрипторов развертывания и использование шаблонного кода. Со временем многие разработчики разочаровались в EJB и его популярность снизилась.

Сегодня процесс разработки Java-компонентов вернулся к своим истокам. Новые технологии программирования, в том числе аспектно-ориентированное программирование (AOP) и внедрение зависимостей (DI), дают JavaBeans большие возможности, ранее заявленные в EJB. Эти технологии предоставляют простые Java-объекты (POJO) как модель декларативного программирования, схожую с EJB, но более упрощенную. Теперь не понадобится написание громоздких компонентов EJB, так как достаточно простого JavaBean.

На сегодняшний день Spring Framework — является ведущей технологией разработки приложений на основе POJO. POJO расшифровывается как «*Plain Old Java Object*» — это чистая структура данных, которая имеет поля с геттерами и, возможно, сеттерами и может переопределять некоторые методы из **Object** (например, `equals`) или некоторый другой интерфейс, например `Serializable`, но не имеет своего собственного поведения. Это эквивалент структуры в языке C.

Что такое Spring?

Хотя греческий философ Гераклит не был известен как разработчик программного обеспечения, он, похоже, хорошо разбирался в этом вопросе. Он говорил: «Перемены — это единственная постоянная вещь». В этом заявлении фиксируется основополагающая истина разработки программного обеспечения.

То, как мы разрабатываем приложения сегодня, отличается от того, что было год назад, 5 лет назад, 10 лет назад, и, конечно, 15 лет назад, когда первоначальная форма Spring Framework была введена в книге его создателя Рода Джонсона «Expert One-on-One J2EE Design and Development». Spring Framework свободно распространяемый фреймворк, который был создан с целью упростить разработку корпоративных приложений и сделать возможным использование простых компонентов JavaBean.

В начале 2000-х годов наиболее распространенными типами приложений были веб-приложения на основе браузера, поддерживаемые реляционными базами данных. Хотя этот тип разработки по-прежнему актуален и Spring хорошо подходит для таких приложений, сегодня более популярными становятся приложения, состоящие из микросервисов, предназначенных для облака, которые сохраняют данные в различных базах данных. И новый интерес к реактивному программированию направлен на обеспечение большей масштабируемости и повышение производительности.

Независимо от того, разрабатываете ли Вы простое веб-приложение с поддержкой базы данных или создаете современное приложение, основанное на микросервисах, Spring — это основа, которая поможет Вам достичь Ваших целей. Этот урок является Вашим первым шагом в путешествии по современной разработке приложений со Spring.

Любое нетривиальное приложение состоит из множества компонентов, каждый из которых несет ответственность за свою часть общей функциональности приложения, взаимодействуя с другими элементами. Когда приложение запускается, эти компоненты каким-то образом необходимо создавать и настраивать их взаимодействие.

По своей сути Spring предлагает контейнер, который часто называют контекстом приложения Spring, который создает и управляет компонентами приложения. Эти компоненты или *beans* соединены вместе в контексте приложения Spring, чтобы создать полноценное приложение, подобно тому как кирпичи, раствор, древесина, гвозди, сантехника и проводка, соединены вместе, чтобы построить дом.

Действие соединительных компонентов вместе основано на шаблоне, известном как внедрение зависимостей (*dependency injection*). Вместо того, чтобы компоненты создавали и поддерживали жизненный цикл других компонентов, от которых они зависят, приложение, зависящее от зависимостей, полагается на отдельный объект (контейнер) для создания и поддержки всех компонентов и внедрения их в нужные им компоненты. Это делается обычно с помощью аргументов конструктора или методов доступа к свойствам.

Например, предположим, что среди многих компонентов некоего учетного приложения есть два, которые взаимодействуют: *Inventory service* (для *контроля запасов*) и *Product service* (для *предоставления базовой информации о продукте*). Продукт зависит от инвентаризации, чтобы иметь возможность предоставлять полный набор информации о продуктах. Рисунок иллюстрирует отношения между этими компонентами и контекстом приложения Spring.

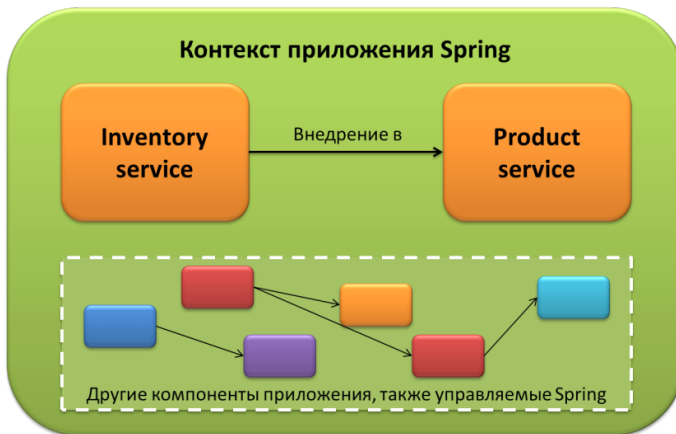


Рисунок 1.

Исторически сложилось так, что можно использовать контекст приложения Spring для объединения beans вместе с одним или несколькими XML-файлами, которые описывают компоненты и их отношение к другим компонентам. Например, следующий XML объявляет два компонента, компоненты `InventoryService` и `ProductService` и связывает `InventoryService` с `ProductService` с помощью аргумента конструктора:

```

<bean id="inventoryService"
      class="com.academy.InventoryService" />
<bean id="productService"
      class="com.academy.ProductService" >
    <constructor-arg ref="inventoryService" />
</bean>

```

Аспектно-ориентированное программирование часто определяется как метод, который способствует разделению задач в программной системе. Системы состоят из нескольких компонентов, каждый из которых несет ответственность за определенную функциональность. К тому же эти компоненты зачастую также несут дополнительную ответственность. Системные службы, такие как логгирование, управление транзакциями или безопасность, часто распределены в разнообразных частях программного кода.

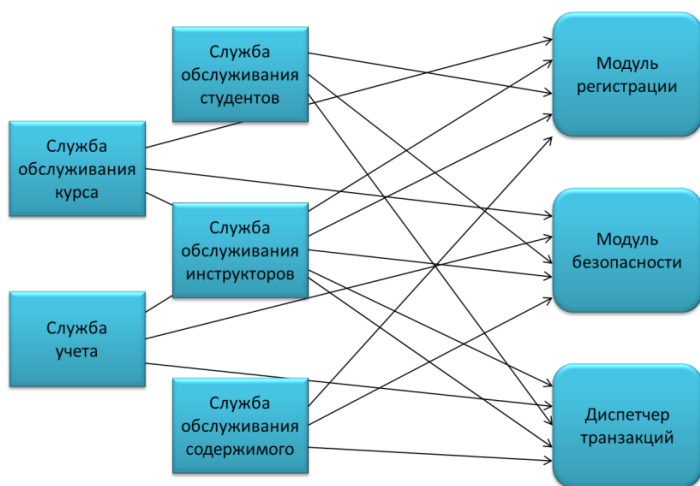


Рисунок 2.

Эти службы называются сквозными функциями, поскольку они могут пересекать другие компоненты в системе. Распространение этих функций в нескольких компонентах, усложняет программный код:

- код, который реализует системные задачи, дублируется по нескольким компонентам. Это означает, что, если нужно изменить логику работы системы, необходимо будет просмотреть все задействованные компоненты;
- Ваши компоненты загромождаются кодом, который не соответствует их основной функциональности. К примеру, метод добавления записи в адресную книгу должен заботиться только о том как добавить адрес, а не о его безопасности или транзакции.

На рисунке 2 проиллюстрирована эта сложность. Бизнес-объекты слева слишком тесно связаны с системными службами. Мало того, что каждый объект знает, что он регистрируется, защищен и участвует в транзакционном контексте, но и каждый объект отвечает за выполнение этих сервисов.

АОР позволяет выносить эти сервисы в модули, а затем применять их декларативно к необходимым компонентам. Это приводит к тому что компоненты становятся сосредоточенными на выполнении своих задач, и совершенно не осведомлены о других сервисах системы, в которые могут быть вовлечены. Короче говоря, аспекты гарантируют, что РОЮ остаются простыми.

Это позволяет рассматривать аспекты как слои, которые охватывают множество компонентов приложения, как показано на следующем рисунке. По своей сути

приложение состоит из модулей, которые реализуют бизнес-функциональность. Благодаря АОР можно обернуть основное приложение слоями функциональности. Эти слои могут применяться декларативно во всем приложении гибким образом, при этом ядро приложения может не знать, что они существуют. Это мощная концепция, поскольку позволяет убрать из ядра операции, связанные с управлением транзакциями, логгированием и безопасностью.



Рисунок 3.

Также в любом приложении обмен сообщениями является важной задачей, которая требует решения. Например, некоторым частям программы может потребоваться знать, когда другие ее части выполняют указанное действие. Часть программы, которая выполняет это действие может зависеть от частей, заинтересованных в этом действии, вызывает методы чтобы уведомить их.

Этот тип жесткой связи трудно поддерживать, и он может быстро стать беспорядочным.

Spring Framework обеспечивает слабосвязанную систему обмена сообщениями, в которой используется шаблон публикации-подписки. Компоненты в системе, подписываясь на определенные оповещения сообщают, что они заинтересованы в них, и инициаторы этих оповещений публикуют сообщения, не заботясь о том, кто подписался. По сути так работает Twitter. Люди, которые хотят рассказать что-то интересное — «твитят» это, и другие люди, которые интересуются этими твитами — подписаны на них. Используя Spring Framework, управляемый Spring компонент подписывается на определенный тип сообщения, реализуя общий интерфейс, а другие объекты, управляемые Spring, публикуют эти сообщения в Spring Framework, которые доставляют их подписчикам. Эта система может быть расширена и настроена для доставки сообщений через кластер приложений.

Помимо основного контейнера, Spring предлагает веб-фреймворк, множество вариантов сохранения данных, структуру безопасности, интеграцию с другими системами, мониторинг времени выполнения, поддержку микросервиса, модель реактивного программирования и многое другое необходимое для современной разработки приложений.

Цели и задачи Spring

Вспомним сервлеты, которые Вы создавали на предыдущих уроках. Каждый из них имел хотя бы один метод `doGet` или `doPost`, содержащий множество операторов `if` или `switch`, ответственных за маршрутизацию запроса на любое количество методов в сервлет. В значительно более сложном корпоративном приложении этот шаблон становится неуправляемым и его чрезвычайно трудно тестировать. Например, сервлет, обрабатывающий пользовательские профили, может иметь десятки методов, каждый из которых имеет другую логику маршрута, установленную в методе `doGet` или `doPost`.

Возможным решением является создание десятков сервлетов, это приведет к более проверяемой и поддерживаемой кодовой базе, но количество сервлетов может быстро стать таким же неуправляемым, как и количество логических ветвей ранее. Если Ваше приложение будет содержать сотни функций, каждая из которых содержит десятки страниц — Вы быстро поймете, что исходный код «раздувается» на тысячи крошечных сервлетов, что нежелательно. Многие проблемы будут решены, если сервлеты будут отображены на уровне отдельных методов, а не только в классе.

При использовании Spring Web MVC Framework, класс контроллера очень похож на сервлет с сопоставлениями на уровне метода. Каждый метод может иметь четкое сопоставление с конкретным URL-адресом, методом запроса,

существованием параметра, значением заголовка, типом контента и/или ожидаемым типом ответа.

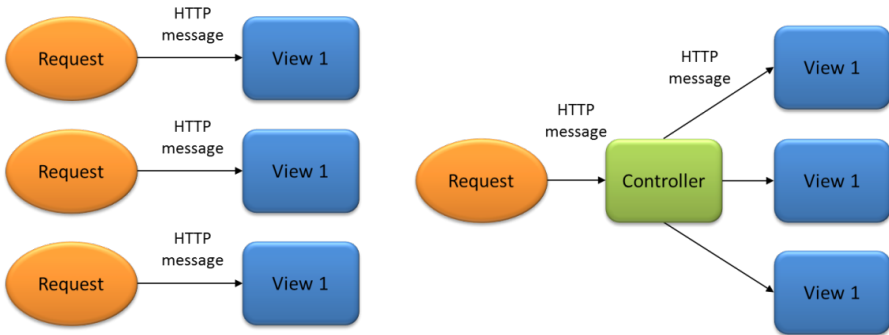


Рисунок 4.

В то время как Ваши юнит-тесты сосредоточены на тестировании небольших единиц кода, которые являются Вашими методами управления, классы контроллеров могут содержать много сопоставленных методов, которые логически сгруппированы вместе. К примеру в профиле пользователя, контроллер может иметь десятки сопоставленных методов, представляющих различные действия, которые могут выполняться в пользовательских профилях, но метод `doGet` или `doPost` не нужен для маршрутизации запросов к надлежащим методам. Spring Framework обрабатывает весь этот анализ и маршрутизацию самостоятельно.

В реальности Вам может быть предложено, создать расширенное приложение с тысячами функций, которые должны быть доступны из десктопных приложений, веб-браузеров, мобильных веб-браузеров, веб-сервисов RESTful и веб-сервисов SOAP. При использовании только сервлетов, эта задача быстро станет сложной. В конечном

итоге Вы будете дублировать много кода или создадите свою собственную систему, в которой бизнес-логика будет абстрагирована с набором других классов, используемых из многих пользовательских интерфейсов.

Spring Framework предоставляет именно такую систему, протестированную и уже готовую. Используя Spring, бизнес-логика инкапсулируется в набор бизнес-объектов, называемых сервисами. Эти сервисы выполняют операции, общие для всех пользовательских интерфейсов, такие как обеспечение правильного указания определенных свойств объекта. Ваше приложение содержит разный набор контроллеров и представлений для каждого пользовательского интерфейса, и они используют общие бизнес-объекты для выполнения критических операций. Контроллеры необходимы только для выполнения операций с конкретным интерфейсом пользователя, таких как перевод представлений форм или тела запроса JSON в сущности и отображение соответствующего представления пользователю. Модульное тестирование становится проще, код используется повторно, и для достижения этого требуется очень мало усилий.

Архитектура Spring

Spring Framework состоит из модулей, которые разделены на категории. Данные модули предоставляют все необходимое для разработки корпоративных приложений.

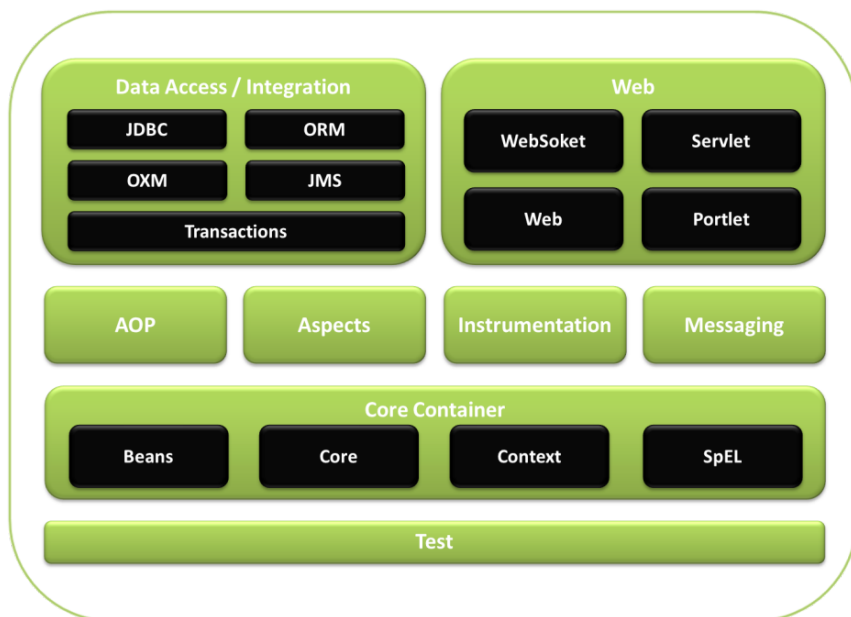


Рисунок 5. Архитектура Spring Framework

Для того чтобы получить представление о структуре Spring, достаточно посмотреть на огромный список флажков в полной версии веб-формы Spring Initializr. Он содержит более 100 вариантов выбора зависимостей, поэтому здесь мы не будем их перечислять, но Вы можете самостоятельно перейти по ссылке <https://start.spring.io/> и увидеть это. А пока что, рассмотрим некоторые основные моменты.

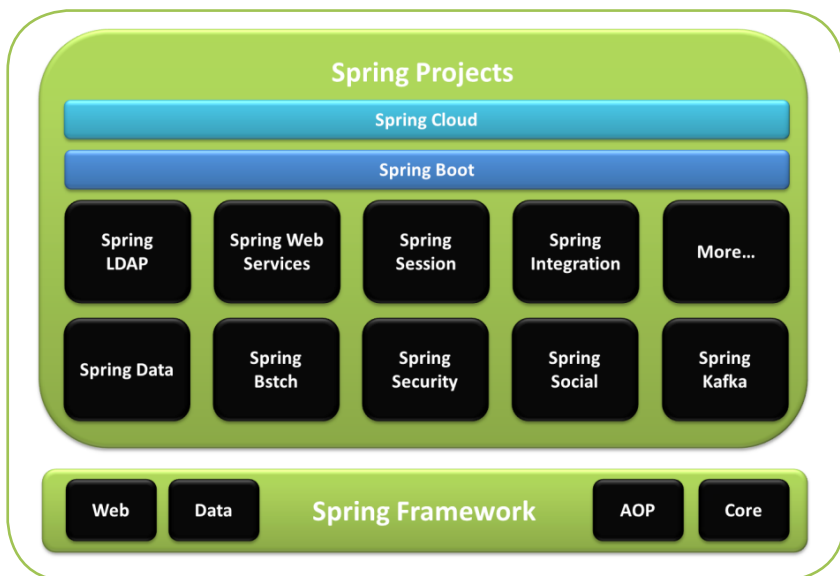


Рисунок 6. Набор инструментов Spring

Ядро Spring Framework является основой всего остального в мире Spring. Он предоставляет базовую платформу для внедрения контейнеров и зависимостей. Он также предоставляет несколько других важных функций. Среди них Spring MVC, веб-фреймворк Spring. Spring MVC можно использовать для записи класса контроллера для обработки веб-запросов, также для создания API REST, которые производят не HTML вывод. Ядро Spring Framework также предлагает некоторую поддержку хранения элементарных данных, в частности поддержку JDBC на основе шаблонов. В версии Spring 5.0.8 была добавлена поддержка для программирования с реактивным стилем, в том числе новая реактивная веб-инфраструктура Spring WebFlux, которая сильно зависит от Spring MVC.

Spring Boot имеет множество преимуществ и в дополнение к стартовым зависимостям и автоконфигурации также предлагает несколько полезных функций:

- Исполнительный компонент обеспечивает представление времени выполнения внутренней работы приложения, включая метрики, информацию о дампе потока, здоровье приложения и свойства среды, доступные для приложения.
- Гибкая спецификация свойств среды.
- Дополнительная поддержка тестирования.

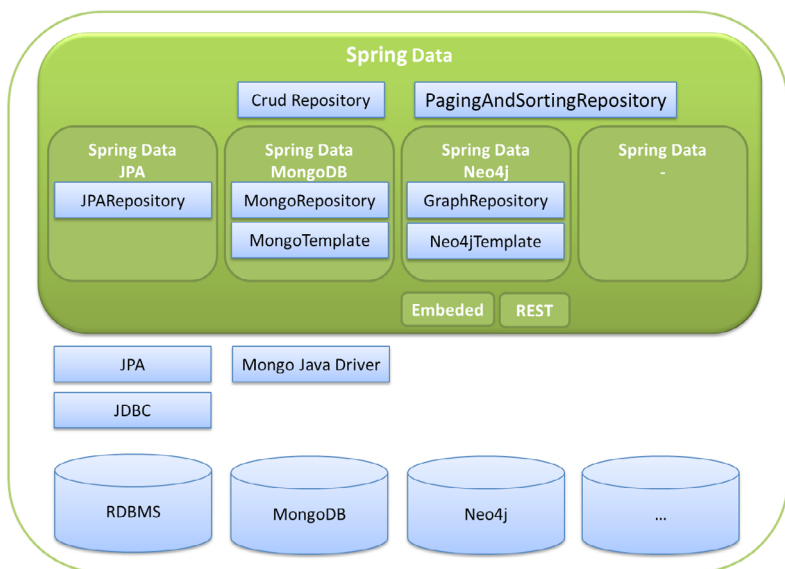


Рисунок 7. Spring Data

Spring Data обеспечивает возможность определять репозитории данных Вашего приложения как простые Java-интерфейсы, используя соглашение об именах при

определении методов, для управления хранением и извлечением данных. Более того, Spring Data способна работать с несколькими различными типами баз данных, включая реляционные (JPA), документоориентированные (Mongo), графовые (Neo4j) и другие.

Spring Security обеспечивает широкий спектр требований безопасности приложений, включая проверку подлинности, авторизацию и защиту API.

В какой-то момент большинство приложений необходимо интегрировать с другими приложениями или даже с другими компонентами одного и того же приложения. Spring Integration и Spring Batch обеспечивают реализацию этих шаблонов для приложений Spring.

Мир разработки приложений входит в новую эпоху, когда больше не понадобится разрабатывать приложения в качестве монолитов единичного развертывания и вместо этого будут создаваться приложения из нескольких отдельных блоков развертывания, известных как микросервисы.

Контейнер компонентов

В приложении на основе Spring Framework объекты приложения содержатся в контейнере Spring. Как показано на рисунке, контейнер будет создавать объекты, связывать их вместе, настраивать и управлять их жизненным циклом, от создания до уничтожения.

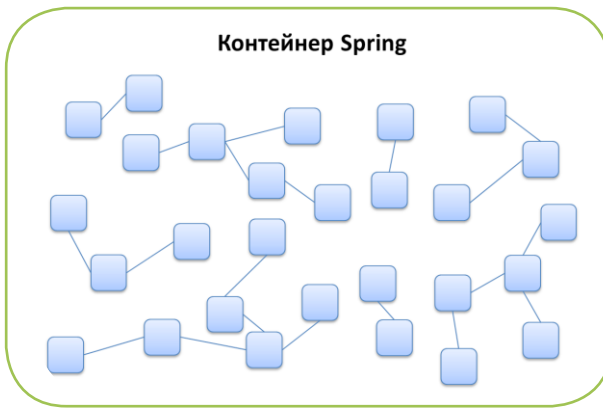


Рисунок 8.

Далее Вы узнаете, как настроить Spring Framework, чтобы указать ему, какие объекты следует создать, сконфигурировать и связать между собой. Сперва необходимо инструктировать контейнер, где будут размещаться объекты. Понимание логики работы контейнера позволит Вам понять, как будут обрабатываться объекты.

Контейнер лежит в основе Spring Framework. Контейнер Spring использует внедрение зависимостей для управления компонентами, составляющими приложение. Это включает создание взаимосвязей между компонентами.

Таким образом, объекты более понятны, поддерживают возможность повторного использования и легко тестируются.

Spring Framework состоит из контейнеров, которые можно разделить на два типа:

1. Фабрики компонентов (интерфейс [org.springframework.beans.factory.BeanFactory](https://docs.spring.io/spring-framework/docs/current/javadoc-api/org/springframework/beans/factory/BeanFactory.html)) являются простейшими контейнерами, предоставляющими базовую поддержку для внедрения зависимостей.
2. Контекст приложений (интерфейс [org.springframework.context.ApplicationContext](https://docs.spring.io/spring-framework/docs/current/javadoc-api/org/springframework/context/Application.html)) основан на понятии фабрики компонентов, реализует прикладные службы фреймворка, такие как возможность приема текстовых сообщений из файлов свойств и возможность публиковать события приложений для заинтересованных слушателей.

Можно работать со Spring, используя либо фабрики компонентов, либо контекст приложения, но фабрики компонентов являются низкоуровневым инструментом для большинства приложений. Поэтому контексты приложений предпочтительнее, чем фабрики компонентов. Мы сосредоточимся на работе с контекстами приложений.

Понятие контекста приложения

Ключевым понятием в Spring является **bean**. Как Вы уже знаете, это объект некоторого класса. Например, в Вашей программе необходимо использовать три объекта: рыбку, лягушку и пеликана. Также есть множество классов с различными методами. Для некоторых методов необходимо использовать иногда только рыбку, иногда лягушку, а иногда в методе могут понадобиться двое из них — рыбка и пеликан, для кормления пеликана. Возможно где-то будут использовать все объекты. Теоретически можно сначала создать их, а затем передать в классы, а внутри классов — в методы. И так по всей программе, но если появится необходимость изменить список принимаемых параметров для методов — тогда придется делать множество исправлений по коду всей программы. А вдруг этих объектов не три, а сотня? Тогда можно собрать такие объекты в один общий список, например **List<Object>**, и использовать его в качестве параметра во всех методах, а внутри методов получать необходимый Вам объект.

```
public void doSome(List <Object> animals) {  
    ...  
    System.out.println(animals.get(1));  
    ...  
}
```

Решение обращаться к элементам списка по индексу будет в корне не правильным, если во время выполнения программы этот список будет изменяться, например произойдет добавление или удаление элемента. В таком случае удобно хранить данные не в списке, а в **Map**, где ключ — имя объекта, а значение — объект.

```
public void doSome(Map <String,Object> animals) {  
    ...  
    System.out.println(animals.get("fish"));  
    ...  
}
```

Доступ к объектам будет производиться по имени: **get(«fish»)**. Также в качестве ключа можно указать класс объекта, а значения — объект, или написать некоторую обертку над **Map**, в которой будут методы, для доступа к объектам как по их имени, так и по классу.

Это и есть контекст приложения в Spring. Таким образом, еще раз повторимся, что контекст приложения это набор компонентов. Нужный компонент можно получить обращаясь к контексту по имени объекта, его типа, или каким либо иным способом.

Можно указать Spring найти в своем контексте нужный bean и передать его в Ваш метод. Например:

```
public void doSome(Fish fish) {  
    ...  
}
```

Spring при вызове этого метода передавал в него объект `fish` из своего контекста. Затем принято решение использовать в этом методе еще и объект `pelican`:

```
public void doSome(Fish fish, Pelican pelican) {  
    ...  
}
```

Spring при вызове этого метода сам поймет, что сюда надо передать рыбку и пеликана, обратится к своему контексту, возьмет оба объекта и передаст их в метод. Так как Spring сам управляет Вашей программой, он так же ответственный за создание и передачу объектов в методы, которые он будет вызывать.

Контекст приложения в Spring Framework представлен интерфейсом `org.springframework.context.ApplicationContext`. Существует ряд расширяемых интерфейсов и классов, реализующих `ApplicationContext`:

- Интерфейс `ConfigurableApplicationContext` настраивается, тогда как базовый `ApplicationContext` доступен только для чтения.
- Интерфейсы `org.springframework.web.context.WebApplicationContext` и `ConfigurableWebApplicationContext` предназначены для веб-приложений Java EE, работающих в контейнерах `Servlet`, и обеспечивают доступ к базовому `ServletContext` и `ServletConfig`.
- Конкретные классы `ClassPathXmlApplicationContext` и `FileSystemXmlApplicationContext` предназначены для загрузки конфигурации Spring из файлов XML в ав-

тономном приложении. `XmlWebApplicationContext` достигает того же самого в веб-приложениях Java EE.

- Для программной настройки Spring с использованием Java вместо XML классы `AnnotationConfigApplicationContext` и `AnnotationConfigWebApplicationContext` работают соответственно в автономных веб-приложениях Java EE. В веб-приложениях Java EE Spring обрабатывает веб-запросы с использованием диспетчера Servlet, который делегирует входящие запросы соответствующим контроллерам и при необходимости переводит объекты запроса и ответа.

Bootstrap Spring Framework

Рассмотрим настольный компьютер. При включении устройство начальной загрузки некоторого типа (обычно это BIOS) инициализирует все оборудование, загружает инструкции запуска из какого-либо места (загрузочный сектор на жестком диске) и запускает операционную систему. В этом смысле аппаратное обеспечение представляет собой контейнер для операционной системы. Операционная система — это еще один контейнер — для программного обеспечения, которое работает внутри него.

Программы на языке C имеют стандартизованный механизм начальной загрузки: метод `main()`. Исполняемый файл виртуальной машины Java, который обычно написан на C, содержит этот метод `main()`. Языковые виртуальные машины, такие как JVM, являются контейнерами для другого программного обеспечения. Для приложений Java, инструкции в файле манифеста приложения (или предоставленные в командной строке) загружают приложение Java, путем предоставления имени класса с помощью метода `public static void main(String...)`.

Как и все эти примеры, Spring Framework является еще одним контейнером. Он может работать в любом контейнере Java SE или EE и служит в качестве среды выполнения приложения. Также Spring должен быть загружен: он должен быть запущен и дать инструкции о том, как запустить приложение, которое он содержит.

Конфигурирование и самонастройка Spring Framework — это две разные задачи, которые могут обрабатываться по-разному независимо друг от друга. Конфигурация сообщает Spring о том, как запустить приложение которое он содержит, а процесс начальной загрузки запускает Spring и передает ему инструкции по настройке. В приложении Java EE есть два варианта: использовать дескриптор развертывания для загрузки Spring с использованием XML или программно загрузить Spring в `javax.servlet.ServletContainerInitializer`. Рассмотрим эти варианты. В традиционных приложениях Spring Framework, Spring всегда загружается в дескриптор развертывания Java EE. Для этого в проекте *Spring-XML-Configuration-v1* в файле *web.xml*, создан экземпляр `DispatcherServlet`, заданный конфигурационным файлом в виде `init` параметра `initConfigLocation`, с инструкцией загрузки при запуске.

```
<servlet>
    <servlet-name>springDispatcher</servlet-name>
    <servlet-class>org.springframework.web.servlet.
        DispatcherServlet
</servlet-class>
    <init-param>
        <param-name>contextConfigLocation</param-name>
        <param-value>/WEB-INF/servletContext.xml
        </param-value>
    </init-param>
    <load-on-startup>1</load-on-startup>
</servlet>
<servlet-mapping>
    <servlet-name>springDispatcher</servlet-name>
    <url-pattern></url-pattern>
</servlet-mapping>
```

Этот код создает единый контекст приложения Spring в настройке `DispatcherServlet` и указывает контейнеру `Servlet` инициализировать `DispatcherServlet` при запуске. При инициализации, `DispatcherServlet` загружает конфигурацию контекста из файла `/WEB-INF/servletContext.xml` и запускает контекст приложения.

Создание одного контекста приложения не очень гибкое решение, более полный дескриптор бутстрапа приведен в проекте *Spring-XML-Configuration-v2*:

```
<context-param>
  <param-name>contextConfigLocation</param-name>
  <param-value>/WEB-INF/rootContext.xml
</param-value>
</context-param>
<listener>
  <listener-class>org.springframework.web.context.
    ContextLoaderListener
  </listener-class>
</listener>
<servlet>
  <servlet-name>springDispatcher</servlet-name>
  <servlet-class>org.springframework.web.servlet.
    DispatcherServlet
  </servlet-class>
  <init-param>
    <param-name>contextConfigLocation</param-name>
    <param-value>/WEB-INF/servletContext.xml
  </param-value>
  </init-param>
  <load-on-startup>1</load-on-startup>
</servlet>
<servlet-mapping>
  <servlet-name>springDispatcher</servlet-name>
  <url-pattern>/</url-pattern>
</servlet-mapping>
```

`ContextLoaderListener` инициализируется при первом запуске веб-приложения, загружает конфигурацию контекста корневого приложения из файла `/WEB-INF/rootContext.xml`, указанного в `init` параметре контекста `contextConfigLocation` и запускает корневой контекст приложения.

Обратите внимание, что `init` параметр контекста `contextConfigLocation` для `Servlet` отличается от `init` параметра `contextConfigLocation` для `DispatcherServlet`. Они не конфликтуют, первый применяется ко всем контекстам, а последний применяется только к сервлету, для которого он указан. Корневой контекст приложения, созданный слушателем, автоматически устанавливается в качестве родительского контекста для всех контекстов приложения, созданных `DispatcherServlet`.

Эти примеры бутстрапов предполагают, что Spring настраивается с помощью XML-файлов. Также можно настроить Spring с помощью Java вместо XML.

Spring Framework обеспечивает интерфейс моста, который делает это намного проще. Класс `org.springframework.web.SpringServletContainerInitializer` реализует `ServletContainerInitializer`, а поскольку JAR, содержащий этот класс, содержит файл поставщика услуг, в котором указано имя класса, его метод `onStartup` вызывается, при запуске приложения. Затем этот класс сканирует приложение для реализации интерфейса `org.springframework.web.WebApplicationInitializer` и вызывает метод `onStartup` всех найденных совпадающих классов. В классе реализации `WebApplicationInitializer` можно программно настраивать слушателей,

сервлеты, фильтры и прочее, без написания XML. Более того, можно загружать Spring из этого класса.

Для этого в проекте Spring-Java-Configuration в пакете config создан класс **Bootstrap**:

```
public class Bootstrap implements
    WebApplicationInitializer {
    @Override
    public void onStartup(ServletContext container) {
        AnnotationConfigWebApplicationContext rootContext =
            new AnnotationConfigWebApplicationContext();
        rootContext.register(com.academy.config.
            RootContextConfiguration.class);
        container.addListener(new ContextLoaderListener(
            rootContext));
        AnnotationConfigWebApplicationContext
            servletContext =
            new AnnotationConfigWeb-
            ApplicationContext();
        servletContext.register(com.academy.config.
            ServletContextConfiguration.class);
        ServletRegistration.Dynamic dispatcher =
            container.addServlet("springDispatcher",
            new DispatcherServlet(
            servletContext));
        dispatcher.setLoadOnStartup(1);
        dispatcher.addMapping("/");
    }
}
```

Этот **bootstrap** использует конфигурацию Spring Java для чистого процесса загрузки и настройки Java.

Конфигурация Spring Framework

Теперь, когда Вы знаете, как загружать Spring Framework, рассмотрим его настройку. Мы будем использовать как XML, так и программную загрузку. Как Вы уже видели, можно настроить контекст приложения Spring с помощью XML или Java. Вы не только можете настроить разные контексты приложений в одном приложении по-разному, но также можете настроить один контекст приложения используя смесь Java и XML. Это необычный подход в современных приложениях, но в некоторых случаях это требуется. Например, конфигурация Java, добавленная в Spring Security 3.2, не охватывает каждый сценарий конфигурации, поэтому для некоторых примеров будем использовать вместо этого XML-конфигурацию Spring Security. Кроме того, инструменты Spring Web Services вообще не поддерживают конфигурацию Java. При настройке контекста приложения, использующего эти инструменты, можно настроить его с помощью Java, но для Spring Security или Spring Web Services импортировать конфигурацию XML.

Рассмотрим базовый код дескриптора развертывания, который будет использован в проектах текущего раздела:

```
<display-name>Spring Application</display-name>  
<jsp-config>  
  <jsp-property-group>
```

```

    <url-pattern>*.jsp</url-pattern>
    <url-pattern>*.jspf</url-pattern>
    <page-encoding>UTF-8</page-encoding>
    <scripting-invalid>true</scripting-invalid>
    <include-prelude>/WEB-INF/jsp/base.jspf
    </include-prelude>
    <trim-directive-whitespaces>true
    </trim-directive-whitespaces>
    <default-content-type>text/html
    </default-content-type>
  </jsp-property-group>
</jsp-config>

<session-config>
  <session-timeout>30</session-timeout>
  <cookie-config>
    <http-only>true</http-only>
  </cookie-config>
  <tracking-mode>COOKIE</tracking-mode>
</session-config>
<distributable />

```

Все проекты также включают в себя базовый интерфейс **HelloService** и реализацию **HelloServiceImpl**.

```

public interface HelloService {
    String sayHello(String name);
}

public class HelloServiceImpl implements HelloService
{
    @Override
    public String sayHello(String name) {
        return "Hello, " + name + ".";
    }
}

```

HelloController отвечает на веб-запросы. В идентичных проектах Вы можете настроить Spring Framework разными способами и достичь тех же целей.

```
@Controller
public class HelloController {
    private HelloService helloService;
    @ResponseBody
    @RequestMapping("/")
    public String helloWorld() {
        return "Hello, World!";
    }
    @ResponseBody
    @RequestMapping("/user")
    public String helloName(@RequestParam("name")
                           String name) {
        return this.helloService.sayHello(name);
    }
    public void setHelloService(HelloService
                               helloService) {
        this.helloService = helloService;
    }
}
```

Создание конфигурации XML

Spring Framework управляет компонентами, и это основное, что необходимо настраивать. Мы сами напишем некоторые из этих компонентов, например **HelloServiceImpl** и **HelloController**. Другие framework beans — это базовые компоненты по умолчанию, которые поставляются с Spring Framework, такие как реализация классов **SpringContext**, **ResourceLoader**, **BeanFactory**, **MessageSource** и **ApplicationEventPublisher**.

Чтобы инструктировать Spring, как настроить эти компоненты, используем пространство имен XML `<beans>`, продемонстрированное в проекте *Spring-XML-Configuration-v1*.

Взгляните на файл `/WEB-INF/servletContext.xml`:

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/
    beans"
    xmlns:xsi="http://www.w3.org/2001/
        XMLSchema-instance"
    xmlns:mvc="http://www.springframework.org/
        schema/mvc"
    xsi:schemaLocation="http://www.springframework.
        org/schema/beans
            http://www.springframework.org/
            schema/beans/spring-beans-4.0.xsd
            http://www.springframework.org/
            schema/mvc
            http://www.springframework.org/
            schema/mvc/spring-mvc-4.0.xsd">
    <mvc:annotation-driven />
    <bean name="helloServiceImpl"
        class="com.academy.HelloServiceImpl" />
    <bean name="helloController"
        class="com.academy.HelloController">
        <property name="helloService"
            ref="helloServiceImpl" />
    </bean>
</beans>
```

XML-файл сообщает Spring о создании `HelloServiceImpl` и `HelloController` и вставляет свойство `helloService` компонента `helloServiceImpl` в компонент `helloController`. Элемент `<beans>` является родительским элементом для хранения конфигурации Spring. Внутри него можно использовать

практически любой другой элемент конфигурации Spring, но, тем не менее, элементы, которые используются в `<beans>`, почти всегда приводят к созданию bean-компонентов. Элемент `<bean>` используется для явного построения компонента данного класса, можно указать аргументы и свойства конструктора для этого компонента в подэлементе элемента `<bean>`. Элемент `<mvc:annotation-driven>` указывает Spring использовать аннотации, такие как `@RequestMapping`, `@RequestBody`, `@RequestParam`, `@PathParam` и `@ResponseBody` для сопоставления запросов с методами контроллера. Использование элемента `<mvc:annotation-driven>` приводит к созданию определенных bean-компонентов за кулисами, о чем нам не нужно беспокоиться. Они существуют для облегчения отображения запросов в методах контроллера.

URL шаблон / сопоставляется с методом `helloWorld` `HelloController`, а URL шаблон `/user` с методом `helloName`. Эти шаблоны URL-адресов относятся к шаблону URL-адреса `DispatcherServlet`, а не к корневому URL-адресу веб-приложения. Поскольку `DispatcherServlet` сопоставляется с корнем приложения, это приводит к тому, что шаблоны URL также относятся к корню приложения. Если Вы сопоставили `DispatcherServlet` с `/folder/*`, шаблоны URL-адресов в аннотациях `@RequestMapping` не изменились бы, но в адресной строке браузера сначала будет `/folder`.

После создания конфигурации Spring, необходимо ее загрузить. Для этого используем дескриптор развертывания, но можно использовать и загрузку Java. Чтобы гарантировать, что Tomcat обрабатывает статические ресурсы, сопоставление сервлетов производится по умолчанию.

```

<display-name>Spring Application</display-name>
<servlet-mapping>
    <servlet-name>default</servlet-name>
    <url-pattern>/resource/*</url-pattern>
</servlet-mapping>
<servlet>
    <servlet-name>springDispatcher</servlet-name>
    <servlet-class>org.springframework.web.servlet.
        DispatcherServlet
    </servlet-class>
    <init-param>
        <param-name>contextConfigLocation</param-name>
        <param-value>/WEB-INF/servletContext.xml
    </param-value>
    </init-param>
    <load-on-startup>1</load-on-startup>
</servlet>
<servlet-mapping>
    <servlet-name>springDispatcher</servlet-name>
    <url-pattern>/</url-pattern>
</servlet-mapping>

```

Теперь скомпилируйте проект, запустите Tomcat и перейдите в браузере по адресу <http://localhost:8080/xml/>. На экране Вы увидите текст «Hello, World!». Затем перейдите по URL <http://localhost:8080/xml/user?name=Ivan>. Теперь будет отображено «Hello, Ivan.». Измените параметр имени, и результат должен измениться соответственно. Ваше первое приложение Spring Framework работает!

В веб-приложении желательно иметь два разных контекста приложения. Здесь мы создали только один. Типичный шаблон помещает все классы, связанные с бизнес-логикой, в контексте корневого приложения, а контроллеры — в контексте приложения Servlet.

Создадим еще один проект *Spring-XML-Configuration-v2* и внесем простые изменения. Необходимо удалить приветствие `helloServiceImpl` из файла *helloContext.xml* (если его не удалить — он будет создан дважды) и создать новый файл */WEB-INF/rootContext.xml*, содержащий только этот компонент: `<?xml version="1.0" encoding="UTF-8"?>`

```
<beans xmlns="http://www.springframework.org/
        schema/beans"
        xmlns:xsi="http://www.w3.org/2001/
        XMLSchema-instance"
        xsi:schemaLocation=
            "http://www.springframework.org/
            schema/beans
            http://www.springframework.org/
            schema/beans/spring-beans-4.0.xsd">
    <bean name= "helloServiceImpl"
        class= "com.academy.HelloServiceImpl"/>
</beans>
```

Затем добавим `ContextLoaderListener` в дескриптор развертывания.

```
<context-param>
    <param-name>contextConfigLocation</param-name>
    <param-value>/WEB-INF/rootContext.xml</param-value>
</context-param>

<listener>
    <listener-class>org.springframework.web.context.
        ContextLoaderListener
    </listener-class>
</listener>
```

Теперь у Вас есть два контекста приложения: контекст корневого приложения содержит компонент `helloServiceImpl`, и контекст приложения `DispatcherServlet` наследует все компоненты корневого контекста, содержит компонент `helloController` и инициализирует сопоставление контроллеров, управляемых аннотациями. Чтобы сделать это более интересным, настройте отображение запроса по методу `helloName` контроллера.

```
@RequestMapping(value = "/", params = {"name"})  
public String helloName(@RequestParam("name")  
                        String name)
```

Метод теперь сопоставляется с тем же URL-адресом, что и метод `helloWorld`, но для этого требуется параметр `name`, который должен присутствовать в запросе. Простое наличие или отсутствие параметра `name` определяет, какой метод `Spring` отправляет запрос. Чтобы проверить это, снова скомпилируйте и запустите приложение и перейдите по адресам `http://localhost:8080/xml/` и `http://localhost:8080/xml/?name=Ivan`.

Создание гибридной конфигурации

Настройка Spring Framework с XML может стать весьма утомительной, и в предыдущем примере даже не продемонстрированы классы `MultiActionController` и `SimpleUrlHandlerMapping` — устаревший механизм (удален в Spring Framework 4.0) для сопоставления запросов контроллерам, в которых задействованы чрезвычайно подробные XML и строгие подписи метода, принимающие `HttpServletRequest`.

Requests и **HttpServletResponses**. В крупном корпоративном приложении может понадобится определить сотни beans, каждый из которых требует трех или более строк XML. Этот способ настройки выглядит не лучше, чем настройка сервлета с использованием XML в дескрипторе развертывания.

Если Вы предпочитаете XML, но не хотите, чтобы файлы конфигурации разрослись и стали чрезвычайно «тяжелыми» для поддержания — можно создать гибридную конфигурацию.

В основе этой гибридной конфигурации лежат концепции сканирования компонентов и конфигурации аннотаций. Используя сканирование компонентов, Spring сканирует пакеты с конкретными аннотациями, которые Вы указываете для классов. Любые классы (в этих пакетах), аннотированные с помощью **@org.springframework.stereotype.Component** становятся управляемыми компонентами Spring, то есть Spring создает их и вводит их зависимости.

Другие аннотации подходящие для сканирования компонентов: любая аннотация, аннотированная с помощью **@Component**, становится аннотацией компонента, и любая аннотация, аннотированная аннотацией другого компонента, становится аннотацией компонента. Поэтому классы, аннотированные с помощью **@Controller**, **@Repository** и **@Service** (все в том же пакете, что и **@Component**), также становятся управляемыми компонентами Spring. Также можно создавать свои собственные аннотации компонентов. Например, можно создать аннотации **@WebController** и **@RestController**, отмеченные с помощью

`@Controller`, чтобы отличать обычные контроллеры от RESTful-контроллеров веб-сервисов.

Еще одна ключевая аннотация, которая работает в сочетании с конфигурацией аннотаций — `@org.springframework.beans.factory.annotation.Autowired`. Можно аннотировать любое приватное, защищенное или публичное поле или публичный мутатор, который принимает один или несколько аргументов с помощью `@Autowired`. `@Autowired` объявляет зависимость или зависимости, которые Spring должен вводить после создания экземпляра, а также может указывать конструктор. Обычно управляемые компоненты Spring должны иметь конструкторы с нулевым аргументом, но при наличии одного конструктора `@Autowired` в классе, Spring использует этот конструктор и вводит все аргументы конструктора.

Если Spring не находит соответствующий компонент для зависимости, он выдает и регистрирует исключение и не запускается. Аналогично, если он находит более одного подходящего компонента для зависимости, он также генерирует и регистрирует исключение и не запускается. Можно избежать второй проблемы, используя либо аннотацию `@org.springframework.beans.factory.annotation.Qualifier`, либо `@org.springframework.context.annotation.Primary`. При использовании `@Autowired` в поле метода, параметра метода или параметре конструктора, `@Qualifier` позволяет указать имя компонента, который должен использоваться. И наоборот, можно аннотировать компонент `@Primary`, чтобы указать, что он должен быть предпочтительным, если он является

одним из нескольких компонентов, которые выполняют зависимость.

Итак, следует ли использовать `@Autowired` с полями, мутаторами или конструкторами? Некоторые разработчики предпочитают иметь поля `@Autowired`, потому что они связаны с наименьшим количеством кода. Другие — предпочитают использовать `@Autowired` конструкторы, потому что это делает невозможным построение объекта без удовлетворения всех его зависимостей. Третьи предпочитают использовать `@Autowired` мутаторы, не смотря на то, что он включает в себя большинство кода, он избегает не приватные поля, а также упрощает модульное тестирование.

Как упоминалось ранее, Spring автоматически предоставляет множество стандартных базовых компонентов, и иногда Вашим компонентам нужны их экземпляры. Как правило, использование базовых компонентов не рекомендуется, потому что это еще больше связывает приложение с Spring Framework и затрудняет переход на что-то другое. Тем не менее, нужно использовать эти компоненты, когда необходимо использовать некоторые функции Spring, такие как обмен сообщениями для оповещения-подписки. Вы можете пометить любой из этих beans как `@Autowired`, но большинство предпочитают получать их по-разному, чтобы сделать соединение с Spring более очевидным. Для этого существуют интерфейсы `org.springframework.beans.factory.Aware`. `Aware` — это просто интерфейс маркера для других интерфейсов, чтобы расширить индикацию определенных фреймов. Например,

интерфейс `org.springframework.context.ApplicationContext-Aware` указывает метод `setApplicationContext` для предоставления экземпляру `ApplicationContext` текущего (не родительского) контекста Вашим компонентам. При необходимости можно получить родительский контекст, вызвав метод `getParent` метода `ApplicationContext`.

Существует множество интерфейсов, расширяющих `Aware`, рассмотрим самые популярные:

Интерфейс	Назначение
<code>ApplicationEventPublisherAware</code>	Для получения компонента, используемого для публикации события приложения
<code>BeanFactoryAware</code>	Для получения <code>BeanFactory</code> , с помощью которого Вы можете вручную извлекать или создавать компоненты
<code>EnvironmentAware</code>	Для получения объекта среды, который можно использовать для получения свойств из источников свойств
<code>MessageSourceAware</code>	Для получения источника сообщений интернационализации
<code>ServletContextAware</code>	Для получения <code>ServletContext</code> в средах веб-приложений Java EE
<code>ServletConfigAware</code>	Для получения <code>ServletConfig</code> в компонентах, управляемых контекстом <code>DispatcherServlet</code>

Зачастую желательно выполнить некоторую инициализацию в классе после того как были введены все его зависимости, но до того как они будут введены в другие бины. Это достигается путем реализации интерфейса `org.springframework.beans.factory.InitializingBean`. После того,

как Ваш компонент будет сконфигурирован (введены зависимости, а также методы [Aware](#)), Spring вызывает метод [afterPropertiesSet](#). Если Вы не хотите привязывать свое приложение к этому интерфейсу Spring, Вы можете создать один общедоступный метод [void](#), без аргументов, аннотированный с помощью [@javax.annotation.PostConstruct](#). Он будет вызываться в то же время, что и [afterPropertiesSet](#). Он будет вызываться, даже если Ваш bean не объявил никаких зависимостей, требующих инъекции.

С другой стороны, если Ваш bean-компонент должен быть закрыт после того, как ссылки на него исчезли, и до того, как Spring отключится, Вы можете реализовать метод [destroy org.springframework.beans.factory.DisposableBean](#) или создать свой собственный публичный метод [void](#), без аргументов, аннотированный с помощью [@javax.annotation.PreDestroy](#).

Инициирование сканирования компонентов и настройки аннотации — простая задача с использованием существующей конфигурации дескриптора развертывания и файлов *rootContext.xml* и *servletContext.xml*. Можно увидеть это, изменив проект Spring-Hybrid-Configuration. В конфигурации *rootContext.xml* просто удалим все определения bean и заменим их элементами [<context: annotation-config>](#) и [<context: component-scan>](#):

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/
        beans"
        xmlns:xsi="http://www.w3.org/2001/
        XMLSchema-instance"
```

```

xmlns:context="http://www.springframework.org/
    schema/context"
xsi:schemaLocation="http://www.springframework.
    org/schema/beans
    http://www.springframework.
    org/schema/beans/
    spring-beans-4.0.xsd
    http://www.springframework.
    org/schema/context
    http://www.springframework.
    org/schema/context/
    spring-context-4.0.xsd">
<context:annotation-config />
<context:component-scan base-package=
    "com.academy">
    <context:exclude-filter type="annotation"
        expression="org.springframework.
            stereotype.Controller"/>
</context:component-scan>
</beans>

```

Атрибут базового пакета указывает Spring Framework проверять все классы на пути класса в пакете *com.academy* или в любых подпакетах для **@Component**, **@Controller**, **@Repository** и **@Service**. Аналогичный файл конфигурации *servletContext.xml* по-прежнему содержит элемент **<mvc:annotation-driven>**:

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/
    beans"
    xmlns:xsi="http://www.w3.org/2001/
        XMLSchema-instance"
    xmlns:context="http://www.springframework.org/
        schema/context"

```

```

xmlns:mvc="http://www.springframework.org/
    schema/mvc"
xsi:schemaLocation="http://www.springframework.
    org/schema/beans
    http://www.springframework.org/schema/
    beans/spring-beans-4.0.xsd
    http://www.springframework.org/schema/
    context
    http://www.springframework.org/schema/
    context/spring-context-4.0.xsd
    http://www.springframework.org/schema/mvc
    http://www.springframework.org/schema/
    mvc/spring-mvc-4.0.xsd">
<mvc:annotation-driven />
<context:annotation-config />
<context:component-scan
    base-package="com.academy"
    use-default-filters="false">
    <context:include-filter type="annotation"
        expression="org.springframework.
        stereotype.Controller" />
</context:component-scan>
</beans>

```

Мы все еще имеем две незначительные детали:

- Для того чтобы указать Spring создать экземпляр **HelloServiceImpl**, аннотируем класс с помощью **@Service**.

```

@Service
public class HelloServiceImpl implements HelloService

```

- Для того чтобы указать Spring ввести **HelloService** в **HelloController**, аннотируем мутатор метода с помощью **@Autowired**.

```
@Autowired
public void setHelloService(HelloService helloService)
```

Эта конфигурация имеет проблему, которую можно заметить, поставив точку останова в мутаторе `setHelloService` перед запуском Tomcat. Эта точка останова дважды срабатывает для двух разных экземпляров `HelloController` и `HelloServiceImpl`. Это также можно посмотреть в файле журнала приложения. Spring дважды запускает `HelloServiceImpl` и `HelloController`: один раз в корневом контексте приложения и один раз в контексте приложения `DispatcherServlet`. Почему это происходит?

Помните, что сканирование компонентов по умолчанию проверяет все `@Component`, и мы включили сканирование компонентов в обоих контекстах приложения. То, что нам действительно нужно — это отделить наши beans. Контекст корневого приложения должен содержать службы, репозитории и другие части бизнес-логики, тогда как контекст приложения `DispatcherServlet` должен содержать веб-контроллеры. У нас есть простой способ изменить алгоритм сканирования компонентов по умолчанию. В проекте `Spring-Hybrid-Configuration` сконфигурируем `rootContext.xml`, добавив исключение для сканирования:

```
<context:annotation-config />
  <context:component-scan base-package="com.academy">
    <context:exclude-filter type="annotation"
      expression="org.springframework.
        stereotype.Controller" />
  </context:component-scan>
```

Этот фильтр исключений указывает Spring сканировать все `@Component` кроме `@Controller`. Помимо этого исключения, по умолчанию все еще применяется шаблон сканирования. Конфигурация `servletContext.xml` использует несколько иной подход и вместо белого списка использует черный список, чтобы указать Spring о том, какие компоненты сканируются:

```
<mvc:annotation-driven />
<context:annotation-config />
<context:component-scan base-package="com.academy"
    use-default-filters="false">
    <context:include-filter type="annotation"
        expression="org.springframework.stereotype.
            Controller" />
</context:component-scan>
```

В этом случае атрибут `use-default-filters`, установленный в `false`, указывает Spring игнорировать его стандартный шаблон сканирования. Вместо `exclude-filter` (в корневом контексте), `include-filter` указывает Spring сканировать только `@Controller`. Даже если `use-default-filters true` — можно использовать `include-filter` для добавления к фильтрам сканирования по умолчанию.

Хотя мы удалили все определения `<bean>` из файлов конфигурации, все равно можно использовать `<bean>` и компонентное сканирование вместе. Существуют случаи, когда нельзя использовать сканирование для регистрации компонента. Одним из примеров этого является регистрация класса, предоставленного третьей стороной в качестве компонента. Поскольку у Вас будет только скомпилированная

версия класса, Вы не сможете добавлять к ней примечания Spring. Другим примером является регистрация базовых компонентов, которые не зарегистрированы по умолчанию, например инструменты Java Persistence API. В любое время, при необходимости можно указать элементы `<bean>` в своей конфигурации, для дополнения того что Spring находит при сканировании компонентов.

Скомпилируйте приложение и запустите Tomcat из среды разработки. Перейдите в браузере по адресу `http://localhost:8080/hybrid/` и `http://localhost:8080/hybrid/?name=Ivan`, и Вы заметите, что приложение работает точно так же, как и при использовании конфигурации XML.

Настройка Spring с помощью Java используя @Configuration

Настройка Spring в значительной степени зависит от личных предпочтений. Некоторые команды разработчиков предпочитают XML-файлы. Тем не менее, существуют определенные неудобства настройки Spring с использованием XML:

- Трудоемкая отладка конфигурации XML. Необходимо загрузить и прикрепить источники Spring Framework к проекту, а также узнать, где разместить точки останова и как пройти через них. Исходный код Spring массивный, и его отладка является не тривиальной задачей.
- Невозможность модульного тестирования конфигурации XML. Естественно, можно программно загружать

конфигурацию Spring в модульном тесте, но это не единственный тест. Это на самом деле тест интеграции, поскольку будет запущен весь контекст приложения и связаны все компоненты. Невозможно тестировать отдельные блоки конфигурации Spring, если они основаны на XML.

Чистая Java конфигурация Spring Framework позволяет программно настроить контейнер Spring, чтобы легко производить отладку и тестирование фрагментов конфигурации.

Ранее мы рассмотрели программную конфигурацию загрузки Spring с помощью `AnnotationConfigWebApplicationContext`. При использовании этого класса с помощью метода `register` регистрируются классы конфигурации. Эти классы конфигурации (должны быть аннотированы `@org.springframework.context.annotation.Configuration` и должны иметь конструктор по умолчанию) регистрируют beans через методы без аргументов, аннотированные `@Bean`. С помощью классов `@Configuration` можно сделать много всего, и при необходимости выполнить единственный тест и отладить каждый метод.

Аннотирование `@Configuration` является мета-аннотированным с `@Component`, то есть классы `@Configuration` имеют право на инъекцию зависимостей с использованием `@Autowired`, могут реализовать любой из `Aware` интерфейсов, `InitializingBean` или `DisposableBean` и могут иметь методы `@PostConstruct` и `@PreDestroy`. Это полезно, если классу `@Configuration` необходимо напрямую обращаться к файловому компоненту или компоненту, созданному

в другом классе `@Configuration`. А также, если ему нужно инициализировать два или более зависимых компонента после ввода собственных зависимостей, но до того, как Spring вызовет свои методы `@Bean`.

Тот факт, что классы `@Configuration` являются `@Components`, также означает, что классы `@Configuration` автоматически подхватываются, если Вы включили сканирование компонентов в пакете, который их содержит. Это может иметь как предполагаемые, так и непреднамеренные последствия. Если класс `@Configuration` включает проверку компонентов в том же пакете, в котором он находится, beans могут быть созданы дважды, что плохо.

С другой стороны, если разделить приложение на множество модулей, каждый из них может иметь свою собственную `@Configuration`, и Вы можете создать ядро `@Configuration`, которое компонент сканирует для `@Configuration` всех модулей. Ключевым моментом для этого является то, что всегда нужно быть осторожным при объединении `@Configuration` и сканирования компонентов.

Часто желательно иметь внешние значения, которые управляют конфигурацией. Например, приложению может потребоваться использование источника данных JNDI, но Вы не хотите жестко закодировать имя этого источника данных. Вместо этого желательно сохранить имя источника данных в некоторых файлах настроек, загруженных во время настройки. Используя аннотацию `@org.springframework.context.annotation.PropertySource`, Вы можете сделать именно это.

```

@Configuration
@PropertySource({
    "classpath:com/academy/config/settings.properties",
    "file:config.properties"
})

public class TestConfiguration
...

```

Для доступ к свойствам в источнике свойств, можно использовать экземпляр `@org.springframework.core.env.Environment` для получения значений вручную или можно использовать аннотацию `@org.springframework.beans.factory.annotation.Value`, для автоматического ввода значений свойств.

```

...
public class TestConfiguration
{
    @Inject Environment environment;
    @Value("my.property.key") String myPropertyValue;
    ...
}

```

Возможно, Вы не можете или не хотите содержать всю конфигурацию в одном классе. В этом случае не удастся настроить все с помощью Java. Как обсуждалось ранее, некоторые инструменты могут по-прежнему нуждаться в конфигурации XML. Аннотации `@org.springframework.context.annotation.Import` и `@ImportResource` обеспечивают поддержку этих потребностей.

```

@Configuration
@Import({ DatabaseConfiguration.class,
          ClusterConfiguration.class })
@ImportResource("classpath:com/academy/config/
                spring-security.xml")
public class TestConfiguration
...

```

В предыдущем фрагменте кода Spring Framework загружает конфигурацию, содержащуюся в `TestConfiguration`, а также конфигурацию, содержащуюся в `DatabaseConfiguration`, `ClusterConfiguration` и `Spring-security.xml`. Он не инициализирует и не выполняет эти конфигурации в определенном порядке, вместо этого он определяет, зависит ли какая-либо из конфигураций от bean-компонентов, предоставляемых любой другой конфигурацией, и инициализирует конфигурации в правильном порядке для выполнения этих зависимостей.

Spring Framework имеет множество функций конфигурации XML которые мы уже использовали, и которые не включают определения `<bean>`. Это приводит к очень специфическим определениям базового компонента за кулисами, но позволяет указать конфигурацию с минимальными усилиями и включать такие вещи, как `<mvc:annotation-driven>` и `<context:component-scan>`. Существуют замены для всех этих элементов пространства имен XML, используя набор аннотаций, связанных с конфигурацией. Можно пометить любой класс как аннотированный `@Configuration` со следующими аннотациями, чтобы включить эти функции.

Аннотация	Комментарий
@ComponentScan	Заменяет <context:component-scan> и включает проверку компонентов в указанном пакете или пакетах. Можно включать или отключать фильтры по умолчанию и настраивать фильтры включения и исключения для точной настройки алгоритма сканирования компонентов.
@EnableAspectJAutoProxy	Заменяет <aop:aspectj-autoproxy> и позволяет поддерживать классы обработчики, помеченные аннотацией @Aspect от AspectJ, и, при необходимости, оповещает методы с использованием аспектно-ориентированного программирования.
@EnableAsync	Заменяет пространство имен <task:*> и включает асинхронный метод @Async в Spring. При использовании с интерфейсом AsyncConfigurer @Configuration может точно настроить конфигурацию асинхронного поведения.
@EnableCaching	Позволяет использовать функции управления кешем Spring, зависящие от аннотаций, и заменяет пространство имен <cache:*>.
@EnableLoadTimeWeaving	Заменяет <context:load-time-weaver> — изменяет работу нескольких функций, таких как @Transactional, @Configurable, @Aspect и т.д. Важно отметить, что он является взаимоисключающим с @EnableAspectJAutoProxy.
@EnableMBeanExport	Заменяет <context:mbean-export> — инструктирует Spring выставять определенные фреймы структуры и любые beans аннотированные @ManagedResource — как JMX MBeans.

Аннотация	Комментарий
<code>@EnableScheduling</code>	Заменяет остальную часть пространства имен <code><task:*></code> Spring и активирует выполнение запланированного метода с помощью <code>@Scheduled</code> , аналогично выполнению метода <code>@Async</code> , разрешенному <code>@EnableAsync</code> .
<code>@EnableSpringConfigured</code>	Активирует инъекцию зависимостей в объектах, которые не управляются Spring и не заменяют <code><context:spring-configuration></code> . Должно происходить во время компиляции, так как оно должно перехватывать конструкцию объекта.
<code>@EnableTransactionManagement</code>	Заменяет <code><tx:annotation-driven></code> и позволяет управлять транзакциями базы данных для методов, рекомендованных аннотацией <code>@Transactional</code> .
<code>@EnableWebMvc</code>	Активирует редактирование запросов контроллера, привязанное к аннотации, и заменяет <code><mvc:annotation-driven></code> . Активирует очень сложную конфигурацию, которую часто необходимо настраивать. Можно настроить всю конфигурацию веб-MVC, создав класс <code>@Configuration</code> с помощью <code>WebMvcConfigurer</code> или, проще, настроить только те части, которые необходимы, расширяя <code>WebMvcConfigurerAdapter</code> .

Проект Spring-Java-Configuration демонстрирует использование чистой конфигурации Java из начальной загрузки `WebApplicationInitializer` в классы `@Configuration` для контекстов приложений `root` и `DispatcherServlet`. Классы `HelloService`, `HelloServiceImpl` и `HelloController` были перенесены в пакет `com.academy.site`, чтобы отделить их от классов конфигурации в пакете `com.academy.config`. Таким

образом, сканирование компонентов не будет случайно определять классы `@Configuration`.

Класс `RootContextConfiguration` в этом проекте очень прост:

```
@Configuration
@ComponentScan(basePackages = "com.academy.site",
                excludeFilters = @ComponentScan.
                               Filter(Controller.class)
)
public class RootContextConfiguration { }
```

Аннотация `@ComponentScan` указывает `Spring` сканировать классы в пакете `com.academy.site` и во всех подпакетах исключая классы `@Controller`, как и гибридная конфигурация. Новый класс `ServletContextConfiguration` аналогичен, отключает фильтры сканирования по умолчанию только для `@Controllers` и включает управляемые аннотацией функции веб-MVC.

```
@Configuration
@EnableWebMvc
@ComponentScan(
    basePackages = "com.academy.site",
    useDefaultFilters = false,
    includeFilters = @ComponentScan.
                    Filter(Controller.class)
)
public class ServletContextConfiguration { }
```

Нужно заметить, что ни один из этих классов не имеет методы `@Bean`. Все необходимые компоненты настраиваются автоматически, с помощью сканирования компонентов

и `@EnableWebMvc`, поэтому нам не нужны какие-либо методы `@Bean`.

Конечно, эти классы конфигурации не могут загружаться самостоятельно, поэтому класс `Bootstrap` реализует `WebApplicationInitializer`.

```
public class Bootstrap implements
    WebApplicationInitializer {
    @Override
    public void onStartup(ServletContext container)
        throws ServletException {
        container.getServletRegistration("default").
            addMapping("/resource/*");
        AnnotationConfigWebApplicationContext
            rootContext = new AnnotationConfig
                WebApplicationContext();
        rootContext.register(RootContextConfiguration.class);
        container.addListener(new ContextLoaderListener(
            rootContext));
        AnnotationConfigWebApplicationContext
            servletContext = new AnnotationConfig
                WebApplicationContext();
        servletContext.register(
            ServletContextConfiguration.class);
        ServletRegistration.Dynamic dispatcher =
            container.addServlet("springDispatcher",
                new DispatcherServlet(servletContext));
        dispatcher.setLoadOnStartup(1);
        dispatcher.addMapping("/");
    }
}
```

`init` параметры, слушатель и `DispatcherServlet`, настроенные ранее, удалены, а `web.xml` теперь содержит только базовую конфигурацию JSP и сессии.

Теперь скомпилируйте приложение, запустите Tomcat из IDE и перейдите по адресу *<http://localhost:8080/java/>* и *<http://localhost:8080/java/?name=Ivan>*. Приложение функционирует идентично приложениям Spring-XML-Configuration-v2 и Spring-Hybrid-Configuration.

Использование профилей определения Bean

Профили определения bean в Spring позволяют легко включать или отключать целые разделы конфигурации с помощью простого переключателя в командной строке, в дескрипторе развертывания или программно. Эта функциональность очень удобна во многих случаях использования. Ниже приводятся некоторые примеры:

- В среде приложений со многими уровнями необходимы некоторые beans для работы на одном уровне, а другие beans — на другом. Используя профили определения bean-компонентов, можно развернуть одно приложение на каждый уровень, в то время как активированный профиль контролирует, какие компоненты зарегистрированы.
- Можно написать коммерческое приложение, которое будет работать со многими различными типами хранилищ данных. Когда конечные пользователи приобретают и устанавливают приложение, они указывают, какой тип хранилища данных они хотят использовать. В приложении может быть профиль API Java Persistence, содержащий репозитории JPA для хранения реляционных баз данных и профиль Spring Data NoSQL, содержащий репозитории NoSQL для записи данных в хранилища без схемы. Пользователи могут установить один и тот же исполняемый файл,

в то время как простой переключатель конфигурации включает правильный профиль.

- Возможно, необходимо создать различные профили разработки (*Development*), обеспечения качества (*Quality Assurance*) и производства (*Production*). В профиле разработки можно корректировать определенные параметры, например подключение к локальной базе данных, которое должно создаваться разработчиками. Аналогично, профиль обеспечения качества также может иметь жестко заданные параметры, которые отличаются от производственного оборудования. Несомненно, команде рабочей среды необходимо изменить настройки приложения, не дожидаясь, когда Вы измените и перекомпилируете ее, поэтому профиль производства загрузит эти параметры из файла свойств, который могут изменить технические специалисты.

Подобно другим технологиям, которые предоставляют профили конфигурации (например, Maven), использование Spring профилей определения [bean](#) имеет два компонента: декларация и активация. Можно объявлять профили, используя элементы `<beans>` в файлах конфигурации XML, аннотацию `@org.springframework.context.annotation.Profile` для классов `@Configuration` или `@Components` или обоих. Любой элемент `<beans>` может содержать атрибут профиля, указывающий, что его компоненты принадлежат определенному профилю. К счастью, не нужно создавать новый файл configura-

ции для каждого создаваемого профиля, потому что мы можем вложить элементы `<beans>`. Следующий фрагмент кода демонстрирует это:

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/
    schema/beans"
    xmlns:xsi="http://www.w3.org/2001/
        XMLSchema-instance"
    xsi:schemaLocation="http://www.springframework.
        org/schema/beans
        http://www.springframework.org/schema/
        beans/spring-beans-4.0.xsd">
    ...
    <beans profile="development,qa">
        <jdbc:embedded-database id="dataSource"
            type="HSQL">
            <jdbc:script location="classpath:com/
                academy/config/sql/schema.sql"/>
            <jdbc:script location="classpath:com/
                academy/config/sql/test-data.sql"/>
        </jdbc:embedded-database>
    </beans>
    <beans profile="production">
        <context:property-placeholder
            location="file:/settings.properties" />
        <jee:jndi-lookup id="dataSource"
            jndi-name="java:/comp/env/
                ${production.dsn}" />
    </beans>
    ...
</beans>
```

В этом примере в разных профилях зарегистрированы два по-разному настроенных компонента **Data-Source**. Встроенная база данных **HyperSQL**, встроенная

в память, создается, если активны профили `development` или `qa`, тогда как источник данных с настроенным именем просматривается в контексте JNDI только в том случае, если профиль `production` активен. Поскольку оба компонента реализуют `DataSource` и имеют один и тот же идентификатор компонента, любой `<bean>` может использовать ссылку на компонент `dataSource`, а любой `@Component` может автоматически вводить соответствующий `DataSource`. Также можно добиться этого, используя `Java @Configuration`, как показано в следующем коде.

```
interface DataConfiguration {
    DataSource dataSource();
}

@Configuration
@Import({DevQaDataConfiguration.class,
        ProductionDataConfiguration.class})
@ComponentScan(
    basePackages = "com.academy.site",
    excludeFilters = @ComponentScan.
                    Filter(Controller.class)
)

public class RootContextConfiguration { }

@Configuration
@Profile({"development", "qa"})
public class DevQaDataConfiguration implements
        DataConfiguration {

    @Override
    @Bean
    public DataSource dataSource() {
        return new EmbeddedDatabaseBuilder()
            .setType(EmbeddedDatabaseType.HSQL)
    }
}
```

```

        .addScript("classpath:com/academy/config/sql/
                    schema.sql")
        .addScript("classpath:com/academy/config/sql/
                    test-data.sql")
        .build();
    }
}

@Configuration
@Profile("production")
@PropertySource("file:settings.properties")
public class ProductionDataConfiguration implements
                                   DataConfiguration {

    @Value("production.dsr")
    String dataSourceName;

    @Override
    @Bean

    public DataSource dataSource() {
        return new JndiDataSourceLookup().
            getDataSource("java:/comp/env/" +
                this.dataSourceName);
    }
}

```

После того, как Вы объявили свои профили, можно активировать их одним или несколькими способами. Можно использовать контекст init-параметр `spring.profiles.active`:

```

<context-param>
    <param-name>spring.profiles.active</param-name>
    <param-value>development</param-value>
</context-param>

```

Или init-параметр сервлета:

```
<servlet>
    ...
    <init-param>
        <param-name>spring.profiles.active</param-name>
        <param-value>development</param-value>
    </init-param>
    ...
</servlet>
```

Параметр контекста влияет на все контексты приложения Spring, запущенные в веб-приложении, тогда как init-параметр сервлета влияет только на контекст приложения `DispatcherServlet`, для которого он установлен.

Также можно использовать параметр командной строки — `Dspring.profiles.active=development` для активации профиля всех контекстов приложений, работающих на виртуальной машине Java. Используя любой из этих методов, можно указать несколько профилей для активации, разделенных запятыми (`<param-value>profile1, profile2</param-value>`, — `Dspring.profiles.active=profile1,profile2`). Имя профиля не может содержать запятые.

Также можете активировать один или несколько профилей программным путем, вызывая метод `setActiveProfiles` на экземпляре `ConfigurableEnvironment`:

```
configurableEnvironment.setActiveProfiles("development");
configurableEnvironment.setActiveProfiles("profile1",
                                           "profile2");
```

Использование этого метода влияет на контекст приложения, содержащий среду и другие дочерние контексты приложений. Более распространено использование параметров инициализации контекста, сервлета или свойств командной строки. Программные настройки профилей полезны для тестирования интеграции.

Еще одна деталь, которую нужно понять, состоит в том, что аннотация `@Profile`, так же как `@Component`, может служить мета-аннотацией. Это означает, что можно создавать свои собственные аннотации, которые служат аннотациями `@Profile`. В качестве примера рассмотрим следующую аннотацию.

```
@Documented
@Retention(value=RetentionPolicy.RUNTIME)
@Target(value={ElementType.TYPE, ElementType.METHOD})
@Profile("development")
public @interface Development { }
```

Пользовательская аннотация `@Development` имеет ту же силу, что и аннотация `@Profile` с именем профиля «development». Теперь использование `@Development` эквивалентно использованию `@Profile` («development»). Это особенно полезно, когда необходимо избежать засорения кода именами строкового литерала с опечатками, которые трудно заменить, при необходимости изменить имя профиля.

При использовании профилей определения bean для решения определенных задач, или по-необходимости, нужно учитывать некоторые важные соображения:

- Существует ли более простой способ добиться того же результата? Если одни и те же компоненты создаются в нескольких профилях, но с разными настройками, возможно, для достижения этой цели следует использовать источник свойств и файлы свойств. Использование профилей в таком случае становится антипаттерном, добавляя большую сложность, чем ценность.
- Какие типы beans присутствуют в профилях? В большинстве случаев два разных профиля должны в основном иметь одни и те же компоненты, но возможно, разные реализации или с большей частью отладки, профилирования или отчетности. Как правило, QA и профили производства должны быть почти идентичными, при этом иметь различие между профилями разработки и QA. В противном случае можно не протестировать все необходимое.
- Каковы последствия для безопасности использования профилей определения bean? Никогда не используйте профили для управления аспектами безопасности приложения. Поскольку конечные пользователи могут активировать или деактивировать профили, просто включив или отключив свойство командной строки JVM, они могут легко обойти ограничения безопасности, определенные таким образом. Один очевидный антипаттерн в этом случае — отключить все проверки лицензий на продукт в профиле разработки, но включить их в QA и профилях производства. Тогда умный пользователь может избежать покупки приложения, просто переключившись с профиля производ-

ства на профиль разработки. Есть способы избежать этого, такие как удаление классов во время сборки или использование Java SecurityManager, но лучше всего просто избежать этой ситуации.

Аннотация `@RequestMapping`

`@RequestMapping` — чрезвычайно мощный инструмент в наборе инструментов Spring, он позволяет сопоставлять запрос с URL-адресом, заголовками запроса `Content-Type` или `Accept`, методами HTTP-запроса, наличием или значением указанных параметров запроса, заголовков или любой их комбинацией.

С `@RequestMapping` больше не нужны операторы множественного выбора или условного перехода в методе `doGet` или аналогичные методы в сервлете. Вместо этого запросы автоматически направляются на правильный контроллер и метод от Вашего имени. Через различные атрибуты аннотации `@RequestMapping` сопоставляется запрос с правильным контроллером и методом. Отображенный метод может иметь любое имя, количество различных аргументов и один из многочисленных поддерживаемых типов возвращаемых данных.

Аннотация `@RequestMapping` ограничивает сопоставление запросов с определенным методом. Вы можете аннотировать `@RequestMapping` только метод контроллера или класс контроллера и его методы. Размещение аннотации, как в классе контроллера, так и его методах устанавливает определенные правила наследования и приоритета относительно сопоставлений. Некоторые атрибуты `@RequestMapping`, установленные

для класса контроллера, наследуются и добавляются к тем, которые установлены для метода, тогда как другие, установленные для метода, переопределяют те, которые установлены для класса. Таким образом, важно понимать назначение каждого атрибута, и будут ли значения в методе добавлять или переопределять значения в классе.

Учитываются только `@RequestMapping`, указанные в классе контроллера, а не те, которые указаны в его суперклассах. Таким образом, нельзя аннотировать абстрактный класс и иметь производные классы, наследующие отображения абстрактного класса.

Ограничения URL

Мы уже использовали `@RequestMapping`, чтобы ограничить то, на какие запросы метод может реагировать, основываясь на своих URL-адресах. Используя атрибут `value` (неявно или явно, используя другие атрибуты), можно указать любой URL-шаблон `Ant`. Это более гибко, чем сопоставление URL-адресов сервлетов (которые могут начинаться с конца или заканчиваться шаблоном и не могут иметь несколько подстановочных знаков или подстановочных знаков в середине URL-адреса). Сопоставление URL-адресов контроллера `DispatcherServlet` построено с использованием сопоставления URL-адреса сервлетов, контроллера (если применимо) и методов, все они разделены прямыми косыми чертами. Рассмотрим следующие методы, сопоставленные с разными URL-адресами:

```

@RequestMapping("viewCard")
public String viewCard(...) { ... }

@RequestMapping("createCard")
public String createNewCard(...) { ... }

@RequestMapping("listCards")
public String listCards(...) { ... }

```

В этом случае, если Вы сопоставляете [DispatcherServlet](#) с корнем контекста (/), относительные URL-адреса для этих методов становятся */viewCard*, */createCard* и */listCards* соответственно. Предположим, что вместо этого Вы сопоставляете [DispatcherServlet](#) для */service/**, URL-адреса этих методов становятся */service/viewCard*, */service/createCard* и */service/listCards* соответственно.

Если Вы обнаружите, что многие URL-адреса в контроллере имеют общий элемент — можете использовать наследование сопоставления для уменьшения избыточности:

```

@RequestMapping("card")
public class CardController {
    @RequestMapping("viewCard")
    public String viewCard(...) { ... }

    @RequestMapping("createCard")
    public String createNewCard(...) { ... }

    @RequestMapping("listCards")
    public String listCards(...) { ... }
}

```

В этом случае, URL-адреса метода становятся */card/viewCard*, */card/createCard* и */card/listCards*, если *DispatcherServlet* сопоставляется с корнем контекста. Аналогично, URL-адреса становятся */servie/card/viewCard*, */servie/card/createCard* и */servie/card/listCards*, если *DispatcherServlet* сопоставляется с */servie/**.

Важным аспектом сопоставлений URL-адресов является то, что если запрос соответствует нескольким различным сопоставлениям URL-адресов, выигрывает наиболее конкретное сопоставление. Таким образом, рассмотрим следующие сопоставления:

```
@RequestMapping("view/*")
public String viewAll(...) { ... }

@RequestMapping("view/*.json")
public String viewJson(...) { ... }

@RequestMapping("view/id/*")
public String view(...) { ... }

@RequestMapping("view/other*")
public String viewOther(...) { ... }
```

Многие URL-адреса потенциально могут сравниться с одним из следующих методов:

- URL */view/other.json* может соответствовать методам *viewAll*, *viewJson* или *viewOther*, но *viewOther* более определен, поэтому он перенаправляется туда.
- */view/id/anything.json* также будет соответствовать *viewAll*, *viewJson* или *view*, но он соответствует *view*.

- Поскольку `viewAll` имеет наименее специфическое сопоставление любого из этих методов, только запросы, которые не соответствуют ни одному из других методов, перенаправляется в `viewAll`.

Используя этот метод, можем создать метод `catchall`, который улавливает все запросы, которые не сопоставляются с методом контроллера:

```
public class HomeController {  
    @RequestMapping(value="/*")  
    public String catchAll(...) { ... }  
}
```

Однако не рекомендуется это использовать. Когда пользователи переходят на недопустимый URL-адрес сайта, просмотр домашней страницы сайта вместо страницы с ошибкой 404 может запутать. Если Spring Framework не находит методов, соответствующих запросу, он автоматически отвечает ошибкой 404. Если нужно настроить эту страницу с ошибкой, можно сделать это, используя элемент описания дескриптора развертывания `<error-page>`.

Атрибут `value @RequestMapping` также может принимать массив URL-адресов для сопоставления. Таким образом, можно сопоставить несколько URL-адресов метода. В примере метод `home` отвечает на URL `/`, `/home` и `/dashboard`:

```
@RequestMapping({"/", "home", "infostand"})  
public String home(...) { ... }
```

Хотя этого строго не требуется, обычно всегда используется атрибут `value`, даже если используются другие атрибуты. Например, сопоставление метода контроллера с запросами `POST` без указания URL-адреса редко бывает целесообразно.

Ограничения методов HTTP-запроса

Также можно ограничить соответствие запросов с методами контроллера при помощи HTTP-методов. Атрибут метода `@RequestMapping` принимает одну или несколько констант `enum.springframework.web.bind.annotation.RequestMethod`. Если сопоставление метода контроллера включает одно или несколько значений для атрибута метода, запрос сопоставляется с этим методом контроллера, только если HTTP-метод запроса соответствует одной из указанных констант. Особенно полезно сопоставить один и тот же URL-адрес с различными методами управления в зависимости от HTTP-метода, как в следующем примере:

```
@RequestMapping("profile")
public class ProfileManagementController {
    @RequestMapping(value="add",
                    method=RequestMethod.GET)
    public String addForm(...) { ... }
    @RequestMapping(value="add",
                    method=RequestMethod.POST)
    public View addSubmit(...) { ... }
}
```

URL-адрес `/profile/add` соответствует методам `addForm` и `addSubmit`. `GET` запрашивает этот URL-адрес

для метода `addForm`, тогда как `POST` запрашивает путь к методу `addSubmit`. Запрос на тот же URL-адрес с использованием любого другого HTTP-метода отклоняется. На самом деле, лучше всего всегда указывать поддерживаемый HTTP-метод для каждого сопоставления. Это затрудняет компрометацию приложения, а также становится очевидным, когда форма которая должна быть отправлена в виде запроса `POST` (логина), случайно отправляется с запросом `GET`. `RequestMethod` поддерживает `OPTIONS`, `HEAD`, `GET`, `POST`, `PUT`, `DELETE`, `PATCH` и `TRACE`.

Важно понимать, что при унаследовании от класса, атрибут метода является ограничением на обоих уровнях. HTTP-метод запроса сначала проверяется на предмет ограничений класса. Если он пропускает эти ограничения, затем он проверяется на ограничения уровня метода. Таким образом, следующий `ProfileManagementController` семантически совпадает с предыдущей версией.

```
@RequestMapping(value="profile",
                 method={RequestMethod.GET,
                        RequestMethod.POST})

public class ProfileManagementController {
    @RequestMapping(value="add",
                    method=RequestMethod.GET)

    public String addForm(...) { ... }
    @RequestMapping(value="add",
                    method=RequestMethod.POST)

    public View addSubmit(...) { ... }
}
```

Ограничения параметров запроса

Используя атрибут `params` аннотации `@RequestMapping` можно указать одно или несколько выражений параметров, которые должны иметь значение `true`. Выражение «`myParam=myValue`» указывает, что параметр запроса `myParam` должен присутствовать, а его значение должно быть равно «`myValue`», тогда как «`myParam!=myValue`» указывает, что параметр запроса `myParam` не равен «`myValue`». Также можно использовать выражение «`myParam`», то есть параметр запроса `myParam` должен присутствовать с любым значением (в т.ч. пустым) и «`!myParam`» — параметр запроса `myParam` не должен присутствовать (пустые значения не допускаются). Если запрос не соответствует всем выражениям, указанным в атрибуте `params`, он не сопоставляется с методом контроллера.

Следующее сопоставление запросов сужает соответствие таким образом, что метод вызывается только в том случае, если параметр `student` присутствует с любым значением, а также присутствует параметр `confirm` равен `true`:

```
@RequestMapping(value="master",  
                 params={"student", "confirm=true"})  
public String master(...) { ... }
```

Ограничения параметров наследуются, подобно ограничениям методов HTTP-запроса. Spring Framework сначала проверяет ограничения параметров для класса контроллера, а затем для метода. Он отображает запрос, только если тот передает оба набора ограничений.

Ограничения заголовка запроса

Ограничения заголовка с использованием атрибута `headers @RequestMapping` работают почти одинаково с ограничениями параметров, включая способ их наследования. Можно указать значения или наличие выражения для любого заголовка и отменить эти выражения используя восклицательный знак, как и с ограничениями параметров.

Ограничения заголовка имеют дополнительную функцию, которая позволяет указать маски значений для заголовков медиа-типа. Таким образом, следующее сопоставление запросов соответствует только запросам, которые содержат заголовок `X-Client` и у которых `Content-Type` является любым текстовым типом. Обратите внимание, что совпадение имени заголовка не зависит от регистра.

```
@RequestMapping(value="student",
                 headers={"X-Client", "content-type=text/*"})
public Student student(...) { ... }
```

Ограничения типа контента

Можно дополнительно ограничить сопоставление запросов, используя тип содержимого запроса или ожидаемого ответа на запрос (или обоих). Хотя любое из этих ограничений возможно с использованием атрибута `headers`, удобными являются атрибуты `consumes` и `produces`.

Атрибут `consumes` принимает один или несколько типов содержимого MIME¹, которые должны соответство-

¹ Типы данных, которые могут быть переданы посредством сети интернет с применением стандарта MIME

вать заголовок запроса **Content-Type**. Таким образом, он определяет, какой тип содержимого может использовать метод. Аналогично, атрибут **generate** принимает один или несколько типов содержимого **MIME**, которые должны соответствовать заголовку **Accept**. Таким образом, он определяет типы содержимого, которые может выполнить метод, чтобы **Spring** мог определить, соответствуют ли они тем типам контента, которые клиент ожидает в ответ. Следующее сопоставление запросов соответствует только запросам с **application/json** или **text/json** **Content-Type** и заголовком **Accept**, содержащим **application/json** или **text/json**.

```
@RequestMapping (value="test",
                  consumes={"text/json", "application/json"},
                  produces={"text/json", "application/json"})
public Test test(...) { ... }
```

Если **@RequestMapping** указан как в классе, так и в методе — атрибуты метода **consumes** и **produces** переопределяют те, которые указаны в классе. Другими словами, значения для класса игнорируются, если для метода также указаны значения.

Определение параметров метода контроллера

Методы контроллера могут иметь любое количество параметров разных типов. **Spring Framework** чрезвычайно гибкий в отношении количества и типов параметров. Простейшие методы могут не иметь параметров вовсе, тогда как сложные методы могут иметь их очень много. **Spring** определяет назначение этих параметров и обе-

спечивает правильное значение во время вызова. Кроме того, некоторая простая конфигурация может расширить типы параметров, которые определяет Spring.

Spring может предоставить аргументы методов для многочисленных типов параметров, связанных с API Servlet. Spring гарантирует что значения, переданные этим параметрам, никогда не являются нулевыми. Методы могут не указывать ни одного, несколько или все эти типы параметров:

Параметр	Назначение
HttpServletRequest	Для использования свойств запроса
HttpServletResponse	Для управления ответом
HttpSession	Для управления объектом сеанса HTTP
InputStream или Reader (но не оба)	Для чтения тела запроса, при окончании работы с ним не нужно закрывать этот объект
OutputStream или Writer (но не оба)	Для записи в тело ответа, при окончании работы с ним не нужно закрывать этот объект
Java.util.Locale	Который идентифицировал клиент, используется для локализации (или локали по умолчанию, если ничего не указано).
org.springframework.web.context.request.WebRequest	Для использования свойств запроса и управления объектом HTTP-сессии без непосредственного использования API-интерфейса Servlet. Не используется, если в том же методе есть параметры типов HttpServletRequest, HttpServletResponse или HttpSession.

Аннотированные свойства запроса

Для того чтобы указать, что значение параметра метода должно быть взято из некоторого свойства запроса можно использовать несколько аннотаций параметров

(которые находятся в пакете *org.springframework.web.bind.annotation*). В большинстве случаев параметр, отмеченный одной из этих аннотаций, может быть любого примитивного типа или типом: *String*, *Class*, *File*, *Locale*, *Pattern*, *java.util.Properties* или *java.net.URL*, массивом или любой коллекцией. Spring автоматически преобразует значение в этот тип, если это возможно. Также можно зарегистрировать свой собственный *java.beans.PropertyEditor* или *org.springframework.core.convert.converter.Converter* из Spring Framework для обработки других типов.

Аннотирование *@RequestParam* указывает, что параметр аннотированного метода должен быть получен из именованного параметра запроса. Вы указываете имя параметра запроса, используя атрибут *value* (неявно или явно). По умолчанию эта аннотация указывает, что требуется параметр запроса, и запрос не будет отображаться без него.

Можно установить требуемый атрибут в *false*, чтобы сделать параметр запроса необязательным, и в этом случае если запрос не содержит этот параметр — значение аргумента метода будет *null*. Также можно использовать атрибут *defaultValue*, чтобы указать не нулевое значение, если в запросе отсутствует параметр. Следующий метод принимает требуемый параметр запроса *id*, необязательный параметр *name* со значением по умолчанию *null* и необязательный параметр *key* с пустой строкой в качестве значения по умолчанию. Имена параметров запроса чувствительны к регистру.

```
@RequestMapping("student")
public String student(@RequestParam("id") long studentId,
    @RequestParam(value="name", required=false) String name,
    @RequestParam(value="key", defaultValue="") String key)
{ ... }
```

Можно получить все значения параметров запроса в `Map`, аннотируя один параметр типа `Map<String, String>` или `org.springframework.util.MultiValueMap<String, String>` с помощью `@RequestParam`.

`@RequestHeader` работает подобно `@RequestParam` в предоставлении доступа к значениям заголовка запроса. Он задает требуемый (по умолчанию) или необязательный заголовок запроса для использования в качестве значения соответствующего параметра метода. Поскольку HTTP-заголовки также могут иметь несколько значений, необходимо использовать массив или коллекцию параметров. Для получения всех значений заголовков запроса можно аннотировать `@RequestHeader` единственный параметр типа `Map<String, String>`, `MultiValueMap<String, String>` или `org.springframework.http.HttpHeaders`. Из следующих трех методов `foo` получает два заголовка по имени, а `bar` и `baz` — все заголовки. Имена заголовков не чувствительны к регистру.

```
@RequestMapping("test1")
public String test1(@RequestHeader("Content-Type")
    String contentType,
    @RequestHeader(value="X-Custom-Header",
        required=false)
```

```

Date customHeader)
{ ... }
@RequestMapping("test2")
public String test2(@RequestHeader
                    MultiValueMap<String, String>
                    headers)
{ ... }
@RequestMapping("test3")
public String test3(@RequestHeader HttpHeaders headers)
{ ... }

```

В Spring Framework URL-адреса не должны иметь статических значений. Вместо этого URL-адрес может содержать шаблон, указывающий, что часть URL-адреса является переменной и ее значение ожидается во время выполнения. Переменные шаблона URI намного удобнее для поисковых систем и являются частью стандарта веб-сервисов RESTful. Следующий фрагмент кода демонстрирует указание переменной шаблона URI в сопоставлении URL-адресов и использование значения переменной шаблона в качестве значения параметра метода с помощью [@PathVariable](#).

```

@RequestMapping(value="student/{studentId}",
                method=RequestMethod.GET)
public String student(@PathVariable("studentId")
                    long studentId) { ... }

```

По умолчанию Spring позволяет значению переменной шаблона содержать любой символ, кроме точки (регулярное выражение `[^\.]*`). Можно настроить это поведение, указав в URL-адресе регулярное выражение, для того

чтобы запретить допустимые символы или разрешить использование точек. Следующее соответствие подобно предыдущему, но ограничивает переменную шаблона `userId` только числовыми символами. URL-адрес запроса, который не содержит эту шаблонную переменную или содержит недопустимые символы, не будет сопоставлен с методом контроллера.

```
@RequestMapping(value="student/{studentId:\\d+}",
                 method=RequestMethod.GET)
public String student(@PathVariable("studentId")
                    long studentId) { ... }
```

URL-адрес может содержать несколько шаблонов переменной, и каждый из них может иметь связанный параметр метода. Кроме того, можно аннотировать один параметр метода типа `Map<String, String>` как `@PathVariable` и он будет содержать все значения шаблонной переменной URI из URL-адреса.

```
@RequestMapping(value="func/{val1}/proc/{val2}")
public String funcProc(@PathVariable("func") String func,
                      @PathVariable("proc") long proc)
{ ... }

@RequestMapping(value="proc/{val1}/func/{val2}")
public String procFunc(@PathVariable
                     Map<String, String> variables)
{ ... }
```

RFC 3986 определяет концепцию параметров пути URI. В отличие от параметров запроса, которые входят

в строку запроса и принадлежат к URI в целом, параметры пути относятся к определенным сегментам пути.

Например, в URL-адресе *http://www.academy.com/building/15; floor=6; room=23/tutor* параметры **floor** и **room** относятся к сегменту пути 15 (**building ID**). Спецификация сервлета API требует, чтобы параметры пути были удалены из URL до сопоставления с сервлетом. Для URL-адреса отеля это означает, что URL-адрес, используемый для соответствия запросу сервлету, фактически */building/15/tutor*. *Spring Framework* делает то же самое, когда устанавливает соответствия запросам меток контроллера. Кроме того, Spring предоставляет аннотацию **@MatrixVariable** для извлечения параметров пути в качестве параметров метода. Подобно **@RequestParam**, **@MatrixVariable** имеет атрибуты **value**, **required** и **defaultValue**.

```
@RequestMapping("building/{ buildingId:\\d+}/tutor")
public String tutorForRoom(@PathVariable("buildingId")
    long buildingId,
    @MatrixVariable("floor") short floorNumber,
    @MatrixVariable("room") short roomNumber)
{ ... }
```

Это сопоставление соответствует предыдущему URL-адресу отеля. Поскольку параметры пути игнорируются при сопоставлении запросов к URL, отображение фактически не объявляет параметры. **@MatrixVariable** также имеет атрибут **pathVar** для указания, к какой переменной шаблона URI (**@PathVariable**) принадлежит параметр пути URI. Однако атрибут **pathVar** является необязательным.

Если только один сегмент пути в URL-адресе может содержать параметр пути с заданным именем — нет причин указывать `pathVar`. Если более одного сегмента в URL-адресе содержит параметр пути с заданным именем — необходимо указать `pathVar`, чтобы определить на какой параметр ссылаетесь. Следующий пример семантически такой же, как предыдущий, но более точный:

```
@RequestMapping("building/{buildingId:\\d+}/tutor")
public String tutorForRoom(@PathVariable("buildingId")
    long buildingId,
    @MatrixVariable(pathVar="buildingId",
        value="floor")
    short floorNumber,
    @MatrixVariable(pathVar="buildingId",
        value="room")
    short roomNumber)
{ ... }
```

Объекты формы ввода

При работе с HTML формами представление клиента может содержать десятки или более полей. Рассмотрим форму регистрации пользователя, в которой могут быть имя пользователя, пароль, подтвержденный пароль, адрес электронной почты, имя и фамилия, номера телефонов, адреса и прочее.

Хотя `@RequestParam`, значимый инструмент — метод содержащий десятки параметров громоздкий и делает модульное тестирование уязвимым. Вместо этого Spring Framework позволяет указать объект формы, также называемый объектом команды, как один из параметров

метода контроллера. Объекты формы — это простые POJO с методами доступа и мутаторами. Им не нужно реализовывать какой-либо специальный интерфейс, и нам не нужно отмечать параметр метода контроллера специальной аннотацией, чтобы Spring распознал его как объект формы.

```
public class CardForm {
    private String topic;
    private String message;
    // other fields, and mutators and accessors
}

@RequestMapping("card")
public class CardController {
    @RequestMapping(value="create",
                    method=RequestMethod.POST)
    public String create(CardForm form) { ... }
}
```

В этом примере Spring в классе `CardForm` просматривает `set` методы. Затем он сопоставляет параметры запроса со свойствами объекта формы, используя имена параметров. Например, вызывается метод `setTopic` со значением параметра запроса `topic` и вызывается метод `setMessage` со значением параметра запроса `message`. Если параметр запроса не соответствует свойствам объекта формы, он просто игнорируется. Аналогично, если у Вас есть свойства объекта формы, которые не сопоставляются с параметрами запроса, они также игнорируются. Строковые значения параметров запроса преобразуются в типы свойств их назначения, используя зарегистрированные

PropertyEditor и **Converter** (те же, которые преобразуют аргументы метода контроллера).

Spring также может автоматически проверять детали объекта формы, что означает, что можно избежать внедрения логики проверки в метод контроллера. Если включена проверка валидации, и Вы поместили параметр объекта формы **@javax.validation.Valid**, параметр, который за ним следует, может иметь тип **org.springframework.validation.Errors** или **org.springframework.validation.BindingResult**. Когда Spring вызывает метод, результатом процесса проверки является значение параметра. Если параметр, следующий за параметром объекта формы, не является объектом **Errors** или **BindingResult** и проверка объекта формы не выполняется, Spring бросает исключение **org.springframework.web.bind.MethodArgumentNotValidException**.

```
@RequestMapping(value="create",
                  method=RequestMethod.POST)
public String create(@Valid CardForm form,
                    BindingResult validation) { ... }
```

Преобразования тела запроса и сущностей

До сих пор инструменты, которые мы видели, подобны **@RequestMapping**, касались методов веб-запросов **GET** и **POST**: заголовков, параметров URL-запроса и тела запроса **x-www-form-urlencoded** (представления форм). Однако запросы **POST** и **PUT** могут содержать данные в форматах, отличных от **x-www-form-urlencoded**. Например, в веб-сер-

весе [RESTful](#) запрос [POST](#) или [PUT](#) может содержать тело запроса в формате JSON или XML для представления более сложных данных, чем с помощью [x-www-form-urlencoded](#). Тело запроса также может содержать двоичные, базовые 64-кодированные данные или практически любой формат, который может понять клиент и сервер. Когда эти данные представляют какой-то объект, его часто называют объектом запроса или HTTP объектом. Используя аннотацию [@RequestBody](#), Spring автоматически преобразует объект запроса в параметр метода контроллера.

```
public class ProfileForm
{
    private String name;
    private String homeAddress;
    private String mobileNumber;
    // other fields, and mutators and accessors
}

@RequestMapping("profile")
public class ProfileController
{
    @RequestMapping(value="update",
                    method=RequestMethod.POST)
    public String update(@RequestBody Profile profile)
    { ... }
}
```

По умолчанию требуются параметры [@RequestBody](#), но можно сделать их необязательными, используя требуемый атрибут. Объекты запроса автоматически преобразуются Spring с помощью преобразователей HTTP-сообщений. Параметры [@RequestParam](#) и [@RequestHeader](#), являются

простыми типами, которые автоматически преобразуются из своего строкового представления в целевой тип с использованием `PropertyEditor` и `Converter`. Объекты запроса должны иметь специализированные конвертеры сообщений, которые могут понимать как исходный формат (JSON, XML, двоичный), так и целевой (POJO или другой сложный объект).

Инициировать проверку содержимого можно, отметив параметры метода `@RequestBody` как `@Valid`, а также, необязательно, указав параметры `Error` или `BindingResult` следом за ними. Опять же, валидация выдает исключение `MethodArgumentNotValidException`, если проверка не выполняется, и нет параметра `Error` или `BindingResult`, следующего за параметром сущности запроса.

```
@RequestMapping(value="update",
                  method=RequestMethod.POST)
public String update(@Valid @RequestBody Profile profile,
                    BindingResult validation) { ... }
```

Вместо использования `@RequestBody` метод может принимать аргумент типа `org.springframework.http.HttpEntity<?>`. Этот тип обеспечивает доступ к заголовкам запроса (`HttpHeader`) и к телу запроса в качестве указанного аргумента типа. Таким образом, можно заменить предыдущий метод следующим:

```
@RequestMapping(value="update",
                  method=RequestMethod.POST)
public String update(HttpEntity<Profile> request) { ... }
```

Однако, используя параметр `HttpEntity`, проверка объекта тела запроса не происходит автоматически, поэтому нет никакой реальной выгоды от его использования.

В среде браузера почти всегда используется многостраничный запрос для загрузки файлов вместе с регулярными данными формы. В этих случаях запрос содержит `Content-Type multipart/form-data` и часть для каждого поля формы. Каждая часть запроса, разделенная определенной границей, имеет `Content-Disposition` из `form-data` и имя, соответствующее имени ввода формы. Если поле формы является стандартным, его часть просто содержит данные из него. Если поле формы является полем типа файл, его часть имеет тип `Content-Type`, соответствующий файлу MIME-типа и содержимое файла в двоичном коде, при необходимости.

Пометка параметра метода контроллера с помощью `@RequestParam` приводит к тому, что `Spring` получает это значение из параметра URL запроса, параметра тела `POST` запроса `x-www-form-urlencoded` или из части многостраничного запроса с этим именем. Однако доступные `PropertyEditor` и `Converter` преобразуют эти параметры из строковых значений в простые типы. Загрузка файлов не является ни строковыми, ни простыми типами.

Аннотацией `@RequestPart` можно пометить любой параметр метода контроллера, который должен поступать из части многопрофильного запроса и должен быть преобразован с помощью конвертера HTTP сообщений вместо `PropertyEditor` или `Converter`. Как и многие дру-

гие аннотации, она также имеет обязательный атрибут, который по умолчанию имеет значение `true`.

```
@RequestMapping(value="form/upload",
                  method=RequestMethod.POST)
public String upload(@RequestParam("clientname")
                    String clientname,
                    @RequestPart("upload")
                    Part upload) { ... }
```

Этот метод может ответить на ранее продемонстрированный многопрофильный запрос, содержащий один файл. Это работает, потому что `Spring` имеет встроенный HTTP-конвертер сообщений, который распознает части файлов. В дополнение к типу `javax.servlet.http.Part` можно конвертировать загрузку файлов в `org.springframework.web.multipart.MultipartFile`. Если в поле файла разрешена загрузка нескольких файлов, как показано во втором многостраничном запросе, просто используйте массив или коллекцию `Part` или `MultipartFile`.

```
@RequestMapping(value="form/upload",
                  method=RequestMethod.POST)
public String upload(@RequestParam("clientname")
                    String clientname,
                    @RequestPart("uploads")
                    List<MultipartFile> uploads) { ... }
```

Вы не ограничены использованием многофайловой загрузки файлов непосредственно в качестве параметров метода контроллера. Объекты формы могут содержать поля `Part` или `MultipartFile`, а `Spring` автоматически знает,

что он должен получать значения из частей файла и соответствующим образом преобразовывать значения.

```
public class UploadForm {
    private String name;
    private List<Part> uploads;
    // mutators and accessors
}

@RequestMapping(value="form/upload",
                method=RequestMethod.POST)
public String upload(UploadForm form) { ... }
```

До сих пор Вы видели многопроцессорную обработку, используемую исключительно для загрузки файлов, но для обработки запросов **Spring** загрузка файлов не является единственным поддерживаемым многочастным содержимым. Вместо **multipart/form-data** запрос может быть **multipart/mixed**. Затем его части могут содержать любой контент, включая объекты запроса в JSON, XML, или двоичной формы. Spring может затем преобразовать эти части в любой тип, для которого он имеет соответствующий HTTP-конвертер сообщений. Можно использовать проверку с нефайловыми частями, как с параметрами **@RequestBody** и объектами формы.

```
@RequestMapping(value="update",
                method=RequestMethod.POST)

public String update(@Valid @RequestPart("profile")
                    Profile profile,
                    BindingResult validation) { ... }
```

Следует отметить, что использование множественных запросов, таким образом, чрезвычайно необычно. Как правило, запросы объектов имеют одночастное тело, многопользовательские запросы поступают из браузеров, загружающих файлы.

Типы моделей

Методы контроллера могут иметь один не аннотированный параметр типа `Map<String, Object>`, `org.springframework.ui.ModelMap` или `org.springframework.ui.Model`. Параметр метода одного из этих типов представляет модель, которую **Spring** передает представлениям для рендеринга, и при необходимости во время выполнения метода можно добавлять к ней любые атрибуты.

Spring Framework необычайно гибкий в отношении того, какие параметры метода может указывать метод `@RequestMapping`. Аналогично, **Spring** довольно гибок в типах, которые может возвращать метод контроллера. Хотя параметры метода связаны с содержимым запроса, возвращаемый тип обычно связан с ответом. Например, возвращаемый тип `void` сообщает **Spring**, что метод обрабатывает запись ответа вручную, и поэтому **Spring** не обрабатывает дальнейшие запросы после того, как метод будет возвращен. Но, более типично, когда метод контроллера возвращает некоторый тип (иногда с аннотацией), указывающий, как Spring должен отвечать на запрос.

Методы контроллера могут возвращать `Map<String, Object>`, `ModelMap` или `Model`. Это альтернатива указанию одного из этих типов в качестве параметра метода.

Spring может распознавать возвращаемый тип в качестве модели и автоматически определять представление с помощью настроенного `org.springframework.web.servlet.RequestToViewNameTranslator` (по умолчанию используется `org.springframework.web.servlet.view.DefaultRequestToViewNameTranslator`).

Типы представлений

Чтобы указать Spring использовать конкретное представление для визуализации ответа, методы контроллера могут иметь несколько возвращаемых типов. Интерфейс `org.springframework.web.servlet.View` (или любой класс, реализующий `View`) указывает, что метод возвращает явный объект представления. После того как метод вернется, обработка запроса передается в это представление. Spring Framework предоставляет десятки реализаций `View` (например, `RedirectView`), или можно создать свои собственные. Методы контроллера могут возвращать строку, указывающую допустимое имя представления. Методы могут возвращать `org.springframework.web.servlet.ModelAndView`. Этот класс предоставляет возможность возвращать `View`, тип модели, или имя представления типа `String` и тип модели одновременно.

Сущности тела ответа

Подобно тому, как тело запроса может содержать HTTP-объекты (объекты запроса), тело ответа также может содержать HTTP-объекты (объекты ответа). Методы контроллера могут возвращать `HttpEntity<?>` или `org.`

`springframework.http.ResponseEntity<?>`, а `Spring` преобразует тело объекта в сущности в правильное содержимое ответа с использованием соответствующего конвертера HTTP-сообщений на основе согласованного типа контента. `HttpEntity` разрешает настройку тела ответа и различных заголовков. `ResponseEntity`, расширяет `HttpEntity` и добавляет возможность устанавливать код состояния ответа в виде `org.springframework.http.HttpStatus`.

```
@RequestMapping(value="card/{cardId}",
                 method=RequestMethod.GET)

public ResponseEntity<Card> getCard(@PathVariable("cardId")
                                   long cardId) {
    Card card = this.cardService.getCard(id);
    return new ResponseEntity<Card>(card, HttpStatus.OK);
}
```

Если Вы не хотите использовать `HttpEntity` или `ResponseEntity`, можно вернуть сам объект `body` и аннотировать метод с помощью `@ResponseBody`, который имеет тот же эффект. Затем также аннотировать метод с помощью `@ResponseStatus`, чтобы указать код состояния ответа (по умолчанию `200 OK`, если не используется `@ResponseStatus`).

```
@RequestMapping(value="card/{cardId}",
                 method=RequestMethod.GET)

@ResponseBody
@ResponseStatus(HttpStatus.OK)

public Card getCard(@PathVariable("cardId") long cardId) {
    return this.cardService.getCard(id);
}
```

Ранее в одном из примеров использовался возвращаемый тип `String` с аннотацией `@ResponseBody`. Когда указано `@ResponseBody`, другие обработчики возвращаемого типа игнорируются. Из-за этого `Spring` возвращает `String` в качестве фактического тела ответа, вместо визуализации представления по имени. `@ResponseBody` всегда имеет приоритет над всеми обработчиками возвращаемого значения метода контроллера.

Методы могут возвращать любой объект, а `Spring` предполагает, что этот объект должен быть атрибутом в модели. Он использует версию имени возвращаемого типа `camelCase`, как имя атрибута модели. Вслучае если метод аннотируется с помощью `@ModelAttribute`, `Spring` использует имя, указанное в аннотации как имя атрибута. В любом случае `Spring` автоматически определяет представление, используя сконфигурированный `RequestToViewNameTranslator`. В следующем примере возвращаемое значение первого метода становится атрибутом модели с именем `studentProfile`, а возвращаемое значение второго метода становится атрибутом модели с именем `student`.

```
@RequestMapping("student/{studentId}")
public StudentProfile viewStudent(
    @PathVariable("studentId") long studentId)
{ ... }

@RequestMapping("student/{studentId}")
@ModelAttribute("student")
public StudentProfile viewStudent(
    @PathVariable("studentId") long studentId)
{ ... }
```

В дополнение методы контроллера могут возвращать `java.util.concurrent.Callable<?>` или `org.springframework.web.context.request.async.DeferredResult<?>`. Эти типы указывают Spring запускать `Callable` или `DeferredResult` в отдельном потоке, используя асинхронную обработку запросов, освобождая поток для ответа на другой запрос. Обычно это делается для запросов, на ответ которых может потребоваться много времени, особенно если они тратят большую часть времени на блокировку ввода/вывода в сети или на диске. Тип параметра (возвращаемый тип) для `Callable` или `DeferredResult` должен быть типом, который Вы обычно возвращаете из метода контроллера, если Вы не хотите вызывать обработку асинхронного запроса (`View`, `String`, `ModelMap`, `ModelAndView`, `ResponseEntity` и прочее). Вы все еще можете аннотировать метод контроллера с помощью `@ResponseBody`, `@ResponseStatus` или `@ModelAttribute`, чтобы вызвать соответствующую обработку после возвращения `Callable` или `DeferredResult`.

Spring MVC. Архитектура Spring MVC

При использовании шаблона проектирования **Model-View-Controller**, контроллеры манипулируют данными в модели, которая представляет информацию, интересующую пользователя, и передает эту модель на уровень представления, что делает модель в некотором роде полезной для пользователя. Пользователь осознанно взаимодействует с уровнем представления, но также неосознанно взаимодействует с контроллером, когда он выполняет какие-то действия. Сервлет можно рассматривать как контроллер, выполняющий действия от имени пользователя по запросу. Сервлет манипулирует моделью в виде атрибутов **HttpServletRequest**, а затем перенаправляет эту модель на представление JSP для рендеринга.

Spring отделяет модель от запроса (**Map<String, Object>**, **ModelMap** или **Model**) и предоставляет представление высокого уровня, интерфейс которого может быть реализован как угодно. **InternalResourceView** и **JstlView** реализуют традиционные и JSTL-улучшенные JSP представления, соответственно. Они заботятся о передаче атрибутов модели в атрибуты запроса и пересылке запроса на соответствующий JSP. Если Вы не предпочитаете JSP, Вы можете выбрать **FreeMarkerView** (поддерживает механизм шаблонов **FreeMarker**), **VelocityView** (поддерживает

механизм шаблонов [Apache Velocity](#)) и [TilesView](#) (поддерживает механизм шаблонов [Apache Tiles](#)). Вы также можете реализовать поддержку для другого механизма шаблонов.

Если нужно преобразовать модель в JSON или в XML ответ — типичный для веб-служб [RESTful](#) и конечных точек запроса Ajax-поддержки — [Spring](#) предоставляет [MappingJackson2JsonView](#) и [MarshallingView](#). Они поддерживают JSON и XML соответственно. Также можно использовать [RedirectView](#), который отправляет ответ перенаправления [Location-header](#) с кодом состояния [302 Found](#) (для совместимости с HTTP 1.0) или [303 See Other](#) (для клиентов HTTP 1.1).

Когда метод контроллера возвращает реализацию [View](#) или [ModelAndView](#) с реализацией представления, переданного в конструктор [ModelAndView](#), [Spring](#) использует это представление напрямую и не требует дальнейшей логики для определения того, как представить свою модель клиенту. Если метод контроллера возвращает строковое имя представления или [ModelAndView](#) (со строковым именем представления), [Spring](#) должен установить это имя представления в фактический [View](#) с помощью настроенного [org.springframework.web.servlet.ViewResolver](#). Если метод возвращает модель или атрибут модели, [Spring](#) должен сначала неявно преобразовать запрос к имени представления, используя сконфигурированный [RequestToViewNameTranslator](#), а затем разрешить это именованное представление, используя [ViewResolver](#). Наконец, когда метод контроллера воз-

возвращает объект ответа, [ResponseEntity](#) или [HttpEntity](#), [Spring](#) использует согласование содержимого, чтобы определить, с каким представлением следует представлять объект.

Использование явных представлений и имен представлений

Проект Spring-MVC загружается и конфигурируется программно: [WebApplicationInitializer](#) запускает [Spring Framework](#), с двумя классами [@Configuration: RootContextConfiguration](#) и [ServletContextConfiguration](#). Для этого проекта все запросы сопоставляются с классом [HomeController](#).

Использование переадресации представления

Как правило, наиболее распространенным представлением, которое Вы явно возвращаете, является [org.springframework.web.servlet.view.RedirectView](#) для отправки клиенту другого URL-адреса. Если URL-адрес начинается с протокола ([http://](#), [https://](#) и прочее) или сетевого префикса ([//](#)), он считается абсолютным URL-адресом. Если URL-адрес относительный, предполагается, что он относится к текущему URL-адресу. Обычно [RedirectView](#) рассматривает URL-адреса, начинающиеся с косой черты, относительно URL-адреса сервера, а не URL-адреса контекста приложения. Поэтому, когда Вы создаете [RedirectView](#), важно включить контекстно-относительные абсолютные URL-адреса, как показано в первом методе в [HomeController](#).

```

@RequestMapping("/")
public View home(Map<String, Object> model) {
    model.put("infostandUrl", "infostand");
    return new RedirectView("/{infostandUrl}", true);
}

```

Второй аргумент (**true**) конструктора **RedirectView** указывает, что URL-адрес является относительным по отношению к контексту, а не серверу. Обратите внимание на шаблон замены **{infostandUrl}** в URL-адресе для **RedirectView**. **RedirectView** заменяет такие шаблоны атрибутами из модели.

В этом случае было бы проще построить представление со статической строкой URL (**new RedirectView(«/infostand», true)**), но использование этого шаблона демонстрирует важность **RedirectView**. Конечный URL-адрес бесполезен без метода обработчика, и второй метод в **HomeController** позаботится об этом.

```

@RequestMapping(value = "/infostand",
                 method = RequestMethod.GET)

public String infostand(Map<String, Object> model) {
    model.put("date", Instant.now());
    model.put("text", "Model attribute.");
    return "home/infostand";
}

```

Этот метод отвечает URL-адресу **/infostand**, возвращая имя строки для представления, добавив атрибуты даты и текста.

Конфигурация отображения представления

Чтобы сопоставить имя представления с фактическим представлением, **Spring** нужен экземпляр **ViewResolver**, который может его распознать. Создание распознавателя представлений — простая задача создания экземпляра структуры в контексте приложения диспетчера сервлетов. Для этого необходимо использовать **InternalResourceViewResolver**, который может преобразовывать имена представлений в имена JSP файлов. Это достигается с помощью метода **viewResolver** для **ServletContextConfiguration**.

```
@Bean
public ViewResolver viewResolver() {
    InternalResourceViewResolver resolver =
        new InternalResourceViewResolver();
    resolver.setViewClass(JstlView.class);
    resolver.setPrefix("/WEB-INF/jsp/view/");
    resolver.setSuffix(".jsp");
    return resolver;
}
```

В соответствии с настройкой, распознаватель представлений создает имена JSP-файлов, используя префикс */WEB-INF/jsp/view*, а затем имя представления, за которым следует *.jsp*. Важно назвать **bean viewResolver**, и поэтому также необходимо назвать **@Bean** метод **viewResolver**.

Создание представления JSP

После того как сконфигурирован распознаватель представлений и имя представления возвращаемое методом **infostand**, для приложения требуется файл */WEB-INF/jsp/*

view/home/infostand.jsp. В этом JSP-файле можно использовать те же функции, что и в любом другом JSP (скриплеты, выражения, язык выражений, теги JSP и прочее).

```
<%--@elvariable id="text" type="java.lang.String"--%>
<%--@elvariable id="date" type="java.time.Instant"--%>

<!DOCTYPE html>
<html>
  <head>
    <title>Information stand</title>
  </head>

  <body>
    Date: ${date}<br/>
    Text: ${text}
  </body>
</html>
```

В результате будет отображено два атрибута модели. Обратите внимание, что Вы получаете доступ к атрибутам модели так же, как при использовании `HttpServletRequest` для доступа к атрибутам. `JstlView` гарантирует, что все атрибуты модели будут отображаться как переменные EL для JSP. Чтобы проверить это, скомпилируйте приложение и запустите Tomcat. Перейдите по адресу `http://localhost:8080/mvc`, и затем `http://localhost:8080/mvc/infostand`. Это `RedirectView` выполняет свою работу в методе `home HomeController`. Теперь браузер должен отображать текст из модели и текущее время в формате ISO 8601. Это означает, что разрешение представления работает правильно, и представление JSP отображает модель через `JstlView`.

Использование неявных представлений с атрибутами модели

Если метод контроллера возвращает модель или атрибут модели, **Spring** должен определить, какое представление использовать автоматически. Для этого ему нужен компонент **RequestToViewNameTranslator** в сочетании с **ViewResolver**.

Конфигурация преобразования имен представлений

Обычно достаточно **DefaultRequestToViewNameTranslator**, хотя при необходимости, можно создать свой собственный **RequestToViewNameTranslator**. Он удаляет URL-адрес контекста веб-приложения и любое расширение файла в конце URL-адреса. То, что остается от URL-адреса, становится именем представления. Например, URL *http://localhost:8080/mvc/block* превращается в имя представления *block*, тогда как URL *http://localhost:8080/mvc/block/page.html* превращается в имя представления *block/page*. Как и в случае с **viewResolver** чтобы настроить **RequestToViewNameTranslator**, нужно добавить компонент в **ServletContextConfiguration**.

```
@Bean
public RequestToViewNameTranslator
    viewNameTranslator() {
    return new DefaultRequestToViewNameTranslator();
}
```

DefaultRequestToViewNameTranslator может преобразовывать имя различными способами, но здесь достаточно

конфигурации по умолчанию. Нужно указать `bean view-NameTranslator`, а также метод `@Bean viewNameTranslator`.

Использование `@ModelAttribute`

Метод `userHome` для `HomeController`, сопоставленный с URL `/user/home`, создает объект `User`, устанавливает в него некоторые значения и возвращает пользователя. Класс `User` в этом случае представляет собой простой POJO с полями `userId`, `name` и `username`, со стандартными мутаторами и аксессорами.

```
@RequestMapping(value = "/user/home",
                 method = RequestMethod.GET)
@ModelAttribute("currentUser")
public User userHome() {
    User user = new User();
    user.setUserId(12L);
    user.setUsername("ivan");
    user.setName("Ivan Ivanov");
    return user;
}
```

Аннотации `@ModelAttribute` сообщают `Spring`, что возвращаемый `User` должен быть добавлен в модель с помощью ключевого слова `currentUser`. Без аннотации, `Spring` по умолчанию использует ключ атрибута `currentUser` (на основе имени класса возвращаемого типа). Для отображения этого пользователя требуется представление, и на основе настроенного преобразования имени представления, имя представления становится `user/home`, поэтому Вы можете создать JSP `/WEB-INF/jsp/view/user/home.jsp`.

```

<!--@elvariable id="currentUser"
         type="com.academy.site.User"-->

<!DOCTYPE html>
<html>
  <head>
    <title>User Home</title>
  </head>

  <body>
    ID: ${currentUser.userId}<br />
    Username: ${currentUser.username}<br />
    Name: ${currentUser.name}<br />
  </body>
</html>

```

Теперь скомпилируйте проект, запустите Tomcat из своей IDE и перейдите к <http://localhost:8080/mvc/user/home> в браузере. Вы должны увидеть ID, имя пользователя и имя пользователя, созданное в методе `userHome` контроллера. Преобразование имени представления и разрешения представления сработали вместе, чтобы обнаружить и отобразить надлежащее представление для запроса.

Возврат объектов ответа

Получение объектов запроса и возврат объектов ответа — это задача зарезервированная для веб-сервисов **RESTful** или других автоматизированных задач. В большинстве случаев браузеры задействованы в этих ситуациях только тогда, когда приложение JavaScript создает AJAX GET или POST на сервер.

Настройка конвертеров сообщений

Когда сервер получает **POST** или **PUT** с телом запроса, это тело обычно называется сущностью запроса или **HTTP**, но также может быть названо сообщением. Чтобы были доступны методы контролера, это сообщение в любом формате должно быть преобразовано в объект Java любого типа. Это происходит на основе заголовка **Content-Type** запроса.

Конвертер сообщений org.springframework.http.converter.FormHttpMessageConverter отвечает за преобразование сообщений **x-www-form-urlencoded** в объекты формы для методов контроллера. Преобразователи сообщений работают в обоих направлениях: они могут конвертировать входящие сообщения в объекты Java, а также конвертировать объекты Java в исходящие сообщения. Они работают по простому принципу распознаваемых типов контента **MIME** и целевых типов Java. Каждый конвертер имеет один или несколько поддерживаемых типов **MIME** и Java, и он может конвертировать сообщения с этими типами **MIME** в объекты Java этих типов и обратно. Это заключительная часть процесса согласования контента. Когда Spring устанавливает тип согласованного контента, он выбирает конвертер, который поддерживает тип источника и целевого типа, и использует его для входящих или исходящих объектов, или и того и другого, если это приемлемо. В одном запросе может содержаться сообщение с содержимым одного типа, в то время как сервер отвечает сообщением с содержимым другого типа.

Spring Framework автоматически создает определенные конвертеры сообщений от Вашего имени, если Вы не настроите это вручную. В большинстве случаев достаточно автоматической конфигурации. В целях демонстрации и дополнительной настройки, проект Spring-MVC настраивает конвертеры сообщений вручную. Для этого класс `ServletContextConfiguration` должен расширять `WebMvcConfigurerAdapter` и переопределять метод `configureMessageConverters`.

```
...
public class ServletContextConfiguration
    extends WebMvcConfigurerAdapter
{
    @Inject ObjectMapper objectMapper;
    @Inject Marshaller marshaller;
    @Inject Unmarshaller unmarshaller;
    @Override
    public void configureMessageConverters(
        List<HttpMessageConverter<?>> converters) {
        converters.add(
            new ByteArrayHttpMessageConverter());
        converters.add(
            new StringHttpMessageConverter());
        converters.add(new FormHttpMessageConverter());
        converters.add(
            new SourceHttpMessageConverter<>());
        MarshallingHttpMessageConverter xmlConverter =
            new MarshallingHttpMessageConverter();
        xmlConverter.setSupportedMediaTypes(
            Arrays.asList(
                new MediaType("application", "xml"),
                new MediaType("text", "xml")
            ));
    }
}
```

```

xmlConverter.setMarshaller(this.marshaller);
xmlConverter.setUnmarshaller(this.unmarshaller);
converters.add(xmlConverter);
MappingJackson2HttpMessageConverter
    jsonConverter =
        new MappingJackson2HttpMessageConverter();
jsonConverter.setSupportedMediaTypes(Arrays.
    asList(
        new MediaType("application", "json"),
        new MediaType("text", "json")
    ));
jsonConverter.
    setObjectMapper(this.objectMapper);
converters.add(jsonConverter);
}
...
}

```

`ByteArrayHttpMessageConverter`, `StringHttpMessageConverter`, `FormHttpMessageConverter` и `SourceHttpMessageConverter` — это все преобразователи, которые Spring настраивает автоматически. Они сконфигурированы здесь, в том же порядке, что и в обычном режиме. Порядок важен, потому что некоторые конвертеры имеют более широкий тип MIME и связанных типов Java, которые могут маскировать другие конвертеры.

`MarshallingHttpMessageConverter` обычно не добавляется в список преобразователей сообщений, но в этом случае он поддерживает конвертирование в объекты XML и обратно. `MappingJackson2HttpMessageConverter` обычно создается автоматически, пока `Jackson Data Processor 2` находится на пути к классам. Однако он создается по умол-

чанию без конфигурации `com.fasterxml.jackson.databind.ObjectMapper` и поддерживает только тип содержимого `MIME application/json`. Конфигурация здесь добавляет поддержку `text/json` и использует предварительно сконфигурированный `ObjectMapper`.

Компоненты `org.springframework.xml.Marshaller`, `org.springframework.xml.Unmarshaller` и `ObjectMapper` настроены и распространены во всем приложении, они создаются в `RootContextConfiguration`. Поскольку `org.springframework.xml.jaxb.Jaxb2Marshaller` является и `Marshaller`, и `Unmarshaller`, он выполняет обе зависимости.

```
@Bean
public ObjectMapper objectMapper() {
    ObjectMapper mapper = new ObjectMapper();
    mapper.findAndRegisterModules();
    mapper.configure(SerializationFeature.
        WRITE_DATES_AS_TIMESTAMPS, false);
    mapper.configure(DeserializationFeature.
        ADJUST_DATES_TO_CONTEXT_TIME_ZONE,
        false);
    return mapper;
}

@Bean
public Jaxb2Marshaller jaxb2Marshaller() {
    Jaxb2Marshaller marshaller =
        new Jaxb2Marshaller();
    marshaller.setPackagesToScan(new
        String[] { "com.academy.site" });
    return marshaller;
}
```

`ObjectMapper` указывает `Jackson` найти и зарегистрировать все модули расширения, такие как модуль поддержки

JSR 310 (Java 8 Date and Time). Затем он отключает сериализацию дат как [timestamp](#) и преобразовывает десериализованные даты в текущий часовой пояс. [Jaxb2Marshaller](#) имеет довольно простую конфигурацию — указывает, какой пакет сканировать для XML-аннотированных объектов.

Настройка согласования контента

Согласование контента — это процесс, посредством которого клиент передает серверу список предпочтительных типов содержимого ответа в порядке предпочтения, а сервер выбирает подходящий тип контента из них, или по умолчанию, если ни один не подходит. Самая большая и сложная задача согласования контента — выяснить, в каком формате клиент хочет получить свои данные. Если запрос содержит объект, этот объект также будет иметь заголовок [Content-Type](#), но не обязательно в формате, в котором клиент ожидает ответ. Заголовок [Accept](#), предназначен для указания форматов ответов, которые клиент готов принять. Ключевым моментом является то, что клиент может указывать множество приемлемых форматов и приоритет для каждого из них.

Последние версии всех основных браузеров сегодня отправляют заголовки [Accept](#), содержащие хотя бы [text/html](#). Кроме того, при использовании приложений JavaScript или клиентских веб-сервисов RESTful, клиентское приложение имеет полный контроль над заголовком [Accept](#), что делает его содержимое более надежным фактором в согласовании типа контента. Браузер редко задействован в согласовании контента, почти во всех

случаях запрос на веб-приложение попадает в одну из трех категорий:

- Если это запрос из-за того, что кто-то щелкнул ссылку или иным образом ввел адрес в адресную строку своего браузера, целью является некоторый ресурс с фиксированным типом контента, который не подлежит согласованию — HTML-страница ([text/html](#) или [application/xhtml+xml](#)), файл для загрузки и т.д.
- Если это запрос [Ajax](#) из клиентского JavaScript приложения, то оно имеет полный контроль над [Accept](#) заголовком для выполнения согласования содержимого.
- Если это клиентское приложение доступа к веб-сервисам, оно имеет полный контроль над [Accept](#) заголовком для выполнения согласования содержимого.

Консолидация контента в [Spring](#) использует многоэтапный подход для определения типа контента, который клиент хочет получить обратно. Иногда один и тот же конвертер сообщений используется для запроса и ответа. Когда запрос содержит объект запроса, конвертер входящих сообщений всегда выбирается на основе заголовка запроса [Content-Type](#). Когда ответ содержит объект ответа, конвертер исходящих сообщений выбирается следующим образом:

1. Spring ищет расширение файла в URL-адресе запроса, если он содержит расширение файла (например, [.html](#), [.xml](#), [.json](#) и т.д.), на основе этого определяется формат запроса. Если расширения файла нет или не распознано, он идет дальше.

2. Spring ищет параметр запроса с именем `format`, если нашел — использует запрошенный формат (`HTML`, `XML`, `JSON`, и т.д.). Если параметр формата не существует или не распознается, он идет дальше.
3. Spring использует заголовок `Accept`, чтобы определить желаемый формат ответа. Можно настроить весь этот процесс в соответствии с потребностями.

Можно полностью устранить один или несколько шагов и изменить то, как работает каждый из них, основываясь на том, как приложению необходимо выполнять согласование контента. Тем не менее, нельзя изменить порядок этапов. Для достижения этой цели в классе `ServletContextConfiguration` проекта `Spring-MVC` переопределим метод `configureContentNegotiation` из `WebMvcConfigurerAdapter`.

```
@Override
public void configureContentNegotiation(
    ContentNegotiationConfigurer configurer){
    configurer.favorPathExtension(true).
        favorParameter(false).parameterName("mediaType").
        ignoreAcceptHeader(false).useJaf(false).
        defaultContentType(MediaType.APPLICATION_XML).
        mediaType("xml", MediaType.APPLICATION_XML).
        mediaType("json", MediaType.APPLICATION_JSON);
}
```

Эта конфигурация позволяет проверять расширение файлов, отключает проверку параметров запроса, устанавливает имя параметра запроса в `mediaType` и обеспечивает то, что заголовок `Accept` не будет игнорирован. Руковод-

ствуясь приоритетом ручного указания поддерживаемых типов носителей отключим использование *Java Activation Framework* (JAF) — инструмент, который сопоставляет расширения файлов с типами носителей. Затем установим тип содержимого по умолчанию `application/xml` и добавим поддержку `application/xml`, а также `application/json`.

Использование `@ResponseBody`

Теперь, когда согласование содержимого настроено правильно, можем использовать его с помощью метода контроллера. Метод `getUser` `HomeController` делает именно это.

```
@RequestMapping(value = "/user/{userId}",
                 method = RequestMethod.GET)
@ResponseBody
public User getUser(@PathVariable("userId")
                   long userId) {
    User user = new User();
    user.setUserId(userId);
    user.setUsername("petr");
    user.setName("Petr Petrov");
    return user;
}
```

Обратите внимание на использование `@ResponseBody`, который вызывает настроенную нами стратегию согласования контента. Этот метод использует переменные шаблона URI и аннотацию `@PathVariable` для получения идентификатора пользователя из URL вместо параметра запроса. Теперь скомпилируйте приложение, запустите Tomcat и перейдите по адресу `http://localhost:8080/mvc/user/16`.

Вы должны увидеть пользователя, представленного в XML (рис. 9), потому что заголовок **Accept**, отправляемый Вашим браузером, включает *application/xml*, или в конфигурации согласования содержимого по умолчанию был установлен XML. Если изменить идентификатор в конце URL-адреса, XML изменится соответственно.



Рисунок 9.

Теперь перейдите по адресу *http://localhost:8080/mvc/user/16.json* — представление изменится с XML на JSON (рис. 10).

Для этого рекомендуется использовать браузер Chrome или Firefox, потому что Internet Explorer предложит загрузить файл JSON. Вам не нужно изменять сопоставление URL для метода контроллера, потому что Spring распознает расширение, игнорирует его при сопоставлении запросов и использует для согласования контента.

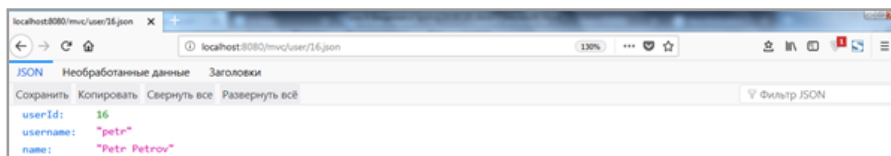


Рисунок 10.

Если изменить расширение с JSON на XML, представление вернется в XML.

Преимущества использования объектов формы

При написании браузерных веб-приложений в Spring Framework одним из самых больших удобств являются объекты формы. Использование объектов формы упрощает определение местоположения и преобразование параметров запроса в бизнес-объекты.

Продemonстрируем, как это делается с помощью простого контроллера и нескольких представлений JSP в проекте Spring-Forms. `StudentController` имеет два приватных поля и синхронизированный приватный метод для атомарной генерации идентификаторов студентов. Он также имеет метод обработчика запроса `displayStudents` для отображения списка существующих студентов.

```
@Controller
public class StudentController {
    private final Map<Long, Student>
        studentBase = new Hashtable<>();
    private volatile long studentId = 1L;
    @RequestMapping(value = "student/list",
        method = RequestMethod.GET)
    public String displayStudents(Map<String,
        Object> model) {
        model.put("studentList",
            this.studentBase.values());
        return "student/list";
    }
}
```

```

...
private synchronized long getNextStudentId() {
    return this.studentId++;
}
}

```

Класс Student — это простой класс POJO с полями `studentId`, `login` и `studentName`, а также соответствующими мутаторами и аксессорами. В представлении `/WEB-INF/jsp/view/student/list.jsp` приведена ссылка для добавления нового студента и используется тег `<c:forEach>` для циклического перехода и отображения существующих студентов.

```

<!--@elvariable id="studentList"
         type="java.util.Collection
         <com.academy.site.Student>"--%>

<!DOCTYPE html>
<html>
    <head>
        <title>Student List</title>
    </head>
    <body>
        <h2>Students</h2>
        [<a href="<c:url value="/student/add" />">
            new student</a>]
        <br />
        <br />
        <c:forEach items="${studentList}"
                 var="student">
            ${student.studentName} (${student.login})
            [<a href="<c:url value="/student/
                edit/${student.studentId}"/>">

```

```

        edit</a>]<br/>
    </c:forEach>
</body>
</html>

```

Добавление объектов формы в модель

Объект формы должен быть доступен в представлении, отображающем форму. Метод `createStudent(Map)` добавляет новый объект `StudentForm` в модель и возвращает `student/add`.

```

@RequestMapping(value = "student/add",
                method = RequestMethod.GET)
public String createStudent(Map<String,
                            Object> model) {
    model.put("studentForm", new StudentForm());
    return "student/add";
}

...
@RequestMapping(value = "student/edit/{studentId}",
                method = RequestMethod.GET)
public String editStudent(Map<String, Object> model,
                        @PathVariable("studentId")
                        long studentId) {
    Student student = this.studentBase.get(studentId);
    StudentForm form = new StudentForm();
    form.setLogin(student.getLogin());
    form.setStudentName(student.getStudentName());
    model.put("studentForm", form);
    return "student/edit";
}

```

Обратите внимание, что метод `editStudent(Map, long)` также добавляет `StudentForm` к модели, но он делает это по-другому. Сначала он извлекает редактируемый `Student`, копирует информацию этого студента в форму помещает форму в модель и затем возвращает имя `student/edit`.

Использование Spring Framework `<form>` Tags

Представления `/WEB-INF/jsp/view/student/add.jsp` и `/WEB-INF/jsp/view/student/edit.jsp` задают переменную заголовка EL и включают файл `/WEB-INF/jsp/view/student/form.jspf`.

```
<c:set var="title" value="Add Student" />
<%@ include file="form.jspf" %>
```

Предыдущие две строки кода составляют весь файл `add.jsp`, а `edit.jsp` — почти идентичен, за исключением того, что для заголовка задано значение «`Edit Student`». Файл `form.jspf` выводит форму, используя атрибут `modelForm` модели.

```
<%--@elvariable id="studentForm"
                    type="com.academy.site.StudentForm"--%>

<!DOCTYPE html>
<html>

    <head>
        <title>${title}</title>
    </head>
```

```

<body>
  <h1>${title}</h1>

  <form:form method="post"
            modelAttribute="studentForm">
    <form:label path="login">
      Login
    </form:label><br />
    <form:input path="login" />
    <br />
    <br />
    <form:label path="studentName">
      Name:
    </form:label><br />
    <form:input path="studentName" />
    <br />
    <br />
    <input type="submit" value="Save" />
  </form:form>
</body>
</html>

```

Первое, на что необходимо обратить внимание, это использование тегов в пространстве имен форм. Эта библиотека тегов в Spring Framework, представляет собой оболочку вокруг стандартных `<form>`, `<input>` и `<textarea>` полей и обеспечивает автоматическое связывание полей для формирования содержимого объекта. При создании нового студента она мало значит, однако при редактировании существующего — значения из атрибута `model` автоматически помещаются в поля формы, принадлежащие им. Это упрощает код и устраняет вопрос надлежащего экранирования значений используемых в форме. Библиотека тегов форм, как и другие библиотеки тегов,

объявляется в файле *base.jspf*, который автоматически включается во все JSP.

```
<%@ taglib prefix="form"
    uri="http://www.springframework.org/
    tags/form" %>
```

Пространство имен форм включает 14 тегов, которые инкапсулируют широкий спектр функций веб-форм. Помимо собственных атрибутов, каждый тег поддерживает все стандартные атрибуты HTML для эквивалентного тега HTML. Тег `<form:form>` — родительский тег указывает, какой объект модели будет привязан к полям формы. Не нужно указывать атрибут действия, потому что по умолчанию `<form:form>` всегда отправляет URL-адрес текущей страницы. Остальные 13 тегов привязаны к различным свойствам компонента объекта формы, указанным атрибутами пути в тегах.

- `<form:errors>` эквивалент `<span>` — относится к автоматической проверке объекта формы.
- `<form:label>` эквивалент `<label>` — указывает текст метки для поля,
- `<form:hidden>` эквивалент `<input type = "hidden">`.
- `<form:input >` эквивалент `<input type = "text">`.
- `<form:password>` эквивалент `<input type = "password">` — имеет атрибут `showPassword` (по умолчанию `false`), который указывает, должен ли отображаться пароль. Когда `showPassword` является `true`, этот тег фактически эквивалентен `<input type = "text">`

- `<form:textarea>` эквивалент `<textarea>`
- `<form:checkbox>` эквивалент `<input type = «checkbox»>` — может поддерживать несколько типов свойств, таких как `boolean`, `Boolean` и числовые типы.
- `<form:checkboxes>` вариант `<form:checkbox>` — автоматически создает несколько полей флажков от Вашего имени. Атрибут `items` используется для указания `Collection`, `Map`, или массива объектов, с помощью которых можно создавать теги. Атрибуты `ItemValue` и `itemLabel` определяют имена значений полей и свойств метки полей, для объектов в коллекции соответственно.
- `<form:radiobutton>` эквивалент `<input type = “radio”>`. Как правило, Вы связываете два или более из них с одним и тем же `path` (свойство объекта формы), а `Spring` автоматически выбирает правильный, основываясь на значении свойства.
- `<form:radiobuttons>` вариант `<form:radiobutton>` — подобен `<form:checkboxes>` и для генерации радиокнопок имеет те же атрибуты `items`, `itemValue` и `itemLabel`. Оба тега `<form:radiobuttons>` и `<form:checkboxes>` подходят для получения полей из перечислений.
- `<form:select>` эквивалент `<select>` — выпадающий список или поле множественного выбора. Работает с `<form:option>`, а также `<form:options>`. Автоматически выбирает правильную опцию, основанную на значении `path`, к которому привязан выбор.
- `<form:option>` эквивалент `<option>` — принадлежит `<form:select>`

- `<form:options>` также как и `<form:checkboxes>` и `<form:radiobuttons>`, имеет `items`, `itemValue` и `itemLabel`, которые помогают сгенерировать несколько элементов `<option>`.

Получение предоставленных данных формы

Когда студент отправляет форму, контроллер просто получает эти данные. `CreateStudent(StudentForm)` а также `editStudent(StudentForm, long)` получают предоставленные объекты `StudentForm`, автоматически конвертированные из параметров запроса посредством `FormHttpMessageConverter`

```
@RequestMapping(value = "student/add",
                  method = RequestMethod.POST)
public View createStudent(StudentForm form) {
    Student student = new Student();
    student.setStudentId(this.getNextStudentId());
    student.setLogin(form.getLogin());
    student.setStudentName(form.getStudentName());
    this.studentBase.put(student.getStudentId(), student);
    return new RedirectView("/student/list",
                           true, false);
}

...
@RequestMapping(value = "student/edit/{studentId}",
                  method = RequestMethod.POST)
public View editStudent(StudentForm form,
                        @PathVariable("studentId")
                        long studentId) {
    Student student = this.studentBase.get(studentId);
    student.setLogin(form.getLogin());
    student.setStudentName(form.getStudentName());
    return new RedirectView("/student/list", true, false);
}
```

Теперь скомпилируйте приложение, запустите Tomcat из IDE и перейдите по адресу *http://localhost:8080/forms/student/list*. Нажмите ссылку **new student** и введите логин и имя для создания нового студента. Добавьте одного или двух студентов, а затем попробуйте отредактировать их. Вы заметите, что Spring автоматически привязывает объект **StudentForm** в модели к полям формы в представлении. При представлении он автоматически преобразует параметры запроса в объекты формы.

Примеры использования Spring MVC и объектов формы

Сейчас вернемся к проекту прошлого урока Support-Service. Используем Spring Framework, переместим существующие классы в пакет *com.academy.site* и заменим сервлеты контроллерами Spring MVC.

Добавим в класс **Card** поле идентификатора обращения (**id**), а в класс **Attachment** поле **fileMimeType** и соответствующие им мутаторы и аксессоры.

Добавим класс **LoginFilter**, который выполняет проверку подлинности при каждом запросе любого метода HTTP и если пользователь не аутентифицирован, перенаправляет на экран входа в систему. Этот фрагмент кода был удален из **doGet** и **doPost** в **CardServlet**.

```
public class LoginFilter implements Filter {
    @Override
    public void doFilter(ServletRequest request,
                        ServletResponse response,
                        FilterChain chain) throws
                        IOException, ServletException {
        HttpSession session = ((HttpServletRequest)
                               request).getSession(false);
        if(session == null || session.
            getAttribute("clientId") == null) {
            ((HttpServletResponse) response).sendRedirect(
                ((HttpServletRequest) request).
                getContextPath() + "/login");
        }
    }
}
```

```

    }
    else chain.doFilter(request, response);
}
@Override
public void init(FilterConfig config) throws
    ServletException { }
@Override
public void destroy() { }
}

```

Затем программно сконфигурируем фильтры в классе **Bootstrap**. Поскольку система обращений позволяет загружать файлы в виде вложений, в классе **Bootstrap** сконфигурируем **DispatcherServlet** для поддержки множественной загрузки файлов. Класс **Bootstrap** для приложения поддержки клиентов приведен ниже:

```

public class Bootstrap implements
    WebApplicationInitializer {
    @Override
    public void onStartup(ServletContext container)
        throws ServletException {
        container.getServletRegistration("default").
            addMapping("/resource/*");
        AnnotationConfigWebApplicationContext
            rootContext =
                new AnnotationConfigWebApplicationContext();
        rootContext.register(RootContextConfiguration.
            class);
        container.addListener(new
            ContextLoaderListener(rootContext));
        AnnotationConfigWebApplicationContext
            servletContext = new
                AnnotationConfigWebApplicationContext();
    }
}

```

```

servletContext.
    register(ServletContextConfiguration.class);

ServletRegistration.Dynamic dispatcher =
    container.addServlet("springDispatcher",
        new DispatcherServlet(servletContext));

dispatcher.setLoadOnStartup(1);
dispatcher.setMultipartConfig(new
    MultipartConfigElement(null, 20_971_520L,
        41_943_040L, 512_000));

dispatcher.addMapping("/");
FilterRegistration.Dynamic registration =
    container.addFilter("loginFilter",
        new LoginFilter());

registration.addMappingForUrlPatterns(
    null, false, "/card", "/card/*");
    }
}

```

Включения поддержки **multipart** в **DispatcherServlet** недостаточно для загрузки файлов с помощью Spring MVC. Spring Framework также поддерживает более старые версии API Servlet. Напомним, что до Servlet 3.0 API Servlet не имел встроенных функций, в многопроцессорной поддержке и сторонних инструментах необходимо было выполнить загрузку файлов. Spring MVC нуждается в экземпляре **MultipartResolver**, чтобы указать, следует ли использовать многопроцессорную поддержку Servlet 3.0+ или какой-либо сторонний инструмент. Класс **Servlet-ContextConfiguration** такой же как в проекте Spring-Forms, и отличается лишь тем, что имеет дополнительный **@Bean** метод **multipartResolver**.

```
@Bean
public MultipartResolver multipartResolver() {
    return new StandardServletMultipartResolver();
}
```

Класс `RootContextConfiguration` оставляем без изменений.

Файл `index.jsp` удален и заменен на `IndexController`:

```
@Controller
public class IndexController {
    @RequestMapping("/")
    public View index() {
        return new RedirectView("/card/list",
                                true, false);
    }
}
```

`LoginServlet` стал `LoginController`. Этот код несколько изменился. Вместо методов `doGet` и `doPost`, проверяющих наличие различных параметров запроса, каждое действие заменено простым методом контроллера.

```
@Controller
public class LoginController {
    private static final Map<String, String>
        clientBase = new LinkedHashMap<>();
    static {
        clientBase.put("Ivanov", "qwert");
        clientBase.put("Petrov", "123456");
        clientBase.put("Sidorov", "master");
    }
    @RequestMapping("logout")
```

```

public View logout(HttpSession session) {
    session.invalidate();
    return new RedirectView("/login", true, false);
}
@RequestMapping(value = "login",
    method = RequestMethod.GET)
public ModelAndView login(Map<String,
    Object> model,
    HttpSession session) {
    if(session.getAttribute("clientName") != null)
        return this.getCardRedirect();
    model.put("loginFailed", false);
    model.put("loginForm", new LoginForm());
    return new ModelAndView("login");
}
@RequestMapping(value = "login",
    method = RequestMethod.POST)
public ModelAndView login(Map<String,
    Object> model,
    HttpSession session,
    HttpServletRequest request,
    LoginForm loginForm){
    if(session.getAttribute("clientName") != null)
        return this.getCardRedirect();
    if(loginForm.getClientName() == null ||
        loginForm.getPassword() == null ||
        !clientBase.containsKey(loginForm.
            getClientName()) ||
        !loginForm.getPassword().
            equals(clientBase.get(loginForm.
                getClientName())
            ))
    {
        loginForm.setPassword(null);
        model.put("loginFailed", true);
        model.put("loginForm", loginForm);
    }
}

```

```

        return new ModelAndView("login");
    }
    session.setAttribute("clientName",
        loginForm.getClientName());
    request.changeSessionId();
    return this.getCardRedirect();
}
private ModelAndView getCardRedirect() {
    return new ModelAndView(new RedirectView(
        "/card/list", true, false));
}
public static class LoginForm {
    private String clientName;
    private String password;
    public String getClientName() {
        return clientName;
    }
    public void setClientName(String clientName) {
        this.clientName = clientName;
    }
    public String getPassword() {
        return password;
    }
    public void setPassword(String password) {
        this.password = password;
    }
}
}

```

CardServlet стал **CardController**. Этот сервлет использовал шаблон действия, в котором параметр запроса **action** контролировал выполнение методов **doGet** или **doPost**. В следствии чего методы **doGet** и **doPost** стали очень громоздкими. Теперь **Spring** заботится об этом, используя

@RequestMapping, устраняя большую часть кода в этих контроллерах.

CardController использует объект формы для представления обращений, и этот объект формы использует **Spring MultipartFile** для получения легкого доступа к загруженным файлам.

```
public static class CardForm {
    private String topic;
    private String message;
    private List<MultipartFile> attachments;
    // mutators and accessors
}
```

Это делает код для метода обработчика создания намного более кратким, чем предыдущий код в **Servlet**.

```
@RequestMapping(value = "create",
                 method = RequestMethod.POST)

public View create(HttpSession session,
                  CardForm cardForm)
    throws IOException {
    Card card = new Card();
    card.setId(this.getNextCardId());
    card.setClientName((String)session.
                       getAttribute("clientName"));
    card.setTopic(cardForm.getTopic());
    card.setMessage(cardForm.getMessage());
    for(MultipartFile filePart :
        cardForm.getAttachments()) {
        Attachment attachment = new Attachment();
        attachment.setFileName(filePart.
                               getOriginalFilename());
```

```

        attachment.setFileMimeType(filePart.
            getContentType());
        attachment.setFileContents(filePart.
            getBytes());
        if((attachment.getFileName() != null &&
            attachment.getFileName().
            length() > 0) ||
            (attachment.getFileContents() !=
            null && attachment.
            getFileContents().length > 0))
            card.addAttachment(attachment);
    }
    this.cardBase.put(card.getId(), card);
    return new RedirectView("/card/view/" +
        card.getId(), true, false);
}

```

Методы **list**, **view** и **create** класса **CardController** выглядят следующим образом:

```

@RequestMapping(value = {"", "list"},
    method = RequestMethod.GET)

public String list(Map<String, Object> model) {
    model.put("cardBase", this.cardBase);
    return "card/list";
}

@RequestMapping(value = "view/{cardId}",
    method = RequestMethod.GET)

public ModelAndView view(Map<String, Object> model,
    @PathVariable("cardId")
    long cardId) {
    Card card = this.cardBase.get(cardId);
}

```

```

        if(card == null)
            return this.getListRedirectModelAndView();
        model.put("cardId", Long.toString(cardId));
        model.put("card", card);
        return new ModelAndView("card/view");
    }

    @RequestMapping(value = "create",
                    method = RequestMethod.GET)
    public String create(Map<String, Object> model) {
        model.put("cardForm", new CardForm());
        return "card/add";
    }

```

А также добавим метод `getNextCardId` для генерации идентификатора обращения:

```

private synchronized long getNextCardId() {
    return this.CARD_ID++;
}

```

Теперь добавим в свое приложение [Spring](#) использование пользовательского класса `com.academy.site.DownloadView` для загрузки файлов с контроллеров [Spring](#).

```

public class DownloadingView implements View {
    private final String filename;
    private final String contentType;
    private final byte[] contents;

    public DownloadingView(String filename,
                           String contentType,
                           byte[] contents) {
        this.filename = filename;
    }
}

```

```

        this.contentType = contentType;
        this.contents = contents;
    }

    @Override
    public String getContentType() {
        return this.contentType;
    }

    @Override
    public void render(Map<String, ?> model,
                      HttpServletRequest request,
                      HttpServletResponse response)
                      throws Exception {
        response.setHeader("Content-Disposition",
                           "attachment; filename=" +
                           this.filename);
        response.setContentType("application/
                                octet-stream");
        ServletOutputStream stream = response.
            getOutputStream();
        stream.write(this.contents);
    }
}

```

Это представление значительно упрощено и имеет строго ограниченное поведение. Добавим метод **download** в **CardController**, который в браузере клиента отправляет вложение для загрузки.

```

public View download(@PathVariable("cardId") long cardId,
                    @PathVariable("attachment")
                    String name) {
    Card card = this.cardBase.get(cardId);
}

```

```

if(card == null)
    return this.getListRedirectView();
Attachment attachment = card.getAttachment(name);
if(attachment == null) {
    return this.getListRedirectView();
}
return new DownloadingView(attachment.getFileName(),
    attachment.getFileMimeType(),
    attachment.getFileContents());
}

```

Методы `getListRedirectModelAndView` и `getListRedirectView` класса `CardController` выглядят следующим образом:

```

private ModelAndView getListRedirectModelAndView() {
    return new ModelAndView(this.
        getListRedirectView());
}
private View getListRedirectView() {
    return new RedirectView("/card/list", true, false);
}

```

Кроме того необходимо изменить наши JSP, а также путь к ним. Файловая структура выглядит следующим образом:

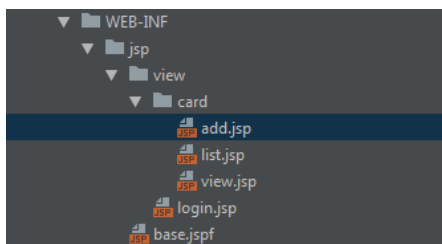


Рисунок 11.

Листинг *login.jsp*:

```

<!--@elvariable id="loginFailed"
        type="java.lang.Boolean"-->
<!--@elvariable id="loginForm" type="com.academy.site.
        LoginController.Form"-->
<!DOCTYPE html>
<html>
<head>
    <title>Support service</title>
</head>
    <message>
        <h1>Login</h1>
        Log in, please.
        <br />
        <c:if test="\${loginFailed}">
            <b>The clientName or password incorrect.</b>
        <br />
        </c:if>
        <form:form method="post"
            modelAttribute="loginForm">
            <form:label path="clientName">
                Client Name
            </form:label>
            <br />
            <form:input path="clientName" />
            <br /><br />
            <form:label path="password">
                Password
            </form:label>
            <br />
            <form:password path="password" />
            <br /><br />
            <input type="submit" value="Log In" />
        </form:form>
    </message>
</html>

```

ListCard.jsp стал *list.jsp*:

```

<%--@elvariable id="cardBase"
                type="java.util.Map<Integer,
                com.acadamy.Card>"--%>
<!DOCTYPE html>
<html>
  <head>
    <title>Support service</title>
  </head>
  <message>
    <a href="<c:url value="/logout" />">Logout</a>
    <h1>Cards</h1>
    <a href="<c:url value="/card/create" />">
      Add card</a><br/>
    <c:choose>
      <c:when test="\${fn:length(cardBase) == 0}">
        <i>Base is empty.</i>
      </c:when>
      <c:otherwise>
        <c:forEach items="\${cardBase}"
                  var="entry">
          Card # \${entry.key}:
          <a href="<c:url value="/card/
            view/\${entry.key}">/>
            </c:url>">
            <c:out value="\${entry.
              value.topic}"/>
          </a>
          Client Name: <c:out value=
            "\${entry.value.
              clientName}"/><br />
        </c:forEach>
      </c:otherwise>
    </c:choose>
  </message>
</html>

```

CardForm.jsp стал *add.jsp*:

```

<!DOCTYPE html>
<html>
  <head>
    <title>Support service</title>
  </head>
  <message>
    <a href="

```

ViewCard.jsp стал *view.jsp*:

```

<%--@elvariable id="cardId"
      type="java.lang.String"--%>
<%--@elvariable id="card"
      type="com.academy.site.Card"--%>
<!DOCTYPE html>
<html>

```

```

<head>
  <title>Support service</title>
</head>
<message>
  <a href="<c:url value="/logout" />">Logout</a>
  <h1>Card ${cardId}: <c:out value="${card.topic}"/>
</h1>
  <i>Client Name - <c:out value="${card.clientName}"/>
</i><br/>
  <c:out value="${card.message}"/><br/>
  <c:if test="${card.numberOfAttachments > 0}">
    Attachments:
    <c:forEach items="${card.attachments}"
      var="attachment"
      varStatus="status">
      <c:if test="${!status.first}">,
    </c:if>
      <a href="<c:url value="/card/${cardId}/
        attachment/${attachment.fileName}" />">
      <c:out value="${attachment.fileName}" /></a>
    </c:forEach><br /><br />
  </c:if>
  <a href="<c:url value="/card" />">
    Back to other cards
  </a>
</message>
</html>

```

Теперь протестируйте рефакторизованное приложение сервисной поддержки. Скомпилируйте проект, запустите Tomcat из IDE и перейдите по адресу *http://localhost:8080/service/*. Проверьте все доступные функции. Клиентская часть приложения выглядит также как и ранее, но сейчас мы использовали Spring Framework и таким образом усовершенствовали наше приложение.

Расширение Model-View-Controller при помощи Controller- Service-Repository

Несмотря на простоту шаблона **Model-View-Controller** (MVC) и тот факт, что контроллеры чище, чем сервлеты, методы контроллера все равно могут выйти из-под контроля. До этого момента бизнес-логика была довольно простой: сохранять представленные данные где-нибудь в памяти., но во время создания и редактирования данных в приложении должны происходить и других действия:

- **Валидация** — при создании данных, необходимо каким-то образом обеспечить их проверку согласно заданным правилам. Например, поля могут быть обязательными или иметь ограниченные значения.
- **Оповещения** — возможно, при изменении данных, необходимо отправить об этом уведомление в текстовом сообщении, по электронной почте, или в мобильное приложение.
- **Другие данные**, существующие в приложении — данные которые возможно потребуется изменить. Например, рассмотрим систему онлайн-форума, где обычно мы видим информацию о количестве сообщений, пользователи и дате последнего сообщения в каждом форуме. Затем, в рамках форума, в каждом сообщении обычно

отображается количество ответов, пользователь и дата последнего ответа. Для повышения эффективности, статистические данные для ответов обычно сводятся к сообщениям, а для сообщений — к форуму. Таким образом, добавив ответ на сообщение в форуме, необходимо обновить данные для сообщений и форума.

- **Сохранение данных** — данные приложения редко сохраняются в памяти. Обычно они находятся в хранилище данных некоторого типа, таком как реляционная база данных, база данных NoSQL или набор файлов. Сохранение этих данных включает в себя логику, которая иногда может быть довольно сложной и содержать много кода.

Итак, принимая все это во внимание, действительно ли лучше всего поставить эту логику в контроллер? Рассмотрим рисунок, в котором показана основная операция трех компонентов в системе MVC. Что делать, если после создания веб-приложения, которое делает именно то, что необходимо заказчику, он требует добавить в приложение веб-службы RESTful и SOAP? Можно ли повторно использовать контроллеры, созданные для веб-приложения? Возможно, нет. Вместо этого придётся выполнить значительный рефакторинг кодовой базы, для того чтобы отделить логику пользовательского интерфейса от бизнес-логики.

Определение различных типов логики программы

Логика пользовательского интерфейса — это любая логика, которая существует исключительно для поддержки конкретного пользовательского интерфейса.

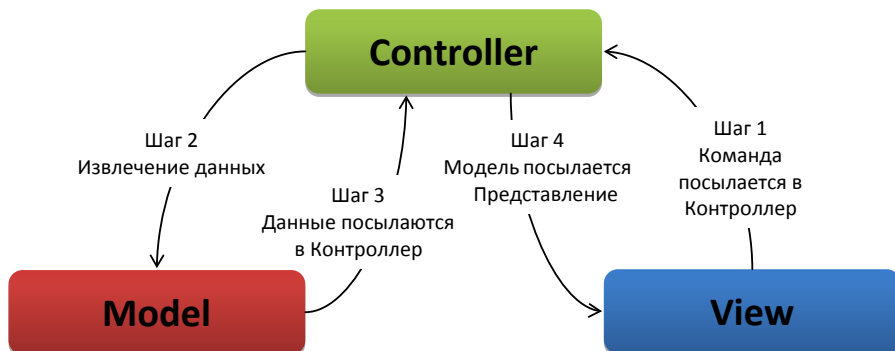


Рисунок 12.

Если часть логики кода необходима независимо от того, как пользователь взаимодействует с приложением, эта логика является бизнес-логикой. Но, если эта часть кода необходима только для конкретного пользовательского интерфейса — это все же логика пользовательского интерфейса. В идеальном приложении хотелось бы, чтобы эти типы логики разделялись на разные уровни. Тогда, появится возможность легко изменять пользовательский интерфейс или использовать несколько пользовательских интерфейсов одновременно, не внося никаких изменений в бизнес-логику. Также можно написать библиотеку, для обработки бизнес-логики, а затем использовать ее в веб- и десктопных приложениях.

Когда данные приложения сохраняются в хранилище данных определенного типа — много кода посвящено логике сохранения этих данных, не связанного с другой бизнес-логикой. В случае когда основное хранилище данных резко меняется, этот слой хранения данных становится весьма бесполезным. Например, фрагмент

кода может быть необходим, когда осуществляется переход от MySQL к PostgreSQL, но если он становится ненужным при переходе к хранилищу данных в памяти или к NoSQL — вероятно, это логика сохранения. Как от бизнес-логики отделяется логика пользовательского интерфейса, так необходимо отделять хранение данных от бизнес-логики. Благодаря этому можно получить следующие преимущества:

- Если позднее будет принято решение изменить хранилище данных, можно сделать это, не изменяя бизнес-логику.
- Тестирование бизнес-логики становится намного проще, потому что можно «заглушить» слой хранения данных и тестировать только бизнес-логику.

В зависимости от характера приложений можно размышлять о других формах программной логики, которые могут и должны быть разделены на отдельные слои. Используя `@Component` (или пользовательские аннотации, мета-аннотированные с помощью `@Component`), можно создавать любые bean-компоненты, зависящие от других `bean`, а Spring создаст и при необходимости внедрит экземпляр для создания структуры программы. Однако Spring также предоставляет шаблон `Controller-Service-Repository`, который удовлетворяет необходимость разделения этих трех общих типов программной логики. Этот шаблон Spring не структурирован, как шаблон MVC, но Spring предоставляет набор маркеров, которые направляют и поддерживают его дополнительную реализацию в приложении.

Репозитории обеспечивают логику сохранения

В шаблоне **Controller-Service-Repository**, репозитории являются самым низким уровнем, ответственным за всю логику, связанную с сохранением данных в хранилище и их извлечением из него (рис. 12). Репозиторий помечается аннотацией **@Repository**. При включенном сканировании компонентов классы **@Repository** автоматически создаются, вводятся и управляются в контексте приложения Spring, к которому они принадлежат. Как правило, каждый репозиторий отвечает за один объект или сущность. Это разбивает слой репозитория на небольшие единицы легко проверяемого кода, и можно заменить объект, если он принадлежит другому хранилищу данных.

Репозитории создаются из интерфейсов и зависимости репозитория программируются от его интерфейса, а не от реализации. Таким образом, репозиторий можно имитировать с использованием платформы предоставления зависимостям имитированной реализации, такой как **EasyMock** или **Mockito**, чтобы облегчить тестирование своих зависимостей, не полагаясь на реальную реализацию. Как правило, хранилища, которые выполняют подобные операции хранения, наследуются от базового класса, который обеспечивает общие операции со всеми подобными репозиториями. Репозитории могут также содержать другие репозитории, но не должны содержать ресурсы в более высоких слоях приложения, таких как службы или контроллеры.

Сервис обеспечивают бизнес-логику

Сервис — это следующий слой над репозиториями. Сервисы инкапсулируют бизнес-логику приложения и содержат другие сервисы и репозитории, но не содержат ресурсы в более высоких слоях приложений, таких как контроллеры, что Spring не активно применяет. Сервисы отмечены аннотацией `@Service`, доступны для автоматического инстанцирования и инъекции зависимостей, среди других преимуществ. Подобно репозиториям, сервисы создаются из интерфейсов, поэтому зависимые ресурсы могут программироваться от интерфейса. Схема базирования каждого слоя на множестве интерфейсов позволяет тестировать каждый слой отдельно от всех остальных. С транзакционной точки зрения, выполнение сервисного метода с более высокого уровня (например, контроллера) можно рассматривать как транзакционную единицу работы. Он может выполнять несколько операций с несколькими репозиториями и другими сервисами в контексте, это значит, что все операции либо выполнятся, либо выйдут из строя как единое целое. Когда метод сервиса выполняется из другого метода сервиса, он обычно считается принадлежащим к той же транзакции, к которой принадлежит вызывающий метод.

Следует отметить, что концепция транзакции не означает, что ее всегда можно обрабатывать с помощью традиционных транзакций реляционной базы данных. Операции, выполняемые во время транзакции, могут иметь несколько последствий для разных хранилищ данных или файлов. Эти операции могут включать в себя передачу сообщений электронной почты, текстовых сообщений или мобильных

уведомлений, которые в большинстве случаев не могут быть отклонены. В простом приложении, посвященном одному хранилищу данных, транзакции примерно эквивалентны транзакциям базы данных. Spring обеспечивает поддержку этих транзакционных потребностей.

Некоторые разработчики не любят использовать термин «сервис» для описания этого уровня приложения, поскольку его иногда можно путать с веб-сервисами. Название бизнес-логики, не имеет значения, даже не нужно использовать аннотацию `@Service`. Вместо этого можно использовать `@Component` или пользовательскую аннотацию, мета-аннотированную `@Component`, название и аннотирование не изменит ее цели.

Контроллеры обеспечивают логику пользовательского интерфейса

Вы уже хорошо знакомы с концепцией контроллеров, `@Controller`, `@RequestMapping` и принципом работы их вместе со Spring MVC. Контроллеры являются верхним слоем «пищевой цепи» в шаблоне `Controller-Service-Repository`. Фактически, эту трехслойную систему можно легко сравнить с естественной пищевой цепью. В этой системе репозитории являются растительной жизнью и поглощают только питательные вещества от природы (база данных). Сервисы — всеядные, которые потребляют репозитории (растительность) или другие сервисы (всеядные). Продолжая эту аналогию, контроллеры являются плотоядными. Они потребляют сервисы (всеядные), но они никогда не потребляют репозитории (растительную

жизнь) напрямую, и они никогда не потребляют другие контроллеры. Контроллеры в той или иной форме управляют пользовательским интерфейсом и, используя сервисы для помощи, подготавливают модель для отображения в представлении.

В парадигме MVC сервисы и репозитории считаются частью контроллера (не `@Controller`). Это показано на рисунке. `@Controller`, `@Service` зависит от того, с чем связаны `@Repository`, с которыми работает `@Service`, и любые слои кэширования, которые лежат между этими компонентами, действуют вместе, чтобы сформировать контроллер в шаблоне `Model-View-Controller`. Все эти компоненты используют модель в той или иной форме.

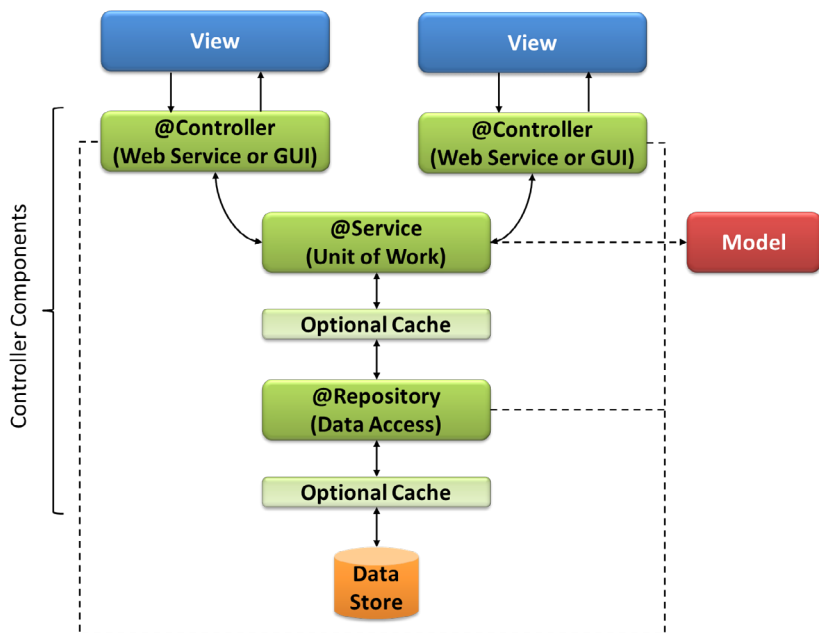


Рисунок 13.

В конечном счете, `@Controller` — для графического веб-интерфейса или API веб-сервиса — передает необходимые части модели в представление для рендеринга. Это представление может быть JSP (для графического веб-интерфейса) или механизмом рендеринга JSON или XML (для графического веб-интерфейса или API веб-сервиса).

Использование корневого контекста приложения вместо контекста веб-приложения

Класс `ServletContextConfiguration` можно назвать как угодно, но его цель остается прежней. В качестве конфигурации контекста приложения Spring для `DispatcherServlet` этот класс отвечает за установку работы Spring при получении HTTP-запросов в контейнере `Servlet`. Это описание немедленно ограничивает возможности для конфигурации.

Вне контейнера `Servlet`, класс `ServletContextConfiguration` не имеет никакой цели. Содержимое конфигурации отражает это. Каждый компонент, сконфигурированный в этом классе, каким-то образом связан с приемом, обработкой и ответом на HTTP-запросы. Если Вы хотите создать веб-службу RESTful или SOAP для своего приложения, Вы, вероятно, создадите отдельный `DispatcherServlet` со своим собственным контекстом приложения и `@Configuration`. Конфигурация будет отличаться, в зависимости от того, как различные контроллеры в этом контексте обрабатывают запросы. Вероятно, Вы отключили бы ненужные компоненты, такие как `ByteArrayHttpMessageConverter`, `StringHttpMessageConverter` и `FormHttpMessageConverter`, поскольку они не будут

применяться веб-сервисами RESTful или SOAP. Также не нужны [ViewResolver](#) и [RequestToViewNameTranslator](#).

Повторное использование контекста корневого приложения для нескольких пользовательских интерфейсов

Различные пользовательские интерфейсы обычно используют одни и те же сервисы. Таким образом, бизнес-логика остается неизменной во всех пользовательских интерфейсах. Для этого не следует управлять сервисами и репозиториями в контексте веб-приложения, но в контексте корневого приложения, которое служит родителем для всех контекстов веб-приложений. Все эти сервисы и репозитории наследуются контекстами веб-приложений, которые управляют различными пользовательскими интерфейсами. Если необходимо создать настольное приложение, используя сервисы и репозитории, написанные для веб-приложения, можно использовать ту же конфигурацию контекста корневого приложения, даже если она загружена через другой механизм.

Ранее мы рассматривали иерархию и наследование контекста приложения, настраивая контекст корневого приложения ([RootContextConfiguration](#)) с помощью контекста веб-приложения ([ServletContextConfiguration](#)) и изменяя сканирование компонентов для сканирования только соответствующих компонентов в каждом контексте. Сканирование компонентов — это ключ к этому разделению. Если Вы используете сканирование компонентов,

Вы должны настроить его правильно. В противном случае Вы можете получить повторяющиеся определения bean-компонентов, или, что еще хуже, некоторые определения bean не смогут быть отображены. Сканирование компонентов работает на двух принципах: сканирование пакетов и фильтрация классов.

При использовании аннотации `@ComponentScan` указывается Spring, какой пакет или Java пакеты сканировать для классов-кандидатов, используя атрибут `String[] basePackages`. Spring находит все классы, принадлежащие этим пакетам или подпакетам, и применяет фильтры ресурсов к каждому классу. Недостатком `basePackages` есть то, что он не безопасного типа, и поэтому опечатка может легко остаться незамеченной. В качестве альтернативы можно использовать атрибут `Class<?>[] basePackageClasses`. Spring определяет имена пакетов для сканирования из указанных классов.

Spring применяет фильтры для каждого класса, который он находит в базовых пакетах. Фильтры разделены на включающие и исключающие. Если класс запускает любой из включающих фильтров и не запускает ни один из исключающих — он становится Spring-компонентом, то есть он сконструирован, инъецирован, инициализирован и что-либо еще, что применимо к компонентам, управляемым Spring. Если атрибут `useDefaultFilters` `@ComponentScan` является истинным, то исключающие фильтры не используются, и по умолчанию используется только один включающий фильтр. Включающий фильтр по умолчанию в классе аннотируется `@Component`. Таким

образом, следующие две конфигурации сканирования компонентов эквивалентны:

```
@ComponentScan(basePackages = "com.academy.site")
@ComponentScan(
    basePackages = "com.academy.site",
    useDefaultFilters = false,
    includeFilters = @ComponentScan.Filter(value =
        Component.class,
        type = FilterType.ANNOTATION)
)
```

Если указать включающие или исключающие фильтры, они будут добавлены к фильтру по умолчанию, если `useDefaultFilters` истинно. Если установить параметр `useDefaultFilters` в `false`, указанные фильтры заменят фильтр по умолчанию. Фильтры сканирования компонентов можно настроить как угодно.

Например, можно создать свой собственный набор аннотаций и указать это Spring. Также можно использовать фильтры назначаемого типа, которые запускаются, когда класс расширяет или реализует класс или один из указанных классов. Можно использовать `org.springframework.context.annotation.FilterType.CUSTOM`, и в этом случае указанным классом или классами должны быть реализации `org.springframework.core.type.filter.TypeFilter`. Вы можете применить комбинацию некоторых или всех этих методов с или без фильтров по умолчанию.

Рассмотрим следующую конфигурацию сканирования компонентов, которая поступает из `RootContextConfiguration`.

```
@ComponentScan (
    basePackages = "com.academy.site",
    excludeFilters = @ComponentScan.Filter(Controller.
                                                class)
)
```

Эта конфигурация сканирует пакет *com.academy.site* для любых классов, которые передают фильтры по умолчанию, но не помечены **@Controller**. Таким образом, классы, отмеченные любой из аннотаций **@Component**, **@Service**, **@Repository**, становятся компонентами, но с **@Controller** этого не происходит. Это дополнение к конфигурации сканирования компонентов для **ServletContextConfiguration**, которая обнаруживает только компоненты, аннотированные с помощью **@Controller**.

```
@ComponentScan (
    basePackages = "com.academy.site",
    useDefaultFilters = false,
    includeFilters = @ComponentScan.Filter(Controller.
                                                class)
)
```

Использование этих компонентных сканирований для отдельных контекстов приложений выполняет задачу централизации бизнес-логики и логики репозитория в контексте корневого приложения, тогда как логика пользовательского интерфейса располагается в контексте приложения **Servlet**. Если фильтры сканирования компонентов не указаны в корневом и сервлет-приложениях, компоненты будут созданы в каждом контексте.

Перемещение бизнес-логики с контроллеров на сервисы

В идеальном мире каждый программист получает возможность писать каждую программу с нуля, используя лучшие инструменты и методы. В реальном мире обычно требуется работать с устаревшими приложениями, которые не всегда используют хорошие инструменты или передовые методы. Необходимо не только поддерживать эти приложения и решать проблемы, о которых сообщают пользователи, необходимо также добавлять к ним новые функции, и это требует реорганизации кода для поддержки новых бизнес-потребностей.

Рассмотрим к примеру форум, какая бизнес-логика может потребоваться? Предполагая, что нет категорий, классификации или групповых чатов, пользователям нужна возможность создавать темы и отвечать на них. При создании темы нужно убедиться, что ее заголовок и тело не пустые. Отвечая на тему, необходимо не только проверять, что ответ не пуст, но и что ответ написан для существующей темы. Как темы, так и ответы должны иметь автоматическую временную метку, а не пользовательскую, поэтому это бизнес-логика. В этом гипотетическом сценарии у Вас есть существующее приложение с двумя контроллерами. [ForumController](#) предоставляет возможность перечислить и создать темы обсуждений, в то время как [TopicController](#) разрешает просматривать тему и отвечать на нее. Эти контроллеры охватывают всю базу кода для Вашего приложения, минус конфигурация и представления. Они управляют логикой пользователь-

ского интерфейса, выполняют бизнес-логику и сохраняют темы и ответы в базе данных.

Теперь Вам назначены три новые задачи:

- Пользователи хотят получать электронные письма с ответами на их пост или тему.
- Пользователи также хотят возможность просмотра форума и тем в качестве RSS-каналов.
- Администраторы сервера для экономии памяти и повышения производительности хотят автоматически удалять темы, с датой публикации или последней активности более года назад.

Поразмыслим об изменениях, которые необходимо внести для поддержки новых функций. Чтобы упростить отправку уведомлений, для каждой темы обсуждения требуется список всех уникальных пользователей, которые на нее ответили. Возможно, некоторые пользователи отпишутся от темы, поэтому намного проще и дешевле сохранение этого списка в самой теме вместо агрегации его из всех ответов каждый раз.

В каждой теме должно быть поле даты, которое представляет собой дату публикации или последней активности, если нет ответов. Оба эти изменения создают проблему, потому что обновления для темы при публикации ответов запускаются из [TopicController](#), но [ForumController](#) содержит логику сохранения записей темы обсуждения. Кроме того, уже есть методы, которые извлекают темы и ответы из базы данных и возвращают представление JSP. Теперь нам нужны новые методы, которые делают то же

самое, но возвращают представления RSS. Это чревато дублированием кода.

Необходимо отделить бизнес-логику от логики пользовательского интерфейса, прежде чем подходить к решению любой из этих задач. Создадим проект **Forum**. Чтобы получить представление о том, какой интерфейс Вам нужен, ознакомьтесь с методами обработчика запросов **TopicController** и **ForumController**. Не беспокойтесь о коде прямо сейчас. Подумайте об интерфейсе. **ForumController** содержит методы для листинга и создания тем.

```
@Controller
@RequestMapping("topic")
public class ForumController {
    @RequestMapping(value = {"", "list"},
                    method = RequestMethod.GET)
    public String listTopics(Map<String,
                             Object> model) { ... }
    @RequestMapping(value = "create",
                    method = RequestMethod.GET)
    public String createTopic(Map<String,
                              Object> model) { ... }
    @RequestMapping(value = "create",
                    method = RequestMethod.POST)
    public View createTopic(TopicForm form) { ... }
}
```

TopicController содержит методы для просмотра темы и ответа на нее.

```
@Controller
@RequestMapping("topic/{topicId:\\d+}")
public class TopicController {
```

```

@RequestMapping(value = {"", "*"},
                 method = RequestMethod.GET)
public String viewTopic(Map<String,
                        Object> model,
                        @PathVariable("topicId")
                        long id) { ... }

@RequestMapping(value = "reply",
                 method = RequestMethod.POST)
public ModelAndView reply(ReplyForm form,
                        @PathVariable("topicId")
                        long id) { ... }
}

```

Создадим **POJO**, которые нам нужны. Предположим, что пользователи — это адреса электронной почты, которые можно представить с помощью **String** вместо некоторого пользовательского объекта. Нам нужен объект **Topic** и объект **Reply** для сохранения данных и перехода к представлению.

```

public class Topic {
    private long id;
    private String user;
    private String subject;
    private String uriSafeSubject;
    private String message;
    private Instant created;
    private Instant lastUpdated;
    private Set<String> subscribedUsers = new HashSet<>();
    // mutators and accessors
}

public class Reply {
    private long id;
    private long topicId;
}

```

```
private String user;
private String message;
private Instant created;
// mutators and accessors
}
```

Логично иметь сервис для каждого объекта. Интерфейсы [TopicService](#) и [ReplyService](#) должны быть простыми. Обратите внимание, что [ReplyService](#) не имеет метода для получения индивидуального ответа по его идентификатору. Сейчас в этом нет необходимости. Если понадобится добавить возможность редактировать ответ, необходимо будет добавить этот метод.

```
public interface TopicService {
    List<Topic> getAllTopics();
    Topic getTopic(long id);
    void saveTopic(Topic topic);
}
public interface ReplyService {
    List<Reply> getRepliesForTopic(long topicId);
    void saveReply(Reply reply);
}
```

На данный момент не имеет значения, как работают сервисные реализации. Вы уже можете реорганизовать свои контроллеры для использования интерфейсов сервисов. Методы [ForumController](#) содержат только код, необходимый для поддержки взаимодействия с пользователем. [Create-Topic](#), в частности, даже не задает все свойства в [Topic](#). Он устанавливает только предоставленные пользователем свойства, предоставляя его сервисам, чтобы установить

сгенерированные свойства, такие как **ID** и **UriSafeSubject**. Ему не нужно знать и заботится о том, как это происходит.

```
@Controller
@RequestMapping("topic")
public class ForumController {
    @Inject TopicService topicService;
    @RequestMapping(value = {"", "list"},
                    method = RequestMethod.GET)
    public String listTopics(Map<String,
                            Object> model) {
        model.put("topics",
                 this.topicService.getAllTopics());
        return "topic/list";
    }
    @RequestMapping(value = "create",
                    method = RequestMethod.GET)
    public String createTopic(Map<String,
                               Object> model) {
        model.put("topicForm", new TopicForm());
        return "topic/create";
    }
    @RequestMapping(value = "create",
                    method = RequestMethod.POST)
    public View createTopic(TopicForm form) {
        Topic topic = new Topic();
        topic.setUser(form.getUser());
        topic.setSubject(form.getSubject());
        topic.setMessage(form.getMessage());
        this.topicService.saveTopic(topic);
        return new RedirectView("/topic/" +
                                topic.getId() + "/" +
                                topic.getUriSafeSubject(),
                                true, false);
    }
}
```

Код для `TopicController` так же прост. Большая часть работы делегируется в `TopicService` и `ReplyService`, оставляя контроллер для обслуживания только пользовательского интерфейса.

```
@Controller
@RequestMapping("topic/{topicId:\\d+}")
public class TopicController {
    @Inject TopicService topicService;
    @Inject ReplyService replyService;
    @RequestMapping(value = {"", "*"},
        method = RequestMethod.GET)
    public String viewTopic(Map<String,
        Object> model,
        @PathVariable("topicId")
        long id) {
        Topic topic = this.topicService.getTopic(id);
        if(topic != null) {
            model.put("topic", topic);
            model.put("replies",
                this.replyService.
                    getRepliesForTopic(id));
            model.put("replyForm", new ReplyForm());
            return "topic/view";
        }
        return "topic/errorNoTopic";
    }
    @RequestMapping(value = "reply",
        method = RequestMethod.POST)
    public ModelAndView reply(ReplyForm form,
        @PathVariable("topicId")
        long id) {
        Topic topic = this.topicService.getTopic(id);
        if(topic != null) {
            Reply reply = new Reply();
            reply.setTopicId(id);
```

```

        reply.setUser(form.getUser());
        reply.setMessage(form.getMessage());
        this.replyService.saveReply(reply);
        return new ModelAndView(new RedirectView(
            "/topic/" + id + "/" +
            topic.getUriSafeSubject(),
            true, false));
    }
    return new ModelAndView("topic/errorNoTopic");
}
}

```

Использование репозитория для хранения данных

При написании реализаций `TopicService` и `ReplyService` становится понятно, что значительная часть необходимого кода предназначена для сохранения данных и их извлечения из базы данных. Чтобы код сервиса был исключительно связан с бизнес-логикой, лучше всего создать слой репозитория для решения проблемы сохранения данных. Благодаря этому методы обслуживания становятся более краткими и легко проверяемыми. Интерфейсы для `TopicRepository` и `ReplyRepository` просты и отражают сервисные интерфейсы.

```

public interface TopicRepository {
    List<Topic> getAll();
    Topic get(long id);
    void add(Topic topic);
    void update(Topic topic);
    void delete(long id);
}

```

```
public interface ReplyRepository {
    List<Reply> getForTopic(long id);
    void add(Reply reply);
    void update(Reply reply);
    void deleteForTopic(long id);
}
```

Теперь, имея интерфейс для сохранения данных в программе, можно реализовать сервисы. Метод `getTopic` `DefaultTopicService` не имеет дополнительной бизнес-логики (хотя когда-нибудь может понадобиться ее наличие), но `getAllTopics` сортирует список перед возвратом, а `saveTopic` выполняет некоторые задачи. Во-первых, он нормализует `subject`, для того чтобы сделать его URI безопасным и устанавливает нормализованное значение в свойство `uriSafeSubject`. Это используется при создании URL-адресов для темы, чтобы сделать удобным их поиск. Он также обновляет временную метку `lastUpdated`. Если тема новая, он обновляет временную метку создания и подписывается на пользователя, который создал ее, прежде чем добавлять тему, в противном случае — обновляет ее.

```
@Service
public class DefaultTopicService implements
    TopicService {

    @Inject
    TopicRepository topicRepository;

    @Override
    public List<Topic> getAllTopics() {
        List<Topic> list = this.topicRepository.
            getAll();
    }
}
```

```

        list.sort((d1, d2) ->
            d1.getLastUpdated().
            compareTo(d2.getLastUpdated()));
        return list;
    }
    @Override
    public Topic getTopic(long id) {
        return this.topicRepository.get(id);
    }
    @Override
    public void saveTopic(Topic topic) {
        String subject = topic.getSubject();
        subject = Normalizer.normalize(subject.
            toLowerCase(),
            Normalizer.Form.NFD)
            .replaceAll("\\p
            {InCombiningDiacriticalMarks}+", "")
            .replaceAll("[^\\p{Alnum}]", "-")
            .replace("--", "-").replace("--", "-")
            .replaceAll("[^a-z0-9]+$", "")
            .replaceAll("[^a-z0-9]", "");
        topic.setUriSafeSubject(subject);
        Instant now = Instant.now();
        topic.setLastUpdated(now);
        if(topic.getId() < 1) {
            topic.setCreated(now);
            topic.getSubscribedUsers().
                add(topic.getUser());
            this.topicRepository.add(topic);
        }
        else this.topicRepository.update(topic);
    }
    @Scheduled(fixedDelay = 15_000L,
        initialDelay = 15_000L)
    public void deleteStaleDiscussions() {
        Instant oneYearAgo = Instant.now().minus(365L,
            ChronoUnit.DAYS);
    }

```

```

        List<Topic> list = this.topicRepository.getAll();
        list.removeIf(d -> d.getLastUpdated().
                        isAfter(oneYearAgo));
        for(Topic old : list)
            this.topicRepository.delete(old.getId());
    }
}

```

Метод `getRepliesForTopic` `DefaultReplyService` сортирует список, возвращаемый репозиторием, прежде чем возвращать его. `SaveReply` используется как репозиториум, так и `TopicService` (использование интерфейса вместо реализации) для выполнения некоторых бизнес-задач. Если ответ новый, он присоединяет автора ответа к теме, устанавливает временную метку создания ответа и добавляет его. Если он не новый, он просто обновляет ответ. В любом случае, он сохраняет тему с помощью `TopicService`.

```

@Service
public class DefaultReplyService implements
    ReplyService {
    @Inject ReplyRepository replyRepository;
    @Inject TopicService topicService;

    @Override
    public List<Reply> getRepliesForTopic(long topicId) {
        List<Reply> list = this.replyRepository.
            getForTopic(topicId);
        list.sort((r1, r2) -> r1.getId() <
            r2.getId() ? -1 : 1);
        return list;
    }
}

```

```

@Override
public void saveReply(Reply reply) {
    Topic topic = this.topicService.
        getTopic(reply.getTopicId());

    if(reply.getId() < 1) {
        topic.getSubscribedUsers().
            add(reply.getUser());
        reply.setCreated(Instant.now());
        this.replyRepository.add(reply);
        Set<String> recipients = new HashSet<>
            (topic.getSubscribedUsers());
        recipients.remove(reply.getUser()); // no
            // need to email replier
    }
    else {
        this.replyRepository.update(reply);
    }

    this.topicService.saveTopic(topic);
}
}

```

Теперь осталось реализовать репозитории. Для этого примера используется [Map. InMemoryTopicRepository](#) довольно предсказуем, но следует отметить, что код становится более сложным, если Вы на самом деле сохраняете базу данных некоторого типа.

```

@Repository
public class InMemoryTopicRepository implements
    TopicRepository {
    private final Map<Long, Topic> database =
        new Hashtable<>();
}

```

```

private volatile long topicIdSequence = 1L;
@Inject ReplyRepository replyRepository;
@Override
public List<Topic> getAll() {
    return new ArrayList<>(this.database.values());
}
@Override
public Topic get(long id) {
    return this.database.get(id);
}
@Override
public void add(Topic topic) {
    topic.setId(this.getNextTopicId());
    this.database.put(topic.getId(), topic);
}
@Override
public void update(Topic topic) {
    this.database.put(topic.getId(), topic);
}
@Override
public void delete(long id) {
    this.database.remove(id);
    this.replyRepository.deleteForTopic(id);
}
private synchronized long getNextTopicId() {
    return this.topicIdSequence++;
}
}

```

В `InMemoryReplyRepository` метод `getForTopic` использует лямбда-выражение подобно выражению `WHERE` в SQL запросе, для исключения ответов, не относящихся к выбранной теме. Ключевое слово `volatile` гарантирует, что разные потоки никогда не видят устаревших значений полей последовательности ID.

```

@Repository
public class InMemoryReplyRepository implements
    ReplyRepository {
    private final Map<Long, Reply> database =
        new Hashtable<>();
    private volatile long replyIdSequence = 1L;

    @Override
    public List<Reply> getForTopic(long id) {
        ArrayList<Reply> list =
            new ArrayList<>(this.
                database.values());
        list.removeIf(r -> r.getTopicId() != id);
        return list;
    }

    @Override
    public synchronized void add(Reply reply) {
        reply.setId(this.getNextReplyId());
        this.database.put(reply.getId(), reply);
    }

    @Override
    public synchronized void update(Reply reply) {
        this.database.put(reply.getId(), reply);
    }

    @Override
    public synchronized void deleteForTopic(long id) {
        this.database.entrySet()
            .removeIf(e -> e.getValue()
                .getTopicId() == id);
    }
    private synchronized long getNextReplyId() {
        return this.replyIdSequence++;
    }
}

```

IndexController содержит следующий код:

```
@Controller
public class IndexController {
    @RequestMapping("/")
    public View index() {
        return new RedirectView("/topic", true, false);
    }
}
```

Файловая структура JSP выглядит следующим образом:

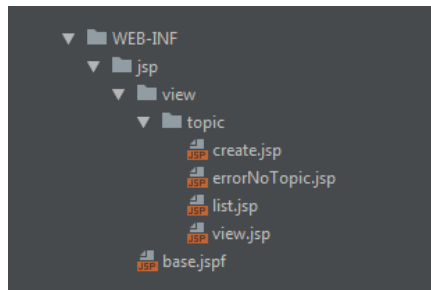


Рисунок 14.

Листинг *create.jsp*:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Create Topic</title>
  </head>
  <body>
    <h2>Create Topic</h2>
    [<a href="<c:url value= "/topic/list" />">
      back to list</a>]<br />
    <br />
```

```

        <form:form method="post"
            modelAttribute="topicForm">
            <form:label path="user">Email
        </form:label><br />
            <form:input path="user" type="email" />
        <br /><br />
            <form:label path="subject">Subject
        </form:label><br />
            <form:input path="subject" /><br />
        <br />
            <form:label path="message">Message
        </form:label>
        <br />
            <form:textarea path="message"
                cols="40" rows="5" />
        <br /><br />
            <input type="submit" value="Submit" />
        </form:form>
    </body>
</html>

```

Листинг *errorNoTopic.jsp*:

```

<!DOCTYPE html>
<html>
    <head>
        <title>Error: Topic Not Found</title>
    </head>
    <body>
        <h2>Error: Topic Not Found</h2>
        The topic was not found.<br />
        <br />
        [<a href="<c:url value="/topic/list" />">
            back to list</a>]
    </body>
</html>

```

Листинг *list.jsp*:

```

<!DOCTYPE html>
<html>
  <head>
    <title>Error: Topic Not Found</title>
  </head>
  <body>
    <h2>Error: Topic Not Found</h2>
    The topic was not found.<br />
    <br />
    [<a href="<c:url value="/topic/list" />">
      back to list</a>]
  </body>
</html>

```

Листинг *view.jsp*:

```

<%--@elvariable id="topic"
                  type="com.academy.site.entity.Topic"--%>
<%--@elvariable id="replies"
                  type="java.util.List<com.academy.
                    site.entity.Reply>"--%>
<!DOCTYPE html>
<html>
  <head>
    <title>Topic: <c:out value="\${topic.subject}" />
    </title>
  </head>
  <body>
    <h2>Topic: <c:out value="\${topic.subject}"/></h2>
    [<a href="<c:url value="/topic/list" />">
      back to list</a>]<br />
    <i>User: <c:out value="\${topic.user}" /> /
      Created: <c:out value="\${topic.created}" />
    </i>

```

```

<br />
<c:out value="\${topic.message}" />
<br />
<br />
<c:choose>
  <c:when test="\${fn:length(replies) > 0}">
    <c:forEach items="\${replies}"
      var="reply" varStatus="i">
      <b>Reply #\${i.count}</b><i> -
      <c:out value="\${reply.user}" /> -
      <c:out value="\${reply.created}" />
      </i><br />
      <c:out value="\${reply.message}" />
      <br />
      <br />
    </c:forEach>
  </c:when>
  <c:otherwise>
    <i>There are no replies yet.</i>
    <br />
    <br />
  </c:otherwise>
</c:choose>
<b>Post Reply</b>
<br />
<br />
<c:set var="action">
  <c:url value="/topic/\${topic.id}/
    reply" />
</c:set>
<form:form method="post"
  action="\${action}"
  modelAttribute="replyForm">
  <form:label path="user">Your Email
</form:label>
<br />
  <form:input path="user" type="email" />

```

```
<br />
<br />
<form:label path="message">Message
</form:label>
<br />
<form:textarea path="message" cols="40"
                rows="5" />

<br />
<br />
<input type="submit" value="Submit" />
</form:form>
</body>
</html>
```

Когда проект готов, скомпилируйте приложение и запустите Tomcat из IDE, затем перейдите по адресу <http://localhost:8080/forum/>. Попробуйте создать и ответить на различные темы.

Данный код был бы гораздо сложнее, если бы мы поместили его в контроллеры. Задача создания просмотров RSS, запрашиваемых пользователями, теперь намного проще, так как можно повторно использовать вызовы служб.

Понятие веб-сервисов REST и SOAP

Создание веб-сервисов для приложений — это задача, с которой Вам неизбежно придется столкнуться в будущем, если Вы еще этого не делали. Веб-сервисы предоставляют возможность взаимодействовать с Вашим веб-приложением клиентам, не являющимися людьми. Обычно есть человек, взаимодействующий с клиентом веб-сервиса, но на данном этапе это не имеет значения.

Обычно для создания веб-сервиса необходим способ, позволяющий клиентам программно использовать и находить ресурсы веб-приложения. Это становится все более важным, поскольку мобильные приложения становятся все более распространенными. Мобильные приложения, как правило, полагаются на веб-приложение которые выполняют основные задачи, выступая только в качестве пользовательских интерфейсов на мобильном устройстве. Это позволяет кому-то, у кого есть множество устройств (мобильных или других) синхронизировать данные между устройствами. Сегодня многие разработчики приложений предпочитают создавать веб-сервисы и писать одностраничные JavaScript-приложения, которые напрямую взаимодействуют с этими веб-сервисами. Эта модель в конечном итоге станет правилом, а не исключением, потому что требует меньшего времени на разработку приложений, давая разработчикам пользовательского

интерфейса больше возможностей на создание простых приложений на разных платформах.

На самом деле существует только один вид веб-сервиса — SOAP. В свою очередь RESTful технически не является веб-сервисом. И в самом деле, *World Wide Web Consortium* (W3C) гласит: «Веб-сервис представляет собой программную систему, предназначенную для поддержки взаимодействия машина-машина в сети. Он имеет интерфейс, описанный в обрабатываемом машинном формате (в частности, WSDL). Другие системы взаимодействуют с веб-сервисом в соответствии с описанием, используя SOAP-сообщения, которые обычно передаются с использованием HTTP с XML сериализацией в сочетании с другими стандартами, связанными с Web».

В рамках этого урока веб-сервис представляет собой любую программную систему, которая поддерживает взаимодействие между машинами через компьютерную сеть, используя совместимый, обрабатываемый машиной протокол и формат. Это включает в себя SOAP и RESTful веб-сервисы и другие технологии веб-сервисов.

Что такое SOAP?

SOAP, который первоначально был *Simple Object Access Protocol*, является протоколом взаимодействия между машинами, используя структурированные данные в форме XML-сообщений. В версии 1.2 протокола (первая принята W3C) акроним был отброшен, а слово SOAP было само по себе. Конечно, SOAP не стал началом веб-сервисов в общем смысле, потому что другие технологии, такие

как XML-RPC, существовали задолго до его появления. Однако принятие SOAP считается поворотным моментом, когда веб-сервисы стали более распространенными среди предприятий.

SOAP обладает тремя основными характеристиками, которые стимулируют его популярность:

- **расширяемость** — легко добавить другие функции, такие как безопасность, поверх базового протокола.
- **нейтральность** — неважно, какой используется транспортный механизм: HTTP, JMS, AMQP, SMTP, TCP или прочие.
- **независимость** — SOAP не поддерживает какую-либо конкретную модель программирования.

Тем не менее, SOAP предпочитает концепцию разработки контрактного дизайна. *Web Services Descriptive Language* (WSDL) определяет способ установления контракта, который сперва должны соблюдать пользователи, а после и веб-сервис соответственно (*contract-first design*). Он определяет расположение веб-сервисов, доступных в определенном месте, и формат данных для запросов и ответов от них. Обычно написанный с использованием XSD (иногда называемый определением XML-схемы), контракт создается до того, как будет написан какой-либо код для веб-сервиса, и веб-сервис создан для выполнения контракта. Это похоже на концепцию разработки через тестирование, которая основывается на повторении очень коротких циклов разработки: сначала пишется тест, покрывающий желаемое изменение, затем пишется код,

который позволит пройти тест, и под конец проводится рефакторинг нового кода к соответствующим стандартам. Эта концепция резко контрастирует с контрактом дизайна, где сначала создается веб-сервис, а затем контракт устанавливается в соответствии с ним (*contract-last design*).

Контрактное проектирование имеет свои достоинства, и, конечно, всегда лучше четко определить потребности пользователей и написать приложение, которое их удовлетворяет, вместо написания приложения, а затем выяснения, как оно отвечает потребностям пользователей. Так как четкое определение потребностей пользователей и запись WSDL — это две разные вещи, гораздо проще явно определить контракт на простом человеческом языке, написать код веб-сервиса, соответствующий требованиям, а затем автоматически генерировать очень сложный документ WSDL, используя инструменты XML.

Каждое сообщение SOAP (запрос или ответ) начинается с корневого элемента, называемого конвертом (SOAP Envelope). В нем содержится вся информация о сообщении. Конверт имеет необязательный элемент заголовок (SOAP Header), не путать с заголовками HTTP, который содержит информацию о конкретном приложении, такую как данные аутентификации. Также в пределах конверта находится обязательный элемент тело (SOAP Body), который содержит инструкции и данные запроса или ответа. Также в оболочке ответа может быть необязательный элемент с сообщением об ошибке (SOAP Fault), описывающий любые ошибки, возникающие при обработке запроса.



Рисунок 15.

Детали каждого из этих элементов в значительной степени определяются WSDL. Написание и работа с документами WSDL может быть сложной и запутанной задачей, и чем больше можно автоматизировать этот процесс, тем лучше. Хуже того, XML — это один из самых подробных и широко распространенных форматов данных с пропускной способностью и процессорами. Хотя SOAP может быть очень полезным и мощным, это не единственный вариант.

Что такое REST?

Representational State Transfer (REST) представляет собой концепцию веб-сервиса во многом схожую с SOAP. Архитектура REST состоит из серверов и клиентов. Клиенты осуществляют переходы состояния посредством действий (запросы на серверы для получения или изменения состояния ресурсов), а серверы обрабатывают запросы и возвращают соответствующие ресурсы. Ресурсом может быть любая логическая концепция, которая понимается как

клиентом, так и сервером, и передается в согласованном представлении. REST не зависит от типа ресурса или его представления. Ресурсы могут быть любыми возможными данными. Они могут представлять собой простой текст, HTML, XML, YAML, JSON, двоичные данные или любой другой формат, понятный как клиенту, так и серверу. Системы REST работают с использованием URL-адресов, указывающих тип ресурса, действие, которые должны выполняться на ресурсе, и типы файлов (MIME), которые указывают представление ресурса в запросе и ответе. Всемирная паутина по своей природе является самой большой системой REST.

Основным принципом веб-сервисов RESTful является то, что не существует большого количества важных операций, которые должны выполняться на ресурсе. Эти процедуры обычно называются операциями CRUD: создание, чтение, обновление и удаление. Методы, обозначенные в спецификации HTTP, легко сопоставляются с этими действиями: **POST**, **GET**, **PUT** и **DELETE** соответственно. SOAP использует элемент конверт для описания выполняемого действия (метода), тогда как REST полагается, что протокол HTTP предоставит конверт. SOAP также использует конверт для определения используемого ресурса, тогда как REST полагается на URL-адрес HTTP.

В веб-сервисе RESTful заголовок запроса **Content-Type** информирует сервер о представлении тела запроса, заголовок **Accept** или расширение файла запрашивает конкретное представление ответа, а заголовок ответа **Content-Type** информирует клиента о его представлении.

Стоит выделить некоторые преимущества и недостатки, возникающие для каждого протокола. Веб-сервисы RESTful привязаны к HTTP, в то время как SOAP не зависит от протокола, в данном случае преимущество SOAP. Хотя было бы невозможно создать веб-службу RESTful без использования HTTP, помните, что веб-сервис RESTful — это веб-сервис, созданный поверх протокола REST, это не сам REST. Таким образом, создание RESTful веб-сервиса без HTTP будет включать создание протокола REST с URL-адресами, действиями и типами мультимедиа, подобными HTTP, которые могут служить в качестве конверта для запроса и ответа. В этот момент можно использовать HTTP. Веб-сервисы RESTful полностью независимы от формата данных, в то время как SOAP требует XML. SOAP также имеет избыточность при использовании с HTTP, потому что это по существу конверт внутри конверта. Элемент конверт дублирует функции, которые могут быть предоставлены с помощью методов URL и HTTP. Это необходимо, чтобы оставаться нейтральным по протоколу.

Одним из ключевых преимуществ веб-сервисов RESTful является отсутствие WSDL. Это не означает, что веб-сервисы RESTful являются контрактными. Они могут работать через contract-first так же легко, как и SOAP, но контракт определяется с использованием документации на открытом языке и свободно доступных публичных API, а не сложного XSD.

Некоторые поставщики веб-сервисов RESTful по-прежнему до сих пор публикуют документы XML Schema

(XSD) или JSON Schema для своих веб-сервисов RESTful на основе XML или JSON, что также может помочь определить контракт. По этим и многим другим причинам, REST стала преобладающей архитектурой веб-сервисов, обгоняя и превосходя SOAP как во внедрении, так и в поддержке фреймворков. Spring Framework, например, поддерживает REST непосредственно в рамках MVC, но для веб-сервисов SOAP необходимо использовать отдельный проект Spring Web Services.

Понятие открытости

Еще одной важной особенностью веб-сервисов RESTful является их открытость. Используя комбинацию URL-адресов и, порой, метод OPTIONS HTTP, клиенты могут обнаруживать ресурсы, доступные в веб-сервисе без доступа к контракту, а затем на каждом ресурсе клиенты могут обнаружить действия, доступные для ресурса. Хотя многие поставщики веб-сервисов RESTful сегодня не поддерживают обнаружение, это центральный принцип архитектуры REST. Всемирная паутина обнаруживается автоматически. В качестве пользователя Вы можете открыть веб-браузер и перейти на любую домашнюю страницу. С этой страницы Вы можете открыть другие страницы на веб-сайте, используя гиперссылки. Нажав гиперссылку, Вы попадаете на новую страницу, со своими свои собственными гиперссылками. Таким образом, без каких-либо предварительных знаний о содержании сайта можно найти ресурсы на нем и каким-то образом с ними взаимодействовать.

Это же ограничение помещается в официальную архитектуру приложения REST. Клиенты REST должны иметь возможность использовать веб-сервис RESTful без каких-либо предварительных знаний о доступных на этом веб-сервисе ресурсах. Эта концепция, известная как *Hypermedia as the Engine of Application State* (HATEOAS), использует гипертекст (XML, YAML, JSON или другой формат) в сочетании с гиперссылками, чтобы информировать клиента о структуре веб-сервиса. Например, запрос и ответ на базовый URL-адрес веб-сервиса с использованием GET может выглядеть так:

```
> GET /Rest/ HTTP/1.1
> Accept: application/json
< 200 OK
< Content-Type: application/hal+json
<
< {
<   "_links": {
<     "self": { "href": "http://academy.net/Rest" },
<     "profile": { "href": "http://academy.net/
<                   Rest/profile" }
<   }
< }
```

С этими данными клиент теперь знает ресурсы, доступные на веб-сервисе. Если он хочет исследовать аккаунт дальше, у него есть несколько вариантов. Во-первых, можно сделать GET запрос ресурсу. Это называется запросом коллекции или запросом к URI коллекции и возвращает каждый доступный ресурс этого типа.

```

> GET /Rest/profile HTTP/1.1
> Accept: application/json
< 200 OK
< Content-Type: application/json
<
< {
<   "value": [
<     {
<       "id": 1,
<       "name": "Ivan Ivanov",
<       ...
<     }, {
<       "id": 2,
<       "name": "Petr Petrov",
<       ...
<     }
<   ]
< }

```

Это имеет несколько недостатков. Во-первых, эта коллекция может быть очень большой. Вы поймете в чем проблема если представите, что веб-сервис *Amazon.com* возвращает все продукты. Чтобы этого избежать, необходимо использовать ресурсы подкачки, и в большинстве случаев требуется разбиение на страницы. Еще одна проблема заключается в том, что этот список содержит данные ресурсов, а не ссылки на них. Один из вариантов — вернуть список ссылок, но это делает использование запроса коллекции бесполезным для отображения списков ресурсов конечному пользователю. По этой причине многие веб-сервисы используют комбинацию свойств и ссылку в ответ коллекции. Еще одно действие, которое клиент может предпринять для изучения ресурса учетных записей — это сделать запрос **OPTIONS** к ресурсу.

```

> OPTIONS /Rest/profile HTTP/1.1
> Accept: application/json
< 200 OK
< Allow: OPTIONS, HEAD, GET, POST, PUT, PATCH, DELETE
<
< {
<   "GET": {
<     "description": "Get a resource",
<     "resourceTemplate":
<       "http://academy.net/Rest/profile/{id}",
<     "parameters": {
<       "$select": {
<         "type": "array/string",
<         "description": "The properties to
<                               be returned for each
<                               resource.",
<       },
<       "$filter" ...
<     }
<   },
<   "POST" ...
< }

```

Ответ **OPTIONS** может не только сообщать клиенту, какие действия доступны для типа ресурса, но также они может быть отфильтрованы в соответствии с привилегиями клиента. Например, если у клиента есть разрешение на просмотр, но не создание, обновление или удаление, значение заголовка ответа **Allow** будет **OPTIONS, HEAD, GET**. Кроме того, различные действия могут быть доступны для разных отдельных ресурсов в зависимости от любого количества факторов (включая привилегии), как показано в следующих двух запросах и их ответах:

```

> OPTIONS /Rest/profile/1 HTTP/1.1
> Accept: application/json
< 200 OK
< Allow: OPTIONS, HEAD, GET, PUT, PATCH, DELETE
< ...
> OPTIONS /Rest/profile/2 HTTP/1.1
> Accept: application/json
< 200 OK
< Allow: OPTIONS, HEAD, GET
< ...

```

Основная проблема возможности обнаружения в веб-сервисах RESTful заключается в том, что нет единого согласованного стандарта содержимого для ответов. Ссылки ответа в первом примере представляют собой *JSON Hypertext Application Language* (HAL), одного из нескольких новых стандартов. Запрос коллекции учетных записей был только стандартной коллекцией JSON. А заголовков **Allow** в ответе **OPTIONS** является стандартным и универсальным, но тело ответа полностью закончено, что и делают многие вендоры.

Отсутствие универсального стандарта привело к тому, что мало поставщиков внедряли возможности обнаружения. В то время Вам необходимо оценить Ваши потребности и технологию, чтобы определить наилучший стандарт для обнаружения. Можно даже не поддерживать открытость и говорить клиентам, что нужно изучать документацию. Внедрение веб-сервисов RESTful в Spring Framework не зависит от ресурсов или представлений, которые Вы принимаете, поэтому принятие одного стандарта над другим не изменит методы **Spring**.

Использование URL-адресов, методов и кодов состояния HTTP

Веб-сервисы RESTful используют URL-адреса запросов для идентификации запрошенных ресурсов и методов HTTP чтобы определить желаемое действие. Они также используют коды состояния HTTP, чтобы показать результат запроса. Различные коды состояния применяются к различным типам запросов. Тем не менее, существует несколько кодов состояния, которые являются универсальными и применяются ко всем типам запросов:

Код	Комментарий
400 Bad Request	Запрос, сделанный клиентом неподдерживаемый или непризнанный. Это происходит из-за не правильного синтаксиса запроса, например, если на POST или PUT требуемое поле ресурса осталось пустым. Тело ответа, в согласованном формате представления, или в формате по умолчанию, должно содержать информацию о том, почему запрос нерабочий
401 Unauthorized	Для доступа к ресурсу или выполнения запрошенного перехода состояния необходимы аутентификация и авторизация. Содержимое ответа зависит от используемого протокола аутентификации (например, OAuth)
403 Forbidden	Клиент, хоть и аутентифицирован, не имеет права доступа к ресурсу или выполнения запрошенного перехода состояния. В зависимости от используемого протокола проверки подлинности, возможно, что повторная аутентификация с более высокими привилегиями может разрешить это условие
404 Not Found	Запрошенный ресурс не существует
405 Method Not Allowed	Запрошенный переход состояния (HTTP-метод) не поддерживается

Код	Комментарий
406 Not Acceptable	Формат представления, запрошенный в заголовке Ассерпт, не поддерживается сервером. Например, клиент может запросить <code>application/xml</code> , но сервер может генерировать только <code>application/json</code> . В этих случаях сервер может просто вернуть стандартное поддерживаемое представление вместо 406 Not Acceptable
415 Unsupported Media Type	Похож на 406 Not Acceptable, указывает, что заголовок запроса <code>Content-Type</code> неподдерживаемого сервером типа. Сервер может включать заголовок ответа Ассерпт, указывающий, какие типы носителей поддерживает сервер. Если и значение заголовка запроса Ассерпт, и значение заголовка запроса <code>Content-Type</code> неподдерживаемого типа — ответ 415 Unsupported Media Type имеет приоритет, поскольку 406 Not Acceptable является необязательным ответом
500 Internal Server Error	При обработке запроса произошла ошибка. Ответ должен в формате представления или по умолчанию, содержать как можно больше информации об ошибке

Рассмотрим синтаксис и семантику каждого метода **HTTP**, поддерживаемого веб-сервисами RESTful, включая другие коды состояния **HTTP**, имеющие значение для этих методов.

OPTIONS

OPTIONS является одним из немногих доступных стандартных механизмов обнаружения. Когда запрос направляется на URL-адрес ресурса с помощью метода **OPTIONS**, сервер должен вернуть ответ **200 OK**, содержащий заголовок **Allow**, содержимое которого является списком методов **HTTP** разделенных запятыми, поддерживаемых ресурсом. Если клиент аутентифицирован,

заголовок **Allow** может вместо этого содержать HTTP-методы, которые клиент имеет право использовать для ресурса. Возможно, сервер также может вернуть тело ответа, описывающее, как использовать каждый метод HTTP для этого ресурса. Тело ответа должно находиться в форме представления, согласованной или представленной по умолчанию при отсутствии какого-либо соглашения. Если метод **PATCH** является одним из поддерживаемых методов, в ответе должен также содержаться заголовок **Allow-Patch**, в котором указывается список представлений разделенных запятыми (типов медиа), разрешенных для запросов **PATCH**, которые могут отличаться от других типов запросов. Существуют некоторые исключения из этих требований:

- Если ресурс не существует, сервер в заголовке **Allow** должен вернуть **404 Not Found** вместо **200 OK**.
- Если на ресурсе не разрешены какие-либо методы без аутентификации, сервер должен вернуть **401 Unauthorized**.
- Если клиент аутентифицирован, но не имеет права на вызов каких-либо действий на ресурсе, сервер должен вернуть **403 Forbidden**.

Запросы **OPTIONS** считаются нулевыми (безопасными), то есть они никогда не при каких обстоятельствах не должны, приводить к изменению любых ресурсов.

Некоторые веб-сервисы обеспечивают использование *Cross-Origin Resource Sharing* (CORS). Этот специальный протокол позволяет авторизованным приложениям брау-

зера обходить политику AJAX одного и того же происхождения, используя набор специализированных заголовков. Если необходимо, чтобы веб-сервисы были напрямую доступны из приложений JavaScript, нужно включить поддержку CORS. Веб-сервисы с поддержкой CORS должны поддерживать запросы OPTIONS для всех ресурсов и при ответе на запросы OPTIONS должны включать заголовок ответа Access-Control-Allow-Methods, содержимое которого идентично заголовку Allow.

HEAD и GET

Всякий раз, когда GET поддерживается и разрешается ресурсом, HEAD также должен поддерживаться и разрешаться. Единственная разница между GET и HEAD заключается в том, что ответ HEAD не должен иметь тело ответа. Ответ HEAD должен иметь заголовки, идентичные тому, что было бы в ответе GET. Запросы GET используются для получения ресурса. URL-адрес, такой как /Rest/profile, указывает, что клиент хочет получить все учетные записи или отфильтрованный список учетных записей. Как правило, команды фильтра, порядка и разбиения на страницы включаются в параметры строки запроса.

URL-адрес запроса, такой как /Rest/profile/1, указывает, что клиент хочет получить одну учетную запись с уникальным идентификатором 1. Некоторые вендоры поддерживают реляционные URL-адреса, такие как /Rest/profile/1/order и /services/Rest/account/1/order/57, соответственно запрос списка заказов (возможно, фильтрованного) для учетной записи 1 и заказа 57 принадлежащего

учетной записи 1. Эта рекурсия может, теоретически, продолжаться до тех пор, пока длина URL не достигнет предела. */Rest/profile/1/order/57/item/64* может вернуть определенную позицию из заказа для учетной записи.

В приведенных URL-адресах сегменты счета и порядка пути являются единственными. Вместо этого URL-адреса могут содержать сегменты учетных записей и заказов. Что использовать, зависит от Вас, необходимо сделать выбор и оставаться постоянным в своих ресурсах. В целом использование особых сегментов пути приводит к сокращению URL-адресов, которые при необходимости требуют большего количества содержимого в строке запроса.

На успешные запросы **GET** и **HEAD** сервер должен ответить **200 OK** и включить ресурсы, запрошенные в согласованном (или по умолчанию) представлении в теле ответа для запросов **GET**.

Запросы **GET** и **HEAD** также недействительны и не должны иметь побочных эффектов для ресурсов на сервере.

POST

Запросы **POST** используются для создания новых ресурсов на сервере. Запрос **POST** всегда должен быть направлен на URI коллекции (*/Rest/profile*), но также может быть направлен на URI подчиненной коллекции (*/Rest/profile/1/order*). Запрос **POST** для URI отдельного элемента (*/Rest/profile/1*) должен приводить к ответу **405 Method Not Allowed**. На успешный запрос **POST** сервер создает запрашиваемый ресурс и возвращает **201 Created**. Ответ

должен включать заголовок **Location** с URL-адресом для вновь созданного ресурса.

Например, запрос **POST** */Rest/profile* для создания новой учетной записи может вернуть *http://www.academy.com/Rest/profile/648* в заголовке **Location**. Тело ответа должно быть созданным ресурсом, потому что он будет возвращен с запросом **GET** в URL-адрес в заголовке **Location**.

Запрос **POST** приводит к изменению одного или нескольких ресурсов, и создание нескольких идентичных запросов **POST** приводит к созданию нескольких ресурсов.

PUT

Запрос **PUT** приводит к замене ресурса на сервере. По этой причине запросы **PUT** используются для обновления существующих ресурсов. Запросы **PUT**, в отличие от запросов **POST**, никогда не принимаются в коллекции URI. Вместо этого они создаются для отдельных элементов URI и подчиненных элементов URI (*/Rest/profile/1*, */Rest/profile/1/order/566*). Запрос **PUT** для URI коллекции или подчиненной URI коллекции должен приводить к ответу **405 Method Not Allowed**. Ответ на успешный запрос **PUT** должен быть **204 No Content**, а его тело — пустым.

Два или более последовательных одинаковых запроса **PUT** не должны иметь побочных эффектов, кроме тех, которые указаны в первом запросе **PUT**.

Проблемой является то, что обновляемый ресурс включает в себя временную метку последнего изменения или номер версии. Служба должна обновлять временную метку ресурса и/или номер версии для запросов **PUT**

только в том случае, если основной объект действительно изменился. Такое требование может быть довольно сложным для реализации, поэтому многие поставщики реализуют **PUT**, документируя, что метка времени и номер версии будут по-прежнему обновляться в одинаковых запросах **PUT**.

PATCH

Запросы **PATCH** очень похожи на запросы **PUT**. **PATCH** — относительно новый метод HTTP. Он не является частью исходной спецификации HTTP/1.1. Запросы **PATCH**, также как и **PUT**-запросы, предназначены для обновления ресурсов в отдельных URI-элементах. Однако запрос **PATCH** предписывает только частичное обновление ресурса, а не полную замену ресурса.

Например, если запрос **PATCH** в `/Rest/profile/1/order/568` содержит только свойство `shippingAddress`, тогда будет обновляться только адрес доставки в заказе. Другие свойства останутся неизменными (за исключением, возможно, временных меток и номеров версий). Чтобы поддерживать **PATCH**, Ваше приложение должно принять очень гибкий набор свойств в объекте запроса, а затем обновить только те свойства ресурса, которые присутствуют в запросе. Вы не можете просто использовать проверку на `null`, потому что **PATCH** может преднамеренно установить значение свойства в `null`.

Успешный ответ на запрос **PATCH** должен быть **200 OK** или **204 No Content**. Вы решаете, следует ли возвращать полный, обновленный объект или ничего в теле

ответа. Если поддерживается **PATCH**, поддерживается тип запрашиваемого носителя, и содержимое запроса **PATCH** воспринимается и анализируется успешно, но сервер все еще не может применить патч (поскольку патч сделает объект недействительным), сервер должен ответить **422 Unprocessable Entity**. Если клиент использует заголовок запроса **If-Match** или **If-Unmodified-Since**, чтобы определить предварительное условие для патча и это условие не выполняется, сервер должен вернуть **412 Precondition Failed**. Если несколько запросов пытаются исправить ресурс одновременно, и это не разрешено, сервер должен вернуть **409 Conflict**.

DELETE

Запрос **DELETE** используется для удаления ресурса. Запрос **DELETE** может быть применен к URI отдельного элемента — в этом случае этот единственный ресурс удаляется — или он может быть применен к URI коллекции (возможно, фильтрованной) — в этом случае все соответствующие ресурсы удаляются. Удаление нескольких ресурсов нежелательно, поэтому эта возможность обычно не поддерживается. При успешном удалении ресурса сервер должен ответить **200 OK** с удаленным ресурсом в теле ответа или **204 No Content** без содержимого в теле ответа. Если по какой-то причине сервер принимает команду **delete**, но не может выполнить ее немедленно (возможно, ресурс используется), он может вернуть **202 Accepted**. В этом случае тело ответа должно содержать URL-адрес ресурса, который клиент может

использовать для отслеживания запроса и проверки его состояния позднее.

Удаление ресурса может привести к установке флага на этом ресурсе, но сохранение его данных (*soft delete*), или это может привести к фактическому и безотзывному устранению данных ресурса (*hard delete*). При использовании *soft delete* несколько идентичных запросов **DELETE** всегда возвращают один и тот же ответ и не имеют дополнительных побочных эффектов. Однако при использовании *hard delete* второй идентичный запрос **DELETE** всегда приводит к **404 Not Found**, потому что ресурс больше не существует.

Настройка и тестирование веб-сервисов RESTful с помощью Spring MVC

Одна из замечательных особенностей веб-сервисов RESTful заключается в том, что они аккуратно работают в рамках существующей инфраструктуры HTTP, и это ясно показано в Spring Framework. С технической точки зрения, Вам не нужно делать ничего особенного, чтобы создавать веб-сервисы RESTful. Вы можете просто создать **@Controller**, добавить к нему некоторые методы **@RequestMapping** и начать работу с веб-сервисом RESTful. Маловероятно, что Вы захотите повторно использовать существующий контекст приложения **DispatcherServlet** для своих веб-сервисов. Некоторые компоненты мы уже предварительно настроили в контексте этого приложения. Существуют различные преобразователи сообщений для обработки всех ви-

дов форматов запросов и ответов. Есть `ViewResolver` и `RequestToViewNameResolver`, которые совершенно не нужны для ответа на запросы веб-сервиса. Тогда логично создать отдельный `DispatcherServlet` и контекста приложения, настроенный соответствующим образом, чтобы сформировать веб-сервисы RESTful.

Разделение контроллеров с стереотипными аннотациями

Аннотация `@Controller` служит двум целям. Как `@Component`, он отвечает за маркировку контроллеров, поскольку beans управляемые Spring имеют право на инкапсуляцию и инъекцию зависимостей. Однако, в контексте Spring MVC, он также отвечает за маркировку bean-компонентов, которые нужно отсканировать для `@RequestMapping`. Маркировка `bean` с помощью `@RequestMapping` не может отвечать на запросы, если она не аннотируется с помощью `@Controller`. Таким образом, REST-контроллеры, как и веб-контроллеры, должны быть помечены `@Controller`.

Это создает определенные проблемы. Если не нужно, чтобы REST-контроллеры были подхвачены сканированием компонентов для веб `DispatcherServlet`, и чтобы веб-контроллеры были подхвачены сканированием компонентов для REST `DispatcherServlet`, необходимо разместить веб-контроллеры в отдельном пакете от REST-контроллеров и проверять компоненты только в соответствующем пакете в каждом контексте приложения. Это, безусловно, легкий подход, и в большинстве случаев он

может удовлетворить любые варианты использования, которые могут возникнуть. Однако есть еще один способ решить эту проблему: мета-аннотации.

В Spring Framework стереотипные аннотации `@Controller`, `@Service` и `@Repository` являются унаследованными от `@Component`. Можно создавать свои собственные стереотипные аннотации, аннотируя их с помощью существующего `@Component`. Можно аннотировать пользовательскую аннотацию с помощью `@Controller`, например, предоставляя аннотации тот же смысл, что и `@Controller`. Это означает, что пользовательские аннотации могут маркировать контроллеры для сопоставления запросов, что довольно хорошо решает проблему сегрегации контроллера. В проекте Web-Academy создадим аннотации `@com.academy.config.annotation.WebController` и `@com.academy.config.Annotation.RestEndpoint`.

```
@Target({ ElementType.TYPE })
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Controller

public @interface WebController {
    String value() default "";
}

@Target({ ElementType.TYPE })
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Controller

public @interface RestEndpoint {
    String value() default "";
}
```

Эти аннотации практически идентичны, но их смысловые значения имеют важные отличия. Обе указывают, что целевой компонент является контроллером, подходящим для сопоставления запросов. Тем не менее, `@WebController` аннотирует контроллеры традиционных веб-запросов, в то время как `@RestEndpoint` обозначает конечные точки веб-сервиса RESTful. Одиночный атрибут `value` выполняет ту же задачу, что и в других стереотипных аннотациях: он предоставляет способ указания имени компонента, которым переопределяется шаблон имени компонента по умолчанию.

Создание отдельных контекстов приложений Web и REST

Эти пользовательские аннотации сами по себе не приносят никакой пользы. Можно разместить их на своих контроллерах и конечных точках, но класс `ServletContextConfiguration`, который мы использовали, отображает их все в своем сканировании компонентов для `@Controller`. Таким образом, необходимо обновить эту конфигурацию для сканирования только для аннотации `@WebController`. Чтобы устранить любую двусмысленность в отношении его назначения, также рекомендуется переименовать этот класс в `WebServletContextConfiguration`.

В коде этого класса изменились типы стереотипных аннотаций, которые маркируют компоненты, принадлежащие этому контексту приложения, и добавился компонент `Spring Validator`.

```

@Configuration
@EnableWebMvc
@ComponentScan(
    basePackages = "com.academy.site",
    useDefaultFilters = false,
    includeFilters = @ComponentScan.
                        Filter(WebController.class)
)
public class WebServletConfiguration extends
                        WebMvcConfigurerAdapter {

    ...
    @Inject SpringValidatorAdapter validator;
    ...
    @Override
    public Validator getValidator() {
        return this.validator;
    }
}

```

Теперь нужно создать новый контекст приложения **RestServletConfiguration**, который ищет **@RestEndpoint** и предназначен для веб-сервисов RESTful.

```

@Configuration
@EnableWebMvc
@ComponentScan(
    basePackages = "com.academy.site",
    useDefaultFilters = false,
    includeFilters =
        @ComponentScan.Filter({RestEndpoint.class,
                                RestEndpointAdvice.class})
)
public class RestServletConfiguration extends
                        WebMvcConfigurerAdapter {
    @Inject ObjectMapper objectMapper;
}

```

```

@Inject Marshaller marshaller;
@Inject Unmarshaller unmarshaller;
@Inject SpringValidatorAdapter validator;
@Override
public void configureMessageConverters(
    List<HttpMessageConverter<?>> converters) {
    converters.add(new
        SourceHttpMessageConverter<>());
    MarshallingHttpMessageConverter xmlConverter =
        new MarshallingHttpMessageConverter();
    xmlConverter.setSupportedMediaTypes(
        Arrays.asList(
            new MediaType("application", "xml"),
            new MediaType("text", "xml")
        ));
    xmlConverter.setMarshaller(this.marshaller);
    xmlConverter.setUnmarshaller(this.unmarshaller);
    converters.add(xmlConverter);
    MappingJackson2HttpMessageConverter jsonConverter =
        new MappingJackson2HttpMessageConverter();
    jsonConverter.setSupportedMediaTypes(
        Arrays.asList(
            new MediaType("application", "json"),
            new MediaType("text", "json")
        ));
    jsonConverter.setObjectMapper(this.objectMapper);
    converters.add(jsonConverter);
}
@Override
public void configureContentNegotiation(
    ContentNegotiationConfigurer configurer) {
    configurer.favorPathExtension(false)
        .favorParameter(false)
        .ignoreAcceptHeader(false)
        .defaultContentType(MediaType
            .APPLICATION_JSON);
}

```

```

@Override
public Validator getValidator() {
    return this.validator;
}
@Bean
public LocaleResolver localeResolver() {
    return new AcceptHeaderLocaleResolver();
}
}

```

Этот класс похож на [WebServletContextConfiguration](#), но также имеет важные отличия. В нем отсутствует разрешение просмотра и многопользовательская поддержка, потому что эти функции не нужны. Он содержит меньше конвертеров сообщений, фокусируясь только на задачах преобразования передаваемых данных в JSON, XML и обратно. Метод [configureContentNegotiation](#) поддерживает только [Accept](#) заголовки для согласования содержимого (стандарта RESTful). Так же необходим Spring Validator. Чтобы использовать валидацию bean в веб-сервисах RESTful можно пометить параметры сущности [@RequestBody](#) с помощью [@Valid](#).

Теперь нам необходимо правильно загрузить REST приложение с новым контекстом. Это, как обычно, делается в классе [Bootstrap](#).

```

AnnotationConfigWebApplicationContext webContext =
    new AnnotationConfigWebApplicationContext();
webContext.register(WebServletContextConfiguration.class);
ServletRegistration.Dynamic dispatcher =
    container.addServlet("springWebDispatcher",
        new DispatcherServlet(webContext)
    );

```

```

dispatcher.setLoadOnStartup(1);
dispatcher.setMultipartConfig(new MultipartConfigElement(
    null, 20_971_520L, 41_943_040L, 512_000));
dispatcher.addMapping("/");
AnnotationConfigWebApplicationContext restContext =
    new AnnotationConfigWebApplicationContext();
restContext.register(RestServletContextConfiguration.
    class);
DispatcherServlet servlet =
    new DispatcherServlet(restContext);
servlet.setDispatchOptionsRequest(true);
dispatcher = container.addServlet(
    "springRestDispatcher", servlet);
dispatcher.setLoadOnStartup(2);
dispatcher.addMapping("/rest/*");

```

Новый сервлет не имеет многострочной конфигурации. Можно использовать множественные запросы с веб-сервисами RESTful. Свойство `DispatcherServlet dispatchOptionsRequest` принимает значение `true`. Обычно `DispatcherServlet` игнорирует запросы `OPTIONS` и отменяет поведение `HttpServlet` по умолчанию. Чтобы поддерживать обнаружение необходимо указать `DispatcherServlet` отправить запросы `OPTIONS` на конечные точки RESTful точно так же, как и любой другой запрос.

Ошибки обработки условий в веб-сервисах RESTful

Потребители API RESTful ожидают осмысленные коды состояний и сообщения об ошибках, в случае если что-то пойдет не так, и не хотят возвращать в контейнере стандартные страницы ошибок HTML-формата.

Поддержка обработки ошибок в Spring Framework значительно изменилась за последние годы. Рассмотрим функции обработки ошибок добавленные в Spring 3.0 и 3.2.

Изменение кодов ответа HTTP

Одна из ситуаций, которую необходимо обрабатывать в веб-сервисах — это запросы к несуществующим ресурсам. Spring может вернуть **404 Not Found**, когда запросы поступают к URL-адресам без сопоставлений методов. Если же сделан запрос `/students/rest/profile/15`, а учетная запись с идентификатором 15 не существует, то согласно стандартам RESTful необходимо также вернуть **404 Not Found**. Самый простой способ сообщить об этой проблеме — выбросить исключение. Итак, создайте исключение.

```
@ResponseStatus(HttpStatus.NOT_FOUND)
public class ResourceNotFoundException extends
    RuntimeException { ... }
```

Аннотация `@org.springframework.web.bind.annotation.ResponseStatus`, добавленная в Spring 3.0 в объявлении исключения сообщает Spring Framework о возврате кода состояния **404 Not Found**. Без нее, бросок такого исключения из метода контроллера приведет к ошибке **500 Internal Server Error**.

Объявление обработчиков исключений

Еще одна вещь, которую стоит рассмотреть — это проблема валидации компонентов. Аннотация `@org.springframework.web.bind.annotation.ExceptionHandler`

добавленная в Spring 3.0, позволяет пометить метод как обработчик исключений. Она имеет почти ту же семантику, что и [@RequestMapping](#), позволяя возвращать большинство одинаковых возвращаемых типов и принимать большинство тех же аргументов метода.

```
@ExceptionHandler({ ConstraintViolationException.  
                    class })  
@ResponseStatus(HttpStatus.BAD_REQUEST)  
@ResponseBody  
public ErrorResponse handleBeanValidationError(  
    ConstraintViolationException e)    {...}
```

Проблема в этом шаблоне заключается в том, что он требует наличия необходимых методов [@ExceptionHandler](#) на всех контроллерах. Можно наследоваться от базового контроллера с указанными методами, но это не позволяет при необходимости расширять другие классы.

Использование шаблона рекомендаций контроллера

Spring 3.2 представила концепцию шаблона `controller advice` с аннотацией [@org.springframework.web.bind.annotation.ControllerAdvice](#). Классы, помеченные [@ControllerAdvice](#), который унаследован от [@Component](#), представляют собой управляемые компоненты Spring, которые «дают советы» контроллерам. Можно добавить методы [@ExceptionHandler](#), [@InitBinder](#) и [@ModelAttribute](#) в класс рекомендаций контроллера, а также рекомендации по предоставлению этих методов для всех контроллеров. Будучи классом [@Component](#), классы [@ControllerAdvice](#) обычно автоматически создаются в контексте корневого

приложения. Если не нужно, чтобы такой совет применялся к веб-контроллерам и контроллерам конечных точек, нужно исключить эту аннотацию из сканирования корневого компонента.

```
...
@ComponentScan(
    basePackages = "com.academy.site",
    excludeFilters =
        @ComponentScan.Filter({Controller.class,
                                ControllerAdvice.class})
)
public class RootContextConfiguration
...
```

Можно создать **controller advice** для контроллеров конечных точек, но в какой-то момент понадобится создать совет для веб-контроллеров. Таким образом, вместо непосредственного использования **@ControllerAdvice** было бы лучше создать собственную стереотипную аннотацию, специфичную для контроллеров конечных точек RESTful.

```
@Target({ ElementType.TYPE })
@Retention(RetentionPolicy.RUNTIME)
@Documented
@ControllerAdvice
public @interface RestEndpointAdvice {
    String value() default "";
}
```

Последнее, что Вам нужно сделать — настроить сканирование компонентов для контекста приложения REST, чтобы он подхватывал классы **@RestEndpointAdvice**.

```

...
@ComponentScan (
    basePackages = "com.academy.site",
    useDefaultFilters = false,
    includeFilters =
        @ComponentScan.Filter({RestEndpoint.class,
                                RestEndpointAdvice.class})
)
public class RestServletContextConfiguration extends
    WebMvcConfigurerAdapter
...

```

Теперь можно создать рекомендации контроллера для обработки ошибок — *com.academy.site.exception.RestExceptionHandler*. Он имеет два внутренних класса: *ErrorItem* и *ErrorResponse*, которые помогают ему отправлять сообщения об ошибках пользователю. Одиночный метод *handle*, аннотированный *@ExceptionHandler*, преобразует все нарушения ограничений в *ErrorItem*, помещает их в *ErrorResponse* и возвращает *ErrorResponse*, завернутый в *ResponseEntity*, с кодом состояния *400 Bad Request*.

Аннотация *@ExceptionHandler* определяет какие исключения обрабатывает метод. Для того чтобы один метод обрабатывал несколько ошибок нужно добавить несколько исключений в эту аннотацию. Поскольку Spring всегда вызывает наиболее специфическое совпадение, также можно создать метод *catchall handle*, который принимал бы любой экземпляр *Exception*. Spring вызовет метод, который наиболее точно соответствует выбранному исключению, а затем — метод *catchall*.

```

@RestEndpointAdvice
public class RestExceptionHandler {
    @ExceptionHandler({ConstraintViolationException.class})
    public ResponseEntity<ErrorResponse>
        handle(ConstraintViolationException e)
    {
        ErrorResponse errors = new ErrorResponse();
        for(ConstraintViolation violation :
            e.getConstraintViolations()) {
            ErrorItem error = new ErrorItem();
            error.setCode(violation.
                getMessageTemplate());
            error.setMessage(violation.getMessage());
            errors.addError(error);
        }
        return new ResponseEntity<>(errors,
            HttpStatus.BAD_REQUEST);
    }
    @SuppressWarnings("unused")
    public static class ErrorItem {
        private String code;
        private String message;
        @XmlAttribute
        public String getCode() {
            return code;
        }
        public void setCode(String code) {
            this.code = code;
        }
        @XmlValue
        public String getMessage() {
            return message;
        }
        public void setMessage(String message) {
            this.message = message;
        }
    }
}

```

```

    }
    @SuppressWarnings("unused")
    @XmlElement(name = "errors")
    public static class ErrorResponse {
        private List<ErrorItem> errors =
            new ArrayList<>();
        @XmlElement(name = "error")
        public List<ErrorItem> getErrors() {
            return errors;
        }
        public void setErrors(List<ErrorItem> errors) {
            this.errors = errors;
        }
        public void addError(ErrorItem error) {
            this.errors.add(error);
        }
    }
}

```

Сопоставление запросов *RESTful* с методами контроллера

Чтобы создать веб-сервис, сначала нужно создать бизнес-уровень для его отображения. Интерфейс *Profile-Service* служит продолжением примера учетной записи с URL-адресами. Он определяет методы для перечисления, получения, сохранения и удаления учетных записей студентов.

```

@Validated
public interface ProfileService {
    @NotNull
    List<Profile> getAllProfiles();
    Profile getProfile(long id);
}

```

```

    Profile saveProfile(
        @NotNull(message = "You cannot save
                        empty profile.")
        @Valid Profile profile
    );
    void deleteProfile(long id);
}

```

DefaultProfileService реализовывает стандартный накопительный учет в памяти.

```

@Service
public class DefaultProfileService implements
    ProfileService {
    private final Map<Long, Profile> profileRepository =
        new Hashtable<>();
    private volatile long profileIdSequence = 1L;
    @Override
    public List<Profile> getAllProfiles() {
        List<Profile> profiles =
            new ArrayList<>(this.
                profileRepository.values());
        profiles.sort((p1, p2) -> p1.getId() <
            p2.getId() ? -1 : 1
        );
        return profiles;
    }
    @Override
    public Profile getProfile(long id) {
        return this.profileRepository.get(id);
    }
    @Override
    public Profile saveProfile(Profile profile) {
        if(profile.getId() < 1)
            profile.setId(this.nextProfileId());
    }
}

```

```

        profile.setLastModified(Instant.now());
        this.profileRepository.put(profile.getId(),
                                   profile);
        return profile;
    }

    @Override
    public void deleteProfile(long id) {
        this.profileRepository.remove(id);
    }
    private synchronized long nextProfileId() {
        return this.profileIdSequence++;
    }
}

```

Реализация этого сервиса, управляемого в контексте корневого приложения, доступна как в Web, так и в **REST** контексте приложения **DispatcherServlet**, как указано в верхней части **ProfileController** и **ProfileRestEndpoint**.

```

@WebController
public class ProfileController {
    @Inject ProfileService profileService;
    ...
}

@RestEndpoint
public class ProfileRestEndpoint {
    @Inject ProfileService profileService;
    ...
}

```

ProfileController — стандартный веб-контроллер, который используется для создания.

```

@RequestMapping(value = "profile/list",
                 method = RequestMethod.GET)

public String list(Map<String, Object> model) {
    model.put("profiles",
             this.profileService.getAllProfiles());
    return "profile/list";
}

```

Использование **@RequestMapping** в конечной точке RESTful очень похоже на использование его в веб-контроллере. С технической точки зрения можно написать методы обработчика запросов со всеми теми же параметрами и возвращаемыми типами, хотя на самом деле это не так. Не имеет смысла возвращать **ModelAndView**, **View**, строковое представление имени, **Model**, **Map**, атрибут модели, **Callable** или **DeferredResult** из метода обработчика веб-сервиса RESTful. Обычно возвращается объект сущности ответа **@ResponseBody** или явный **ResponseEntity <?>**, который может быть возвращен клиенту. И **void** или **ResponseEntity <void>**, если контент не должен быть возвращен клиенту. Параметры метода состоят из **@PathVariable** и **@RequestParam**, а для запросов **POST** и **PUT** — объектов запроса **@RequestBody**. Как и в случае с веб-контроллерами, объекты запроса должны представлять собой объекты формы или команды, не имеющие полей, которые не должны устанавливать Ваши клиенты (идентификатор ресурса). Объекты ответа, возвращаемые из методов, могут быть теми же бизнес-объектами, с которыми Вы обычно работаете, но не стоит возвращать коллекции

любого типа или [Map](#). Они могут быть легко преобразованы в представление JSON, но не в XML.

Первые методы в [ProfileRestEndpoint](#) — это методы обнаружения, которые отвечают на запросы [OPTIONS](#). Существует два метода: один для запросов на сбор и один для индивидуальных запросов ресурсов. Эти методы довольно просты и всегда возвращают одинаковые допустимые параметры, но здесь можно выполнить проверки безопасности и возвращать только те параметры, к которым имеет доступ аутентифицированный клиент.

```
@RequestMapping(value = "profile",
                  method = RequestMethod.OPTIONS)
public ResponseEntity<Void> discover() {
    HttpHeaders headers = new HttpHeaders();
    headers.add("Allow", "OPTIONS, HEAD, GET, POST");
    return new ResponseEntity<>(null, headers,
                               HttpStatus.NO_CONTENT);
}

@RequestMapping(value = "profile/{id}",
                  method = RequestMethod.OPTIONS)
public ResponseEntity<Void>
    discover(@PathVariable("id") long id) {
    if(this.profileService.getProfile(id) == null)
        throw new ResourceNotFoundException();
    HttpHeaders headers = new HttpHeaders();
    headers.add("Allow",
                "OPTIONS, HEAD, GET, PUT, DELETE");
    return new ResponseEntity<>(null, headers,
                               HttpStatus.NO_CONTENT);
}
```

Как указано в ответе **OPTIONS**, клиенты могут получить (GET) либо коллекцию, либо отдельные ресурсы. Это реализовано в методе **read**. Чтение коллекции небезопасно, потому что оно не показывает результаты. Если в базе данных 1 000 000 учетных записей, этот метод вернет все 1 000 000 из них. Необходимо использовать параметры запроса или заголовки для фильтрации и ограничения результатов, возвращаемых этим методом. Также лучше автоматически ограничивать количество возвращаемых результатов, если клиент не указал какие-либо фильтры или лимиты. Эти методы демонстрируют использование **@ResponseBody** для возврата объектов ответа и **@ResponseStatus** для указания с каким статусом **HTTP Spring** должен ответить на запрос.

```
@RequestMapping(value = "profile",
                  method = RequestMethod.GET)
@ResponseBody @ResponseStatus(HttpStatus.OK)
public ProfileList read() {
    ProfileList list = new ProfileList();
    list.setValue(this.profileService.getAllProfiles());
    return list;
}

@RequestMapping(value = "profile/{id}",
                  method = RequestMethod.GET)
@ResponseBody @ResponseStatus(HttpStatus.OK)
public Profile read(@PathVariable("id") long id) {
    Profile profile = this.profileService.getProfile(id);
    if(profile == null)
        throw new ResourceNotFoundException();
    return profile;
}
```

Индивидуальное чтение возвращает один профиль и выдает исключение `ResourceNotFoundException` (404 Not Found), если запрашиваемый профиль не найден. В этом случае `@ResponseStatus` для метода игнорируется, а для исключения (или обработчика исключений) получает более высокий приоритет. Между тем, чтение коллекции возвращает `ProfileList` (вместо `List<Profile>`). Если веб-сервис поддерживает только JSON, нужен `List<Profile>`, а не объект `ProfileList`. `ProfileList` позволяет также отвечать на XML. На это необходимо обратить особое внимание при написании конечных точек веб-сервиса RESTful. Представление объектов запроса и ответа намного проще с JSON, чем с XML, и поэтому необходимо учитывать некоторые особенности. Некоторые поставщики услуг предпочитают поддерживать только запросы и ответы JSON, чтобы избежать этих сложностей. Вы также можете создавать отдельные методы обработчика для ответов XML и JSON.

```
@XmlElement(name = "profiles")
public static class ProfileList {
    private List<Profile> value;

    @XmlElement(name = "profile")
    public List<Profile> getValue() {
        return value;
    }

    public void setValue(List<Profile> profiles) {
        this.value = profiles;
    }
}
```

Методы `create` и `update` (POST и PUT), показанные в следующем фрагменте кода, отвечают за добавление новых профилей студентов и сохранение изменений в существующих. Обновление генерирует исключение `ResourceNotFoundException`, как и индивидуальный метод `read`, если обновляемый ресурс не существует. Он также возвращает `void` и отвечает `204 No Content`, когда ресурс успешно обновлен, тогда как `create` отвечает `201 Created`, возвращает созданный объект и включает заголовок `Location` в ответе с URL-адресом для созданного ресурса. Он использует файл `org.springframework.web.servlet.support.ServletUriComponentsBuilder` для создания URL-адреса, который содержит имя домена и контекстный путь приложения.

```
@RequestMapping(value = "profile",
    method = RequestMethod.POST)
public ResponseEntity<Profile>
    create(@RequestBody ProfileForm form) {
    Profile profile = new Profile();
    profile.setName(form.getName());
    profile.setHomeAddress(form.getHomeAddress());
    profile.setMobileNumber(form.getMobileNumber());
    profile = this.profileService.saveProfile(profile);
    String uri = ServletUriComponentsBuilder
        .fromCurrentServletMapping()
        .path("/profile/{id}")
        .buildAndExpand(profile.getId())
        .toString();
    HttpHeaders headers = new HttpHeaders();
    headers.add("Location", uri);
    return new ResponseEntity<>(profile,
        headers, HttpStatus.CREATED);
}
```

```

@RequestMapping(value = "profile/{id}",
                  method = RequestMethod.PUT)
@ResponseStatus(HttpStatus.NO_CONTENT)
public void update(@PathVariable("id") long id,
                  @RequestBody ProfileForm form) {
    Profile profile = this.profileService.getProfile(id);
    if(profile == null)
        throw new ResourceNotFoundException();
    profile.setName(form.getName());
    profile.setHomeAddress(form.getHomeAddress());
    profile.setMobileNumber(form.getMobileNumber());
    this.profileService.saveProfile(profile);
}

```

Метод **delete** подобно **read** и **update** генерирует исключение **ResourceNotFoundException**, если ресурс не существует. И удаляет ресурс, если он существует, и отвечает **204 No Content**.

```

@RequestMapping(value = "profile/{id}",
                  method = RequestMethod.DELETE)
@ResponseStatus(HttpStatus.NO_CONTENT)
public void delete(@PathVariable("id") long id) {
    if(this.profileService.getProfile(id) == null)
        throw new ResourceNotFoundException();
    this.profileService.deleteProfile(id);
}

```

Улучшение обнаружения с конечной точкой индекса

Методы обнаружения в **ProfileRestEndpoint** помогают клиентам узнать о профиле и о том, что они могут с ним сделать, но не завершают механизм обнаружения RESTful.

Как правило, создается конечная точка индекса, в которой перечислены доступные ресурсы веб-сервиса. В простейшем смысле это статический список ссылок в коде, созданный с именем домена и контекстом приложения — [IndexRest-Endpoint](#). Он возвращает ссылки на ресурсы, используя стандарт HAL, и поскольку этот стандарт имеет очень разные представления XML и JSON, он использует некоторые вспомогательные POJO и два разных метода обнаружения для возврата соответствующего ответа. Атрибут [@RequestMapping](#) позволяет определить, какой метод следует вызывать на основе заголовка [Accept](#).

```
@RestEndpoint
public class IndexRestEndpoint {
    @RequestMapping(value = {"", "/"},
                    method = RequestMethod.GET,
                    produces = {"application/json",
                               "text/json"})
    @ResponseBody @ResponseStatus(HttpStatus.OK)
    public Map<String, Object> discoverJson() {
        ServletUriComponentsBuilder builder =
            ServletUriComponentsBuilder.
                fromCurrentServletMapping();
        Map<String, JsonLink> links =
            new Hashtable<>(2);
        links.put("self",
            new JsonLink(builder.path("").
                build().toString()));
        links.put("profile",
            new JsonLink(builder.path("/profile").
                build().toString()));
        Map<String, Object> response =
            new Hashtable<>(1);
        response.put("_links", links);
    }
}
```

```

        return response;
    }
    @RequestMapping(value = {"", "/"},
                    method = RequestMethod.GET,
                    produces = {"application/xml",
                               "text/xml"})
    @ResponseBody @ResponseStatus(HttpStatus.OK)
    public Resource discoverXml() {
        ServletUriComponentsBuilder builder =
            ServletUriComponentsBuilder.
                fromCurrentServletMapping();
        Resource resource = new Resource();
        resource.addLink(new Link("self", builder.
            path("").build().toString()));
        resource.addLink(new Link("profile",
            builder.path("/profile").build().
                toString()));
        return resource;
    }
    @SuppressWarnings("unused")
    public static class JsonLink {
        private String href;
        public JsonLink(String href) {
            this.href = href;
        }
        @XmlAttribute
        public String getHref() {
            return href;
        }
        public void setHref(String href) {
            this.href = href;
        }
    }
    @SuppressWarnings("unused")
    public static class Link extends JsonLink {
        private String rel;
        public Link(String rel, String href) {

```

```

        super(href);
        this.rel = rel;
    }
    @XmlAttribute
    public String getRel() {
        return rel;
    }
    public void setRel(String rel) {
        this.rel = rel;
    }
}
@SuppressWarnings("unused")
@XmlRootElement
public static class Resource {
    private List<Link> links = new ArrayList<>();
    @XmlElement(name = "link")
    public List<Link> getLinks() {
        return links;
    }
    public void setLinks(List<Link> links) {
        this.links = links;
    }
    public void addLink(Link link) {
        this.links.add(link);
    }
}
}

```

Тестирование конечных точек веб-службы

Тестирование веб-сервисов, будь то RESTful или SOAP, представляет собой интересную задачу. В отличие от стандартных веб-страниц, Вы не можете просто открыть браузер и перейти к URL приложения. Можно создать пользовательский интерфейс для сервиса, и на самом деле

так и тестируются веб-сервисы. Этот способ имеет недостаток: что, если Ваш пользовательский интерфейс также содержит ошибки? Для тестирования работы веб-сервиса без других «переменных» необходим так называемый инструмент для тестирования веб-сервисов.

Выбор инструмента тестирования

Существует множество доступных инструментов, и каждый из них зависит от того, какой тип тестирования необходимо выполнить. Разумеется, можно тестировать свои веб-сервисы точно так же, как бизнес-логику и хранилища, используя автоматизированный инструмент тестирования модулей, такой как JUnit или TestNG. Существует также интеграционное тестирование, которое включает в себя тестирование многих компонентов системы, взаимодействующих, как в реальном мире. Другим распространенным требованием к тестированию является нагрузочное тестирование, которое помогает определить характеристики производительности, узкие места и точки разлома системы. Для этого доступно множество инструментов, включая бесплатные инструменты, такие как [JMeter](#) и [The Grinder](#), и платные корпоративные инструменты, такие как [NeoLoad](#) и [LoadRunner](#).

Общий подход к тестированию, который мы с Вами использовали — это функциональное тестирование. Иногда самый простой и быстрый способ оценить, работает ли система — открыть ее и проверить. Мобильное приложение достаточно использовать на телефоне или планшете. Функциональное тестирование десктопного приложения также включает открытие и использование его на рабочей

станции или ноутбуке. Тестирование веб-приложения так же просто, необходимо открыть браузер и ввести URL-адрес приложения.

Возникает вопрос: как функционально тестировать веб-сервис? Вам нужен инструмент, который позволяет легко создавать и обрабатывать HTTP-запросы, а также просматривать ответы. Инструмент командной строки `cURL` является конкурентоспособным, но он требует большого количества служебных и ручных команд для каждого запроса, кроме того затруднена любая аутентификация, более сложная, чем базовая аутентификация HTTP. Было бы полезно, если бы инструмент мог запомнить аутентификацию, предустановить значения заголовков и форматировать ответы JSON и XML, чтобы сделать их более читаемыми. Одним из вариантов функционального тестирования веб-сервисов, который поддерживает эти функции, является Fiddler. Тем не менее, Fiddler имеет много других применений и может быть немного громоздким для использования именно для этой цели. Существует несколько удобных для тестирования веб-сервисов RESTful плагинов для веб-браузеров Chrome и Firefox. Сейчас мы будем использовать плагин RESTClient Firefox, но есть множество инструментов для выбора, и Вы должны выбрать тот, который лучше всего Вам подходит.

Выполнение запросов к Вашему веб-сервису

После установки расширения **RESTClient** в браузер выполните следующие действия:

1. Откройте **RESTClient**, нажав на кнопку с логотипом.
2. Чтобы создать новый заголовок, перейдите в меню **Headers** выберите **Custom Header**.

3. В появившейся форме в поле **Name** введите **Content-Type** и в поле **Attribute Value** — **application/json**, установите флажок **Favorite header** и нажмите **OK**.
4. Повторите шаг 3 со следующими значениями: **Content-Type** — **application/xml**, **Accept** — **application/json** и **Accept** — **application/xml**. Данная манипуляция позволяет сохранить заголовки для быстрого доступа, которые можно легко выбрать при необходимости. Затем необходимо очистить панель **Headers** в **RESTClient** нажав кнопку **Remove all**.
5. Скомпилируйте и запустите приложение **Web-Academy** из **IDE**.
6. Теперь из меню быстрого доступа **Headers** добавьте заголовки **Content-Type: application/json** и **Accept: application/json**. В поле **Method** из выпадающего списка выберите **GET**, в **URL** введите **http://localhost:8080/students/rest**, затем нажмите **Send**. Тело ответа будет содержать список ссылок возвращаемых контроллером **RestIndexEndpoint**.
7. В поле **Method** из выпадающего списка выберите **OPTIONS** и укажите **URL** **http://localhost:8080/students/rest/profile**. **ProfileRestController** ответит с заголовком **Status Code: 204 No Content** и **Allow: OPTIONS, HEAD, GET, POST**, который содержит действия разрешенные для коллекции ресурса.
8. В поле **Method** снова выберите **GET**, введите **http://localhost:8080/students/rest/profile** в **URL** и нажмите **Send**. Тело ответа должно иметь пустой список. Теперь попробуйте **URL** **http://localhost:8080/students/rest/**

profile/1, и веб-сервис должен ответить **Status Code: 404 Not Found**. Так как у Вас пока еще нет профилей студентов.

9. Для того чтобы исправить это, измените **URL** на **http://localhost:8080/students/rest/profile**, измените значение **Method** на **POST**. В поле **Body** введите значение **{«name»:»Ivan Ivanov»}**. После того как Вы отправите этот запрос, веб-сервис ответит **Status Code: 400 Bad Request**.
10. Перейдите на вкладку **Preview**, и тело ответа будет выглядеть следующим образом:

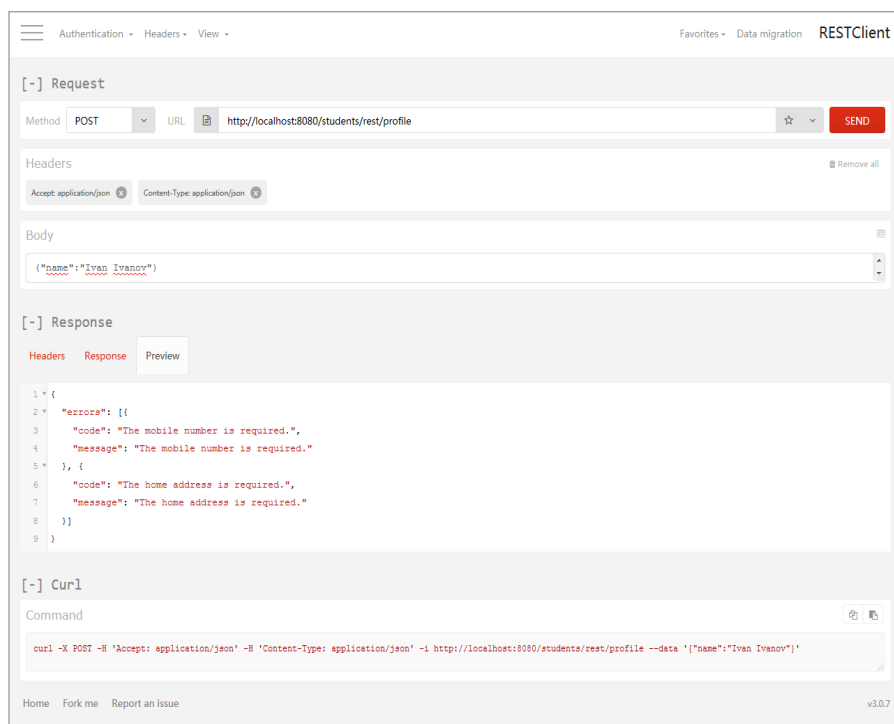


Рисунок 16.

Это означает, что валидация bean-компонента создала исключение `ConstraintViolationException`, а `RestExceptionHandler` преобразовал его в ответ JSON REST.

11. Прodelайте то же с XML-заголовками `Content-Type: application/xml` и `Асcept: application/xml`. В поле `Body` укажите следующее значение:

```
<?xml version="1.0" encoding="UTF-8"
  standalone="yes"?>
<profile><name>Ivan Ivanov</name></profile>
```

Вы получите тот же ответ, только отформатированный XML вместо JSON.

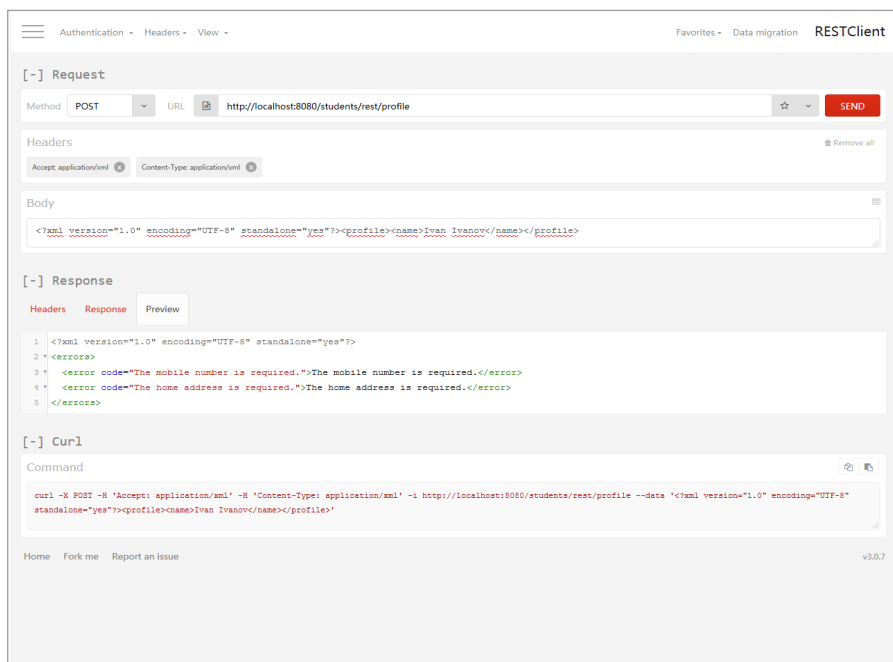


Рисунок 17.

12. На следующем рисунке показан успешный **POST** с полным блоком запроса **JSON** для создания профиля студента. Выполните этот **POST** и посмотрите заголовки ответов, где Вы увидите URL-адрес местоположения для только что созданного ресурса.

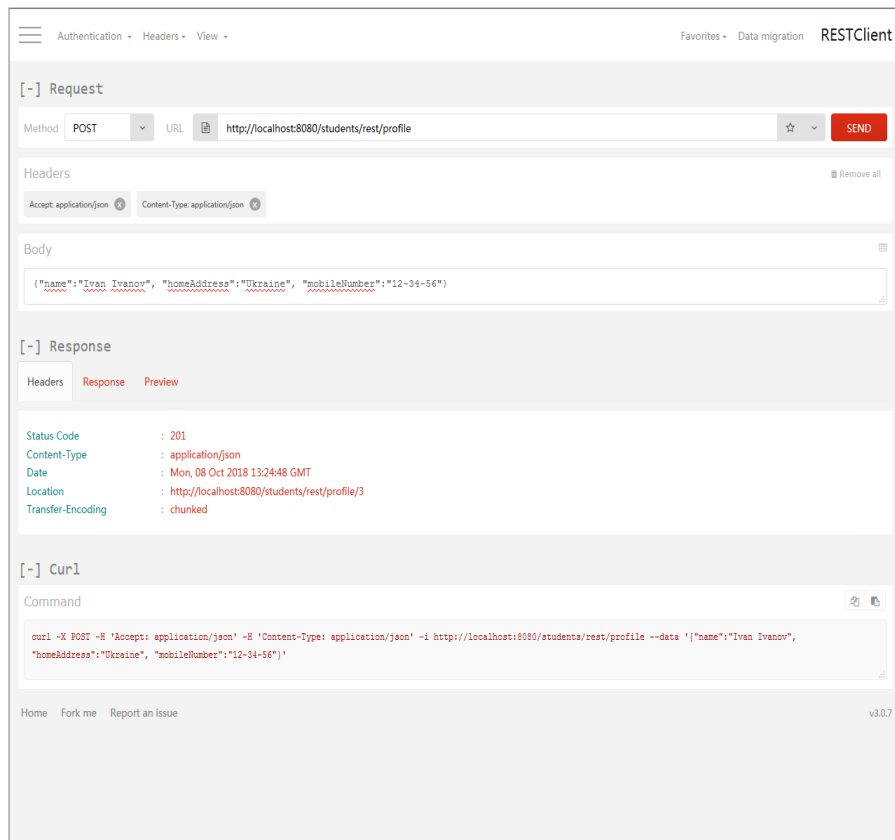


Рисунок 18.

13. Скопируйте этот **URL** и вставьте его в поле **URL**, очистите тело запроса и измените метод на **GET**. Ответом будет только что созданная Вами учетная запись.

14. Создайте еще несколько учетных записей, используя XML, а затем снова попробуйте метод **GET**. Вместо пустого списка Вы должны увидеть все созданные вами учетные записи. Это тестирование означает, что у Вас есть полностью функционирующий веб-сервис **RESTful**, способный отвечать как на запросы JSON, так и XML.

Использование Spring-сервисов для SOAP

Spring Web Services — это инструмент, который поможет Вам быстро создать простые службы SOAP. Больше о нем узнать можно, ознакомившись со справочной документацией проекта Spring Web Services по [адресу](#). Spring Web Services — это отдельный проект, связанный с Spring Framework и зависящий от него. Он предназначен для совместной работы с существующим кодом Spring, таким как веб-контроллеры и конечные точки RESTful.

Чтобы использовать Spring Web Services, Вам потребуется дополнительная зависимость Maven:

```
<dependency>
  <groupId>org.springframework.ws</groupId>
  <artifactId>spring-ws-core</artifactId>
  <version>2.1.4.RELEASE</version>
  <scope>compile</scope>
  <exclusions>
    <exclusion>
      <groupId>javax.xml.stream</groupId>
      <artifactId>stax-api</artifactId>
    </exclusion>
  </exclusions>
</dependency>
```

Переходная зависимость Java Streaming API для XML (StAX) исключена, поскольку она является частью API стандартной версии Java 6 и выше.

В этой части урока вернемся снова к проекту Support-Service. Здесь мы расширим приложение при помощи **Controler-Service-Repository**, а затем создадим конечную точку для управления обращениями через веб-службу SOAP.

Controler-Service-Repository

Сперва займемся **LoginController**. Создадим интерфейс **LoginService**:

```
public interface LoginService {  
    Principal authenticate(String username,  
                           String password);  
}
```

Класс **TemporaryLoginService** расширяет данный интерфейс:

```
@Service  
public class TemporaryLoginService implements  
        LoginService {  
  
    @Inject UserRepository userRepository;  
    @Override  
    public Principal authenticate(String username,  
                                  String password) {  
        String currentPassword =  
            this.userRepository.  
                getPasswordForUser(username);
```

```

        if(currentPassword == null) {
            return null;
        }

        if(!currentPassword.equals(password)) {
            return null;
        }
        return new UserPrincipal(username);
    }
}

```

Класс `TemporaryLoginService` содержит компонент `UserRepository`. Интерфейс `UserRepository` выглядит следующим образом:

```

public interface UserRepository {
    String getPasswordForUser(String username);
}

```

Этот интерфейс расширяет класс `InMemoryUserRepository`. В котором хранится временная база пользователей.

```

@Repository
public class InMemoryUserRepository implements
    UserRepository {
    private final Map<String, String> clientBase =
        new Hashtable<>();

    public InMemoryUserRepository() {
        this.clientBase.put("Ivanov", "qwert");
        this.clientBase.put("Petrov", "123456");
        this.clientBase.put("Sidorov", "master");
    }
}

```

```

@Override
public String getPasswordForUser(String username) {
    return this.clientBase.get(username);
}
}

```

Метод `getPasswordForUser` класса `TemporaryLogin-Service` возвращает новый объект `UserPrincipal`. Класс `UserPrincipal`:

```

public class UserPrincipal implements Principal,
                                   Cloneable, Serializable {
    private final String username;
    public UserPrincipal(String username) {
        this.username = username;
    }
    @Override
    public String getName() {
        return this.username;
    }
    @Override
    public int hashCode() {
        return this.username.hashCode();
    }
    @Override
    public boolean equals(Object other) {
        return other instanceof UserPrincipal &&
            ((UserPrincipal)other).username.
                equals(this.username);
    }
    @Override
    @SuppressWarnings("CloneDoesntDeclareCloneNot-
                        SupportedException")
    protected UserPrincipal clone() {
        try {

```

```

        return (UserPrincipal)super.clone();
    } catch (CloneNotSupportedException e) {
        throw new RuntimeException(e); // not
                                     // possible
    }
}
@Override
public String toString() {
    return this.username;
}
public static Principal getPrincipal(HttpSession
                                     session) {
    return session == null ? null :
        (Principal)session.getAttribute(
            "com.academy.user.principal");
}
public static void setPrincipal(HttpSession session,
                                Principal principal) {
    session.setAttribute(
        "com.academy.user.principal",
        principal);
}
}

```

Класс **LoginController** после изменений:

```

@Controller
public class LoginController {
    @Inject
    LoginService loginService;
    @RequestMapping("logout")
    public View logout(HttpServletRequest request,
                       HttpSession session) {
        session.invalidate();
        return new RedirectView("/login", true, false);
    }
}

```

```

@RequestMapping(value = "login",
                 method = RequestMethod.GET)
public ModelAndView login(Map<String, Object> model,
                        HttpSession session) {
    if(UserPrincipal.getPrincipal(session) != null)
        return this.getCardRedirect();
    model.put("loginFailed", false);
    model.put("loginForm", new LoginForm());
    return new ModelAndView("login");
}

@RequestMapping(value = "login",
                 method = RequestMethod.POST)
public ModelAndView login(Map<String, Object> model,
                        HttpSession session,
                        HttpServletRequest request,
                        LoginForm form,
                        Errors errors) {
    if(UserPrincipal.getPrincipal(session) != null)
        return this.getCardRedirect();
    if(errors.hasErrors()) {
        form.setPassword(null);
        return new ModelAndView("login");
    }
    Principal principal;
    try {
        principal = this.loginService.
            authenticate(
                form.getClientName(),
                form.getPassword()
            );
    }
    catch (ConstraintViolationException e) {
        form.setPassword(null);
        model.put("validationErrors",
                e.getConstraintViolations());
        return new ModelAndView("login");
    }
}

```

```

        if(principal == null) {
            form.setPassword(null);
            model.put("loginFailed", true);
            model.put("loginForm", form);
            return new ModelAndView("login");
        }
        UserPrincipal.setPrincipal(session, principal);
        request.changeSessionId();
        return this.getCardRedirect();
    }
    private ModelAndView getCardRedirect() {
        return new ModelAndView(new RedirectView(
            "/card/list", true, false));
    }
    public static class LoginForm {...}
}

```

Класс [LoginFilter](#) также имеет изменения:

```

public class LoginFilter implements Filter {
    @Override
    public void doFilter(ServletRequest request,
                        ServletResponse response,
                        FilterChain chain)
                        throws IOException,
                        ServletException {
        HttpSession session = ((HttpServletRequest)
                               request).getSession(false);
        final Principal principal = UserPrincipal.
            getPrincipal(session);
        if(principal == null) {
            ((HttpServletResponse) response).
                sendRedirect(
                    ((HttpServletRequest) request).
                        getContextPath() + "/login");
        }
    }
}

```

```

        else {
            chain.doFilter(
                new HttpServletRequestWrapper((
                    HttpServletRequest)request) {
                    @Override
                    public Principal getUserPrincipal() {
                        return principal;
                    }
                },
                response
            );
        }

    }

    @Override
    public void init(FilterConfig config)
        throws ServletException { }

    @Override
    public void destroy() { }
}

```

Теперь рассмотрим что необходимо изменить касательно обращений.

Добавим интерфейсы **CardService** и **CardRepository**:

```

public interface CardService {
    List<Card> getAllCards();
    Card getCard(long id);
    void save(Card card);
    void deleteCard(long id);
}

public interface CardRepository {
    List<Card> getAll();
    Card get(long id);
}

```

```

void add(Card card);
void update(Card card);
void delete(long id);
}

```

Данные интерфейсы будут расширять классы **DefaultCardService** и **InMemoryCardRepository** соответственно.

```

@Service
public class DefaultCardService implements CardService {
    @Inject
    CardRepository cardRepository;
    @Override
    public List<Card> getAllCards() {
        return this.cardRepository.getAll();
    }
    @Override
    public Card getCard(long id) {
        return this.cardRepository.get(id);
    }
    @Override
    public void save(Card card) {
        if(card.getId() < 1) {
            this.cardRepository.add(card);
        }
        else this.cardRepository.update(card);
    }
    @Override
    public void deleteCard(long id) {
        this.cardRepository.delete(id);
    }
}

@Repository
public class InMemoryCardRepository
    implements CardRepository {

```

```

private volatile long CARD_ID = 1L;
private final Map<Long, Card> cardBase =
    new LinkedHashMap<>();

@Override
public List<Card> getAll() {
    return new ArrayList<>(this.cardBase.values());
}

@Override
public Card get(long id) {
    return this.cardBase.get(id);
}

@Override
public void add(Card card) {
    card.setId(this.getNextCardId());
    this.cardBase.put(card.getId(), card);
}

@Override
public void update(Card card) {
    this.cardBase.put(card.getId(), card);
}

@Override
public void delete(long id) {
    this.cardBase.remove(id);
}

private synchronized long getNextCardId() {
    return this.CARD_ID++;
}
}

```

Внесем изменения в класс **CardController**:

```

@Controller
@RequestMapping("card")
public class CardController {
    @Inject CardService cardService;
}

```

```

@RequestMapping(value = {"", "list"},
                 method = RequestMethod.GET)
public String list(Map<String, Object> model) {
    model.put("cards",
             this.cardService.getAllCards());
    return "card/list";
}

@RequestMapping(value = "view/{cardId}",
                 method = RequestMethod.GET)
public ModelAndView view(Map<String, Object> model,
                        @PathVariable("cardId") long cardId) {
    Card card = this.cardService.getCard(cardId);
    if(card == null)
        return this.getListRedirectModelAndView();
    model.put("cardId", Long.toString(cardId));
    model.put("card", card);
    return new ModelAndView("card/view");
}

@RequestMapping(
    value =("/{cardId}/attachment/
           {attachment:..+}",
    method = RequestMethod.GET
)
public View download(@PathVariable("cardId")
                    long cardId,
                    @PathVariable("attachment")
                    String name) {
    Card card = this.cardService.getCard(cardId);
    if(card == null)
        return this.getListRedirectView();
    Attachment attachment =
        card.getAttachment(name);
    if(attachment == null) {
        return this.getListRedirectView();
    }
    return new DownloadingView(attachment.
                               getFileName(),

```

```

        attachment.getFileMimeType(),
        attachment.getFileContents());
    }
    @RequestMapping(value = "create",
        method = RequestMethod.GET)
    public String create(Map<String, Object> model) {
        model.put("cardForm", new CardForm());
        return "card/add";
    }
    @RequestMapping(value = "create",
        method = RequestMethod.POST)
    public ModelAndView create(Principal principal,
        CardForm form, Errors errors,
        Map<String, Object> model)
        throws IOException {
        if(errors.hasErrors())
            return new ModelAndView("card/add");
        Card card = new Card();
        card.setClientName(principal.getName());
        card.setTopic(form.getTopic());
        card.setMessage(form.getMessage());
        for(MultipartFile filePart :
            form.getAttachments()) {
            Attachment attachment = new Attachment();
            attachment.setFileName(filePart.
                getOriginalFilename());
            attachment.setFileMimeType(filePart.
                getContentType());
            attachment.setFileContents(filePart.
                getBytes());
            if((attachment.getFileName() != null &&
                attachment.getFileName().
                    length() > 0) ||
                (attachment.getFileContents() !=
                    null &&
                    attachment.getFileContents().
                        length > 0))
                card.addAttachment(attachment);
        }
    }
}

```

```

    }
    try {
        this.cardService.save(card);
    }
    catch (ConstraintViolationException e) {
        model.put("validationErrors",
            e.getConstraintViolations());
        return new ModelAndView("card/add");
    }
    return new ModelAndView(new RedirectView(
        "/card/view/" +
        card.getId(), true, false
    ));
}
private ModelAndView getListRedirectModelAndView() {
    return new ModelAndView(this.
        getListRedirectView());
}
private View getListRedirectView() {
    return new RedirectView("/card/list", true, false);
}
public static class CardForm { ... }
}

```

Написание контракта — первый XSD и WSDL

Spring Web Services обеспечивает **contract-first** разработку. Необходимо определить свой контракт XML, прежде чем сделать что-либо еще. Сначала необходимо определить то, как должно выглядеть обращение в формате XML.

```

<card xmlns="http://academy.com/xmlns/service">
<id>8</id>
<clientName>Ivan Ivanov</clientName>
<topic>some topic</subject>

```

```

<message>some message</message>
<attachment>
  <name>New Document.png</name>
  <mimeType>image/png</mimeType>
  <contents>
    R0lGODlhAQABAAAAABAAEAAAICTAEAOwAACH5BAEKAAEALAAAA==
  </contents>
</attachment>
</card>

```

Для того чтобы создавать новые обращения необходимо элемент `<cardForm>`.

```

<cardForm xmlns="http://academy.com/xmlns/service">
  <topic>some topic</subject>
  <message>some message</message>
  <attachment>
    <fileName>New Document.png</fileName>
    <fileMimeType>image/png</fileMimeType>
    <fileContents>R0lGODlhAQABAAAAABAAEAAAI-
      CTAEAOwAACH5BAEKAAEALAAAA==
    </fileContents>
  </attachment>
</cardForm>

```

Для того чтобы прочитать и удалить отдельный билет необходим элемент `<cardRequest>` и идентичный элемент `<deleteCard>`. Наконец, нам нужно выбрать все билеты, поэтому пустой элемент `<cardRequest>` должен сделать трюк.

```

<cardRequest xmlns="http://academy.com/xmlns/service">
  <id>8</id>
</cardRequest>

```

Учитывая эти примеры, IDE может помочь сгенерировать код схемы XSD. Например, в [IntelliJ IDEA](#) можно создавать временные XML-файлы с каждым элементом, выполнив следующие шаги:

1. Щелкните правой кнопкой мыши в любом месте содержимого XML и выберите [Generate XSD Schema from XML File](#) (создать схему XSD из файла XML).
2. В появившемся диалоговом окне убедитесь, что для поля [Design Type](#) заданы [local elements/global complex types](#), а затем нажмите [OK](#).
3. Сгенерированный файл схемы XSD довольно близок к тому, что Вы хотите, и требует лишь незначительных изменений, указания типов ожидаемых элементов, определения того, как часто могут появляться типы и т.д. для всех действий в Вашем веб-сервисе все файлы XSD необходимо объединить в один файл XSD.

Результирующий файл схемы XSD */WEB-INF/xsd/soap/support.xsd* выглядит следующим образом:

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema attributeFormDefault="unqualified"
            elementFormDefault="qualified"
            targetNamespace="http://academy.com/xmlns/
                           service"
            xmlns:xs="http://www.w3.org/2001/
                           XMLSchema"
            xmlns:service="http://academy.com/xmlns/
                           service">
  <xs:element name="cardsRequest"
              type="service:cardsRequestType" />
```

```

<xs:element name="cardRequest"
            type="service:selectCardType" />
<xs:element name="createCard"
            type="service:createCardType"/>
<xs:element name="deleteCard"
            type="service:selectCardType"/>
<xs:element name="card"
            type="service:cardType" />
<xs:element name="cards"
            type="service:cardsType" />

<xs:complexType name="cardType">
  <xs:sequence>
    <xs:element type="xs:long"
                name="id" minOccurs="0"/>
    <xs:element type="xs:string"
                name="clientName"
                minOccurs="0"/>
    <xs:element type="xs:string"
                name="topic"/>
    <xs:element type="xs:string"
                name="message"/>
    <xs:element type="service:attachmentType"
                name="attachment"
                minOccurs="0"
                maxOccurs="unbounded"/>
  </xs:sequence>
</xs:complexType>

<xs:complexType name="cardsRequestType">
  <xs:sequence />
</xs:complexType>

<xs:complexType name="selectCardType">
  <xs:sequence>
    <xs:element type="xs:long" name="id"/>
  </xs:sequence>

```

```

</xs:complexType>

<xs:complexType name="createCardType">
  <xs:sequence>
    <xs:element type="service:cardType"
      name="card"/>
  </xs:sequence>
</xs:complexType>

<xs:complexType name="cardsType">
  <xs:sequence>
    <xs:element type="service:cardType"
      name="card"
      minOccurs="0"
      maxOccurs="unbounded"/>
  </xs:sequence>
</xs:complexType>

<xs:complexType name="attachmentType">
  <xs:sequence>
    <xs:element type="xs:string"
      name="fileName"/>
    <xs:element type="xs:string"
      name="fileMimeType"/>
    <xs:element type="xs:base64Binary"
      name="fileContents"/>
  </xs:sequence>
</xs:complexType>

</xs:schema>

```

XSD — это все, что нам нужно в этом случае: Spring Web Services считает что этого достаточно для контракта и автоматически создает WSDL. Чтобы произвести создание WSDL, необходимо сначала настроить Spring Web Services.

В классах `Attachment` и `Card` добавлены аннотации

```
@XmlElement(name = "attachment")
public class Attachment {
    ...
    @XmlSchemaType(name = "base64Binary")
    public byte[] getFileContents() {...}
    ...
}

@XmlRootElement(namespace = "http://academy.com/
                xmlns/service",
                name = "card")
public class Card {
    ...
    @XmlElement(name = "attachment")
    public Collection<Attachment> getAttachments() {...}
    ...
}
```

Добавление конфигурации SOAP-диспетчера

Создадим аннотацию `@com.academy.config.annotation.WebController`:

```
@Target({ ElementType.TYPE })
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Controller
public @interface WebController {
    String value() default "";
}
```

Она указывает, что целевой компонент является контроллером, подходящим для сопоставления запросов.

Переименуем класс `ServletContextConfiguration` в `WebServletContextConfiguration`. В коде этого класса изменились типы стереотипных аннотаций, которые маркируют компоненты, принадлежащие этому контексту приложения.

```
@Configuration
@EnableWebMvc
@ComponentScan(
    basePackages = "com.academy.site",
    useDefaultFilters = false,
    includeFilters = @ComponentScan.
                        Filter(WebController.class)
)
public class WebServletContextConfiguration extends
    WebMvcConfigurerAdapter {
    ...
}
```

В отличие от Spring Framework, Spring Web Services не поддерживает программную конфигурацию на Java. Тем не менее, требуется сканирование компонентов для контекста приложения SOAP, поэтому проще всего создать гибридную конфигурацию. `SoapServletContextConfiguration` проверяет параметры `@org.springframework.ws.server.endpoint.annotation.Endpoint` и импортирует XML-конфигурацию `soapServletContext.xml`.

```
@Configuration
@ComponentScan(
    basePackages = "com.academy.site",
    useDefaultFilters = false,
    includeFilters = @ComponentScan.
                        Filter(Endpoint.class)
)
```

```

@ImportResource("classpath:com/academy/config/
                soapServletContext.xml")
public class SoapServletContextConfiguration {
    @Bean
    public WebServiceMessageFactory messageFactory() {
        SaaJSoapMessageFactory factory =
            new SaaJSoapMessageFactory();
        factory.setSoapVersion(SoapVersion.SOAP_12);
        return factory;
    }
}

```

Явный компонент `messageFactory` переопределяет фабрику сообщений по умолчанию, так что поддерживаемая версия SOAP 1.2 вместо стандартного 1.1. Файл *soapServletContext.xml* сообщает Spring Web Services настраивать конечные точки на основе аннотаций и использовать ранее созданный файл схемы XSD для создания WSDL.

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
        xmlns:sws="http://www.springframework.org/
            schema/web-services"
        xmlns:xsi="http://www.w3.org/2001/
            XMLSchema-instance"
        xsi:schemaLocation="http://www.springframework.
            org/schema/beans
            http://www.springframework.
            org/schema/beans/spring-
            beans-4.0.xsd
            http://www.springframework.
            org/schema/web-services
            http://www.springframework.
            org/schema/web-services/
            web-services-2.0.xsd">

```

```

<sws:annotation-driven marshaller="jaxb2Marshaller"
                        unmarshaller="jaxb2Marshaller" />
<sws:dynamic-wsdl id= "service"
                  portTypeName="Service"
                  locationUri="/Soap/"
                  createSoap11Binding="false"
                  createSoap12Binding="true"
                  targetNamespace="http://
                                academy.com/xmlns/service">
    <sws:xsd location="/WEB-INF/xsd/soap/
                service.xsd" />
</sws:dynamic-wsdl>
</beans>

```

Так как [@Endpoint](#) также является [@Component](#), необходимо изменить [RootContextConfiguration](#), чтобы гарантировать, что только контекст сервлета SOAP-диспетчера создает экземпляры и управляет классами конечных точек SOAP.

```

...
@ComponentScan (
    basePackages = "com.academy.site",
    excludeFilters = @ComponentScan.Filter({
        Controller.class,
        ControllerAdvice.class,
        Endpoint.class
    })
)
public class RootContextConfiguration
...

```

Необходимо загрузить диспетчер SOAP Servlet, чтобы он использовал [SoapServletContextConfiguration](#). Spring Web

Services имеет специальную реализацию `DispatcherServlet` `org.springframework.ws.transport.http.MessageDispatcherServlet`, которая обрабатывает запросы SOAP. Следующий код из класса `Bootstrap` создает этот сервлет, устанавливает его контекст приложения и инструктирует его преобразовывать местоположения WSDL в запросы.

```
...
AnnotationConfigWebApplicationContext webContext =
    new AnnotationConfigWebApplicationContext();
webContext.register(WebServletContextConfiguration.
    class);
ServletRegistration.Dynamic dispatcher =
    container.addServlet(
        "springWebDispatcher",
        new DispatcherServlet(webContext)
    );

dispatcher.setLoadOnStartup(1);
dispatcher.setMultipartConfig(new MultipartConfigElement(
    null, 20_971_520L, 41_943_040L, 512_000
));

dispatcher.addMapping("/");
AnnotationConfigWebApplicationContext soapContext =
    new AnnotationConfigWebApplicationContext();
soapContext.register(SoapServletContextConfiguration.
    class);
MessageDispatcherServlet soapServlet =
    new MessageDispatcherServlet(soapContext);
soapServlet.setTransformWsdLocations(true);
dispatcher = container.
    addServlet("springSoapDispatcher",
        soapServlet);
dispatcher.setLoadOnStartup(3);
dispatcher.addMapping("/Soap/*");
```

```

FilterRegistration.Dynamic registration =
    container.addFilter(
        "loginFilter", new LoginFilter()
    );
registration.addMappingForUrlPatterns(
    null, false, "/card", "/card/*"
);
...

```

После того как Spring Web Services настроен, перейдем к созданию конечной точки SOAP.

Создание конечной точки SOAP

Конечные точки являются ключевой концепцией поддержки сервера Spring Web Service. В отличие от контроллеров, методы обработчиков которых напрямую привязаны к HTTP-запросам и ответам, конечные точки Spring SOAP могут обслуживать запросы SOAP, выполненные через HTTP, XMPP, SMTP, JMS и прочее. Подобно тому, как `@Controller` помечает контроллер, у которого должны проверяться и сопоставляться с запросами методы `@RequestMapping`, `@Endpoint` помечает конечную точку у которой обработчиками для входящих запросов SOAP по любому протоколу являются методы `@org.springframework.ws.server.endpoint.annotation.PayloadRoot`, `@org.springframework.ws.soap.server.endpoint.annotation.SoapAction` и/или `@org.springframework.ws.soap.addressing.server.annotation.Action`. Параметры методов конечных точек соответствуют элементам запроса, тогда как возвращаемые типы указывают на содержимое ответа.

Что касается параметров метода, у Вас есть большая гибкость в отношении того, какие типы контента Вы хотите принять, в зависимости от того, как Вы предпочитаете получать доступ к содержимому XML в запросах. Например, параметр метода типа `SoapMessage`, `SoapBody`, `SoapEnvelope` или `SoapHeader` (в пакете `org.springframework.ws.soap`) передает соответствующий контент из запроса SOAP. Аналогично `@RequestBody`, если параметр метода аннотируется `@org.springframework.ws.server.endpoint.annotation.RequestPayload`, Spring Web Services преобразует полезную нагрузку запроса в указанный тип. Поддерживаемые типы полезной нагрузки запроса:

- `javax.xml.transform.Source` или его подинтерфейсы;
- `org.w3c.dom.Element`;
- `org.dom4j.Element`, если `dom4j` находится в пути к классам;
- `org.jdom.Element`, если JDOM находится в пути к классам;
- `nu.xom.Element`, если XOM находится в пути к классам;
- `javax.xml.stream.XMLStreamReader` или `javax.xml.stream.XMLEventReader` в Java SE 6 или выше, или если StAX находится в пути к классам;
- Любой тип аннотированный с помощью `@ javax.xml.bind.annotation.XmlRootElement` в Java 6 или более поздней версии или если JAXB находится в пути к классам;
- Любой тип, поддерживаемый Spring OXM Unmarshaller, если `<sws:annotation-driven>` настроен с атрибутом `unmarshaller`.

Аналогично, методы, аннотированные `@org.springframework.ws.server.endpoint.annotation.ResponsePayload` могут иметь возвращаемые типы `Source` или его под-интерфейсы, `org.w3c.dom.Element`, `org.dom4j.Element`, `org.jdom.Element`, `nu.xom.Element`, аннотированный объект `@XmlRootElement` любого типа и любой тип, поддерживаемый `Spring OXM Marshaller` (если `<sws:annotation-driven>` настроен с атрибутом `marshaller`). Метод, возвращающий `void`, не имеет содержимого ответа (пустой SOAP-конверт).

Из-за этого метода и поддержки возвращаемого типа значения можно повторно использовать несколько типов параметров и возвращаемых данных, созданных для `CardSoapEndpoint`. Эта конечная точка имеет много общего с конечной точкой REST, но она также имеет несколько отличий. Она использует имена корневых элементов полезных данных для соответствия методам действий с конечной точкой, используя аннотацию `@PayloadRoot`, но вместо этого можно использовать заголовок SOAPAction с аннотацией `@SoapAction` или стандартом адресации веб-сервисов с аннотацией `@Action`.

```
@Endpoint
public class CardSoapEndpoint {
    private static final String NAMESPACE =
        "http://academy.com/xmlns/service";

    @Inject
    CardService cardService;

    @PayloadRoot(namespace = NAMESPACE,
        localPart = "cardsRequest")
    @ResponsePayload
```

```

public CardWebServiceList read() {
    CardWebServiceList list =
        new CardWebServiceList();
    list.setValue(this.cardService.getAllCards());
    return list;
}
@PayloadRoot(namespace = NAMESPACE,
               localPart = "cardRequest")
@Namespace(uri = NAMESPACE, prefix = "s")
@ResponsePayload
public Card read(@XPathParam("/s:cardRequest/id")
                 long id) {
    return this.cardService.getCard(id);
}
@PayloadRoot(namespace = NAMESPACE,
               localPart = "createCard")
@ResponsePayload
public Card create(@RequestPayload CreateCard
                  form) {
    Card card = new Card();
    card.setClientName("WebServiceAnonymous");
    card.setTopic(form.getTopic());
    card.setMessage(form.getMessage());
    if(form.getAttachments() != null)
        card.setAttachments(form.getAttachments());
    this.cardService.save(card);
    return card;
}
@PayloadRoot(namespace = NAMESPACE,
               localPart = "deleteCard")
@Namespace(uri = NAMESPACE, prefix = "s")
public void delete(@XPathParam("/s:deleteCard/id")
                  long id) {
    this.cardService.deleteCard(id);
}
@XmlRootElement(namespace = NAMESPACE,
                 name = "createCard")

```

```

public static class CreateCard {
    private String topic;
    private String message;
    private List<Attachment> attachments;
    public String getTopic() {
        return topic;
    }
    public void setTopic(String topic) {
        this.topic = topic;
    }
    public String getMessage() {
        return message;
    }
    public void setMessage(String message) {
        this.message = message;
    }
    @XmlElement(name = "attachment")
    public List<Attachment> getAttachments() {
        return attachments;
    }
    public void setAttachments(List<Attachment>
                                attachments) {
        this.attachments = attachments;
    }
}

@XmlRootElement(name = "cards")
public class CardWebServiceList {
    private List<Card> value;
    @XmlElement(name = "card")
    public List<Card> getValue() {
        return this.value;
    }
    public void setValue(List<Card> value) {
        this.value = value;
    }
}

```

Скомпилируйте и запустите приложение поддержки клиентов и проверьте его. Перейдите в <http://localhost:8080/service/Soap/service.wsdl> в браузере, чтобы просмотреть автоматически созданный WSDL. Создайте несколько обращений используя с графический интерфейс пользователя. Для проверки интерфейса веб-службы SOAP можно использовать RESTClient. Необходимо использовать заголовки запросов **Accept** и **Content-Type** со значением **application/soap+xml**. Поскольку запросы конверта SOAP могут быть сложными для создания вручную, сформированы четыре следующих запроса, которые можно использовать для проверки службы. Запросы следует направлять по адресу <http://localhost:8080/service/Soap/> с методом POST независимо от действия.

```
<soap:Envelope xmlns:soap="http://www.w3.org/2003/05/soap-envelope">
  <soap:Header/>
  <soap:Body>
    <service:cardsRequest xmlns:service="http://academy.com/xmlns/service"/>

  </soap:Body>
</soap:Envelope>
<soap:Envelope xmlns:soap="http://www.w3.org/2003/05/soap-envelope">
  <soap:Header/>
  <soap:Body>
    <service:cardRequest xmlns:service="http://academy.com/xmlns/service">

      <id>1</id>
    </service:cardRequest>
  </soap:Body>
</soap:Envelope>
```

```

<soap:Envelope xmlns:soap="http://www.w3.org/2003/05/
    soap-envelope">
  <soap:Header/>
  <soap:Body>
    <service:createCard xmlns:service="http://academy.
        com/xmlns/service">
      <topic>some topic</topic>
      <message>some message</message>
      <attachment>
        <fileName>New file.png</fileName>
        <fileMimeTypeContentType>image/png
        </fileMimeTypeContentType>
        <fileContents>R0lGODlhAQABAAAAACH5BAEICTAEAOw
        </fileContents>
      </attachment>
    </service:createCard>
  </soap:Body>
</soap:Envelope>

<soap:Envelope xmlns:soap="http://www.w3.org/2003/05/
    soap-envelope">
  <soap:Header/>
  <soap:Body>
    <service:deleteCard xmlns:service="http://academy.
        com/xmlns/service">
      <id>1</id>
    </service:deleteCard>
  </soap:Body>
</soap:Envelope>

```

Spring Boot

Что такое Spring Boot? Цели и задачи Spring Boot

Постоянное развитие Spring привело к созданию проекта Spring Boot, который вызвал огромный положительный успех для уже очень популярной структуры.

В целом Spring Boot — это проект, который построен в довершение Spring Framework. Он обеспечивает гораздо более простой и быстрый способ установки, настройки и запуска как простых, так и веб-приложений.

В основной структуре Spring нам необходимо все настраивать самостоятельно. Следовательно, проект может содержать множество конфигурационных файлов, таких как дескрипторы XML. Это одна из основных проблем, которую решает Spring Boot.

Он умело составляет зависимости, автоматически настраивает все функции, которые необходимо использовать, и можно запустить приложение одним щелчком мыши. Кроме того, он также упрощает процесс развертывания приложения.

Рассмотрим ключевые характеристики Spring Boot, делающие его уникальным и простым в использовании:

- **Автоконфигурация:** настраивает приложение на основе окружающей среды, а также рекомендаций, предоставленных разработчиками.
- **Автономность:** буквально полностью автономный. Следовательно, нам не нужно разворачивать прило-

жение на веб-сервере или в какой-либо специальной среде. Наша единственная задача — щелкнуть по кнопке или выдать команду `run`.

- **Своевольность:** выбирает способы по умолчанию.

Попытка автоматической настройки приложения происходит на основе того, какие зависимости Вы добавили. Давайте посмотрим пример в соответствии с компонентом базы данных.

Если Вы добавите зависимость к `rom.xml`, которая относится к базе данных, предполагается, что Вы, вероятно, хотели бы использовать базу данных. Следовательно, Spring Boot автоматически настраивает Ваше приложение для доступа к базе данных.

Кроме того, если зависимость обозначается для очень конкретной базы данных, например Oracle или MySQL. Он может сделать более определенное предположение и, вероятно, настроит эту конкретную базу данных так, как Вам нужно.

Spring Boot позволяет легко создавать автономные, производственные Spring приложения, которые Вы можете запустить.

Изначально можно подумать, что запуск Java приложения очень прост. Отдайте команду запуска, и все работает. Честно говоря, это не так просто.

Процесс запуска веб-приложения на основе Java:

- Прежде всего, необходимо упаковать свое приложение.
- Выбрать, какой тип веб-серверов необходимо использовать и загружать.

- Настроить этот конкретный веб-сервер.
 - После этого необходимо организовать процесс развертывания и запустить веб-сервер.
- При загрузке Spring Вам потребуется:
- Пакет приложения.
 - Запустить приложение с помощью некоторой простой команды.

Spring Boot заботится об остальном, от запуска и настройки встроенного веб-сервера до развертывания на нем приложения.

Принимает взвешенные решения при создании готовых Spring приложений.

При написании Java-приложений, у Вас есть множество вариантов, начиная с определения веб-страницы, ведения журнала, структуры коллекции, используемого инструмента построения.

Несмотря на это, в большинстве случаев разработчики используют одни и те же самые популярные библиотеки. Все, что делает Spring Boot — загружается и настраивает их наиболее стандартным образом. Поэтому разработчикам не нужно тратить много времени, чтобы снова и снова настраивать одно и то же.

Spring Boot поддерживает соглашение по настройке и призван поднять и запустить приложение как можно быстрее.

Следовательно, у разработчиков больше времени для написания кода и удовлетворения потребностей бизнеса. Например, посмотрите <https://start.spring.io/>, где можно

легко указать свои потребности и просто загрузить соответствующее готовое приложение.

Выше приведенный пример — отличный пример того, как можно воспользоваться преимуществом Spring Boot, и начать свой проект как можно быстрее.

Примеры использования Spring Boot

Создадим простое веб-приложение, имеющее только один API, который возвращает текст «[Hello World](#)». Это приложение создается с использованием Spring Boot и IntelliJ IDEA Spring Initializr.

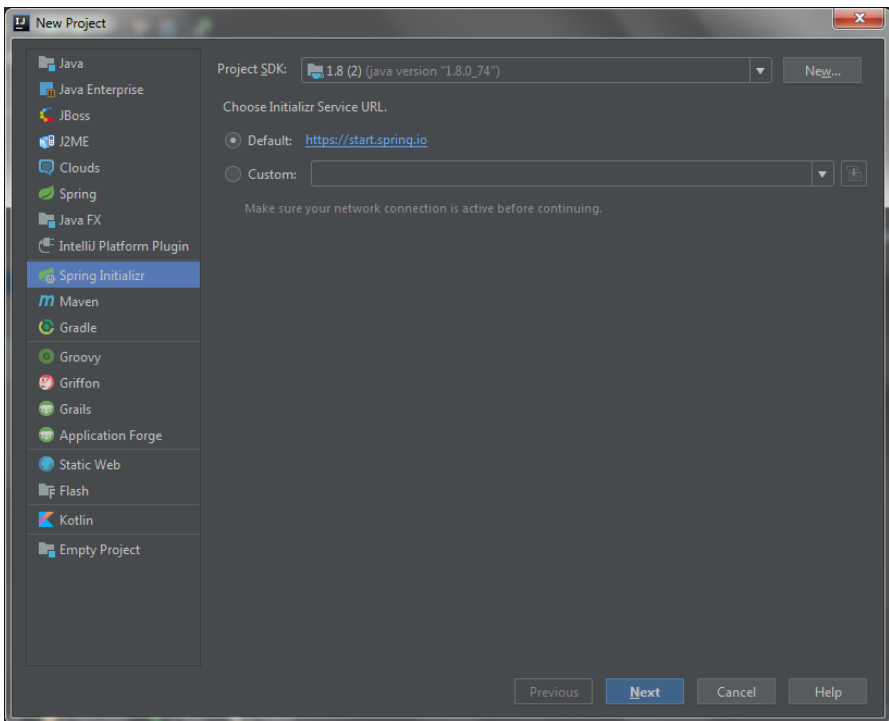


Рисунок 19.

При создании нового проекта на левой панели необходимо выбрать Spring Initializr и указать версию SDK, затем нажать **Next** (рис. 18).

В следующем окне определим значения группы и артефакта проекта, а также укажем имя и описание (рис. 19).

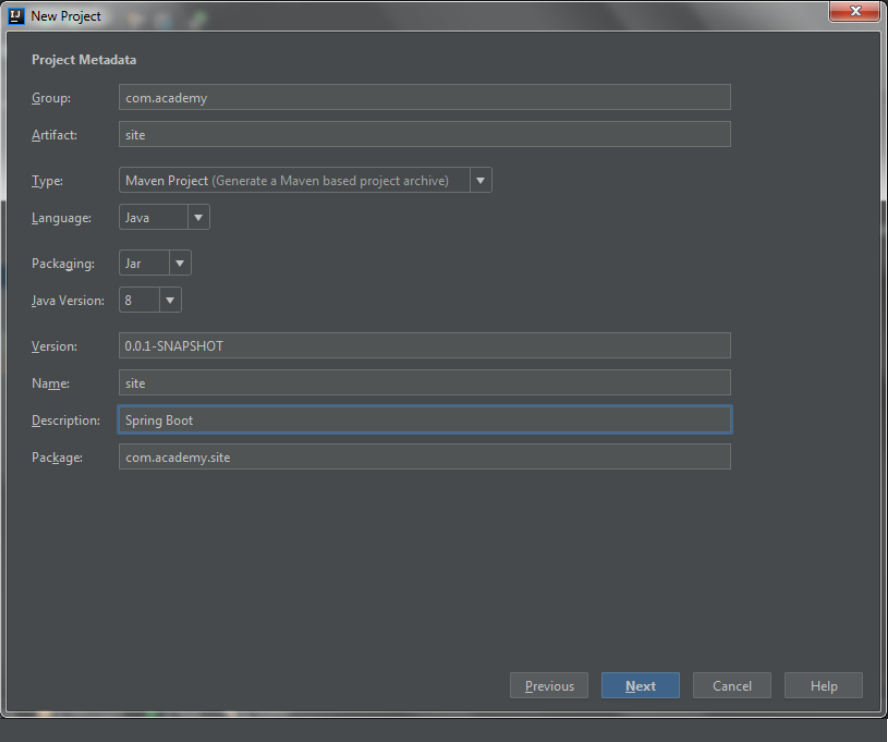


Рисунок 20.

После нажатия **Next** откроется новое окно, и здесь можно выбрать свои зависимости. Для этого проекта просто выберем **Web**. Теперь можно переходить далее и создать проект (рис. 20).

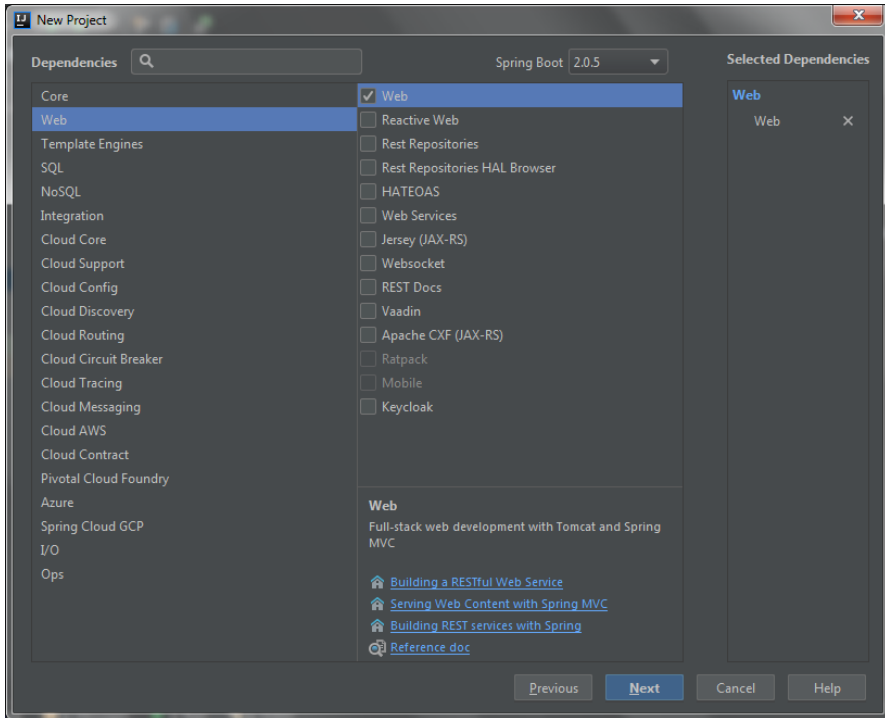


Рисунок 21.

Здесь Вы увидите, что вся структура проекта уже создана, а также создан основной класс приложения.

```
@SpringBootApplication
public class SiteApplication {
    public static void main(String[] args) {
        SpringApplication.run(SiteApplication.class,
                               args);
    }
}
```

Вы можете запустить приложение и перейти по ссылке *http://localhost:8080*, но увидите сообщение об ошибке,

потому что пока нет сервиса. Теперь создадим новый класс с именем **HelloController**.

```
@RestController
public class HelloController {
    @RequestMapping("/hello")
    public String sayHello(@RequestParam(value = "name")
                           String name){
        return "Hello " + name + ".";
    }
}
```

Необходимо определить этот класс как **RestController**, поэтому добавим аннотацию **@RestController**. Затем добавим новый метод обработки запроса **sayHello** и вернем ответ.

@RequestMapping сопоставляет наш метод с определенным путем «/hello» и содержит один параметр запроса **name**. Теперь запустите приложение по адресу *http://localhost:8080/hello?name=World* и убедитесь что приложение Spring Boot работает.

Домашнее задание

Доработать приложение «Интернет-магазин» из домашнего задания предыдущего урока используя Spring Framework. Создать и настроить конфигурацию Spring Framework при помощи XML или Java. Применить паттерны проектирования **Model-View-Controller** и **Controler-Service-Repository**. Разделить приложение на три слоя: **repository**, **service**, **web**. Протестировать работоспособность приложения.

Создать веб-сервисы REST и SOAP для приложения. Для веб-сервиса SOAP сгенерировать и доработать код схемы XSD. Протестировать приложение при помощи RESTClient. Проверить корректность методов: **OPTIONS**, **GET**, **POST** как формате XML, так и JSON.